JOHANNES GUTENBERG UNIVERSITÄT MAINZ

JG|U

Optimization of Explicit Runge-Kutta Methods for Multiscale Problems

Master's thesis submitted on August 4th, 2026 by

Christian Bartash

to Faculty 08: Physics, Mathematics and Computer Science
Johannes Gutenberg University Mainz.

First examiner: Univ.-Prof. Dr. Hendrik Ranocha
Second examiner: Prof. Dr. Julian Koellermeier

Contents

Abstract **4**

Zusammenfassung **5**

1 Introduction **6**

2 Projective Integration **9**
2.1 Runge-Kutta schemes 9
2.2 Projective Forward Euler 10
2.2.1 Stability of PFE and the choice of K 11
2.3 Projective Runge–Kutta 13

3 Optimization of stability polynomials for Runge-Kutta methods **16**
3.1 Stability regions of Runge-Kutta methods 16
3.1.1 Stability optimization problem 17
3.1.2 Reformulation as a least-deviation problem 17
3.1.3 Low-storage Runge-Kutta methods 18
3.1.4 Construction of Runge-Kutta methods 21
3.1.5 Choice of polynomial basis 22
3.1.6 Chebyshev interpolation 24
3.1.7 Numerical implementation 28

4 Numerical experiments **30**
4.1 Isolated two-scale spectrum 30
4.1.1 Projective Integration methods 32
4.1.2 Optimized Runge-Kutta methods 34
4.1.3 Additional grid-based line search 37
4.1.4 Construction of Runge-Kutta methods with RK-Opt 38
4.1.5 Comparison with Projective Integration methods 39
4.2 Definition of the two-scale cluster spectrum 41
4.2.1 Projective Integration methods 43
4.3 Two-scale cluster spectrum with fast radius $r_{\text{fast}} = 5$ 44
4.3.1 Projective Integration methods 44
4.3.2 Optimized Runge-Kutta methods 46
4.3.3 Additional grid-based line search 48
4.3.4 Construction of Runge-Kutta methods with RK-Opt 49
4.3.5 Comparison with Projective Integration methods 49

4.4 Two-scale cluster spectrum with fast radius $r_{\text{fast}} = 50$ 53
4.4.1 Projective Integration methods 53
4.4.2 Optimized Runge-Kutta methods 55
4.4.3 $3S^*$ methods with the Chebyshev basis 55
4.4.4 $2N$ methods 59
4.4.5 Chebyshev interpolation 61
4.4.6 Repeated RK-Opt runs 63
4.4.7 Low-storage evaluation in RK-Opt 65
4.4.8 Comparison with Projective Integration methods 66
4.5 Semi-discrete moment model test problem 69
4.5.1 Results for $\varepsilon = 10^{-3}$ 71
4.5.2 Results for $\varepsilon = 10^{-4}$ 75
4.6 Generalization of the two-scale system 79
4.6.1 Projective Integration methods 80
4.6.2 Optimized Runge-Kutta methods 81
4.6.3 Comparison with Projective Integration methods 84

5 Conclusion and outlook **88**

6 Appendix **90**
6.1 Flux Jacobian of the hyperbolic moment model 90
6.2 Butcher tableau of the PRK4 method 90
6.3 Use of artificial intelligence (Nutzung KI-Tools) 91
6.4 GitHub repository 91

Abstract

This thesis compares two explicit approaches for the numerical solution of stiff differential equations with separate slow and fast time scales. In Projective Integration, several small inner time steps are first used to damp the fast components. Afterwards, a larger outer step is taken. The second approach uses explicit Runge-Kutta methods whose stability polynomials are adapted to the spectrum of the given problem. The methods are compared using the maximal stable step size, the step size per right-hand side evaluation, and the numerical errors.

For the optimized Runge-Kutta methods, a stability polynomial is first computed with PolyOpt. Afterwards, RK-Opt is used to construct a corresponding low-storage Runge-Kutta method. The methods are studied for an isolated two-scale spectrum, spectral clusters with different radii, a semi-discrete hyperbolic moment model, and a generalized two-scale system.

The results show that the comparison strongly depends on the spectrum and the order of the methods. For the isolated spectrum and the smaller spectral clusters, optimized Runge-Kutta methods often achieve the largest stable step size per right-hand side evaluation. For the larger spectral clusters of the moment model and the generalized two-scale system, Projective Forward Euler achieves the largest value.

At the maximal stable step size, the optimized Runge-Kutta methods give smaller errors in most of the error comparisons. For the generalized two-scale system, however, the Projective Integration methods are more accurate at the longer final time. Their outer step size can be reduced to further decrease the error. For the optimized Runge-Kutta methods, this is not as easy because the stable step sizes do not always form one connected interval.

The different representations of the stability polynomial mainly affect the construction of methods with many stages. Overall, optimized Runge-Kutta methods usually give better results for smaller spectral clusters, while Projective Integration performs better for larger spectral clusters.

Zusammenfassung

In dieser Arbeit werden zwei explizite Ansätze zur numerischen Lösung steifer Differentialgleichungen mit getrennten langsamen und schnellen Zeitskalen verglichen. Bei der Projective Integration werden zunächst mehrere kleine innere Zeitschritte zur Dämpfung der schnellen Anteile durchgeführt und anschließend ein größerer äußerer Schritt verwendet. Als zweiter Ansatz werden explizite Runge-Kutta-Verfahren betrachtet, deren Stabilitätspolynome an das Spektrum des jeweiligen Problems angepasst werden. Der Vergleich erfolgt anhand der maximal stabilen Schrittweite, der Schrittweite pro Auswertung der rechten Seite und der numerischen Fehler.

Für die optimierten Runge-Kutta-Verfahren wird zunächst mit PolyOpt ein Stabilitätspolynom bestimmt. Anschließend wird mit RK-Opt ein zugehöriges Low-Storage-Runge-Kutta-Verfahren konstruiert. Die Verfahren werden für ein isoliertes Zwei-Skalen-Spektrum, Spektralcluster mit unterschiedlichen Radien, ein semi-diskretes hyperbolisches Momentenmodell und ein verallgemeinertes Zwei-Skalen-System untersucht.

Die Ergebnisse zeigen, dass der Vergleich stark vom Spektrum und von der Ordnung der Verfahren abhängt. Für das isolierte Spektrum und die schmaleren Spektralcluster erreichen optimierte Runge-Kutta-Verfahren häufig die größte stabile Schrittweite pro Auswertung der rechten Seite. Für die breiteren Spektralcluster des Momentenmodells und des verallgemeinerten Zwei-Skalen-Systems erzielt dagegen Projective Forward Euler den größten Wert.

Bei maximal stabiler Schrittweite liefern die optimierten Runge-Kutta-Verfahren in den meisten Fehlervergleichen kleinere Fehler. Beim verallgemeinerten Zwei-Skalen-System sind dagegen die Projective-Integration-Verfahren bei der längeren Endzeit genauer. Deren äußere Schrittweite kann zur weiteren Fehlerreduktion verkleinert werden. Bei den optimierten Runge-Kutta-Verfahren ist dies wegen der nicht immer zusammenhängenden stabilen Schrittweiten nicht ohne Weiteres möglich.

Die untersuchten Darstellungen des Stabilitätspolynoms beeinflussen insbesondere die Konstruktion von Verfahren mit vielen Stufen. Insgesamt sind optimierte Runge-Kutta-Verfahren bei schmaleren Spektren meist vorteilhaft, während Projective Integration bei breiteren Spektralclustern besser abschneidet.

1 Introduction

Many problems in science and engineering require the numerical solution of stiff ordinary differential equations (ODEs) of the form

$$\frac{\mathrm{d}u}{\mathrm{d}t} = f_\varepsilon(u(t)), \qquad u(t_0) = u_0, \tag{1.1}$$

where $u(t) \in \mathbb{R}^N$ is the unknown solution, $0 < \varepsilon \ll 1$ is a stiffness parameter, and f_ε denotes an autonomous right-hand side depending on ε. Small values of ε lead to stiff systems of ODEs.

In this thesis, we focus on stiff problems with separated slow and fast modes. This means that the modes evolve on different time scales. For linearized problems, this is the case if the eigenvalues of $\mathrm{D}f_\varepsilon(u)$ lie in separated regions in the complex plane.

Such systems arise in many physical applications. Examples are rarefied gas flows modeled by hyperbolic moment equations [KS21], shallow water moment models [AK22], and kinetic equations in the diffusion limit [LS12].

For standard explicit Runge-Kutta methods, this separation can lead to a strong stability restriction on the time step. As a result, the step size may be much smaller than what would be necessary to resolve the slow dynamics.

One approach for solving such systems is to use implicit methods. These can avoid the stability restriction of standard explicit methods. However, they require the solution of a system of equations at every time step, which can be computationally expensive, particularly for large-scale problems.

Another approach is to treat the stiff dynamics implicitly and the non-stiff dynamics explicitly. Such methods are called implicit-explicit Runge-Kutta (IMEX-RK) methods [PR05]. They require a decomposition of the right-hand side,

$$f_\varepsilon(u) = f_\varepsilon^{\mathrm{E}}(u) + f_\varepsilon^{\mathrm{I}}(u), \tag{1.2}$$

where $f_\varepsilon^{\mathrm{E}}$ denotes the non-stiff part and $f_\varepsilon^{\mathrm{I}}$ denotes the stiff part. This can reduce the computational cost compared with a fully implicit method. However, a system of equations still has to be solved for the stiff part, and a reasonable splitting of the right-hand side is required.

The first approach considered in this work is Projective Integration. Projective Integration methods are explicit methods for the time integration of stiff ODEs with separated slow

and fast dynamics [GK03]. The basic idea is to first perform a small number of inner time steps of size δt, which are used to damp the fast modes. This is followed by an extrapolation step with outer step size Δt over a much larger interval. Thus, the aim is to reduce the stability restriction caused by the fast modes, so that the slow modes determine the outer step size. The effect can be illustrated by the isolated two-scale spectrum

$$\Lambda_0 = \{-1, -1000\}. \tag{1.3}$$

The eigenvalue -1 corresponds to the slow mode, whereas the eigenvalue -1000 corresponds to the fast mode with $\varepsilon = 10^{-3}$. For Forward Euler, the maximal stable step size is

$$\Delta t_{\max}^{\mathrm{FE}} = 2 \cdot 10^{-3}. \tag{1.4}$$

The calculation is given in Section 2.2.1. Hence, the step size is determined by the fast eigenvalue.

To see how Projective Integration changes this stability restriction, we apply the simplest Projective Integration method, Projective Forward Euler (PFE), with inner step size $\delta t = 10^{-3}$ and $K = 1$. For this choice, the maximal stable outer step size is

$$\Delta t_{\max}^{\mathrm{PFEK1}} \approx 2.002. \tag{1.5}$$

Compared with Forward Euler, the damping of the fast mode allows a much larger outer step size. Thus, the stability restriction is governed by the slow eigenvalue rather than by the fast eigenvalue.

The second approach considered in this work is the optimization of Runge-Kutta stability polynomials for a given spectrum [KA12]. For a fixed order and number of stages, we seek a stability polynomial whose stability region contains the scaled spectrum for the largest possible time step. Since the methods are intended for large-scale problems, we restrict the construction to low-storage Runge-Kutta methods [Ket10], which require only a small number of memory registers. Once a target stability polynomial has been obtained, we use RK-Opt [PKD13] to construct a corresponding low-storage Runge-Kutta method. In this way, the spectrum is used directly to obtain a method for the given problem.

The aim of this work is to compare Projective Integration methods with explicit Runge-Kutta methods whose stability polynomials are optimized for a given multiscale spectrum. In particular, we investigate whether the optimized Runge-Kutta methods allow larger stable time steps and whether this gives an advantage in terms of accuracy and computational work.

This thesis is organized as follows. Chapter 2 first recalls explicit Runge-Kutta methods and then introduces Projective Forward Euler and Projective Runge-Kutta methods in Runge-Kutta form, following [KS25].

Chapter 3 presents the optimization of stability polynomials for given spectra following [KA12]. It also describes the construction of corresponding low-storage Runge-Kutta methods with RK-Opt [PKD13], using the low-storage formulations from [Ket10].

Chapter 4 presents the numerical experiments. The methods are studied for an isolated two-scale spectrum, clustered two-scale spectra with increasing radii, a semi-discrete hyperbolic moment model, and a generalized two-scale system. The comparison focuses on stability limits, errors, and the number of right-hand side evaluations.

Chapter 5 summarizes the main results of this thesis and gives an outlook on possible future work.

2 Projective Integration

Projective Integration is the first explicit approach considered in this work. In this chapter, we introduce Projective Forward Euler and Projective Runge-Kutta methods and rewrite them as explicit Runge-Kutta methods. We first recall standard explicit Runge-Kutta schemes and then derive the corresponding Runge-Kutta representations. The notation and formulation follow [KS25].

2.1 Runge-Kutta schemes

We first recall explicit Runge-Kutta (RK) schemes [HNW08, Section II.1] for autonomous initial value problems of the form

$$u'(t) = f(u(t)), \qquad u(t_0) = u_0. \tag{2.1}$$

An s-stage Runge-Kutta method is described by the Butcher coefficients $A = (a_{ij}) \in \mathbb{R}^{s\times s}$, $b \in \mathbb{R}^s$, and $c \in \mathbb{R}^s$. For a step size Δt, one time step is given by

$$k_j = f\left(u^n + \Delta t \sum_{l=1}^{j-1} a_{j,l}\, k_l\right), \qquad j = 1, \dots, s, \tag{2.2}$$

$$u^{n+1} = u^n + \Delta t \sum_{j=1}^{s} b_j\, k_j. \tag{2.3}$$

Here, $A = (a_{ij})$ contains the stage coefficients, $b = (b_i)$ contains the weights, and $c = (c_i)$ contains the nodes. The stage k_i approximates the derivative at time $t_n + c_i\Delta t$. The coefficients are summarized in the Butcher tableau

$$\begin{array}{c|c} c & A \\ \hline & b^\top \end{array} \tag{2.4}$$

The method is explicit if $a_{ij} = 0$ for all $j \geq i$. The nodes in the Butcher tableau are given by

$$c_i = \sum_{j=1}^{s} a_{ij}, \qquad i = 1, \dots, s. \tag{2.5}$$

The method is consistent if

$$\sum_{i=1}^{s} b_i = 1. \tag{2.6}$$

To achieve order p, the coefficients must satisfy the corresponding order conditions. These conditions form a nonlinear system of equations and can be derived using rooted trees [HNW08, Section II.2].

2.2 Projective Forward Euler

The simplest Projective Integration method is Projective Forward Euler (PFE) [GK03]. It combines $K+1$ small Forward Euler time steps of size δt with one extrapolation step over the remaining interval $\Delta t - (K+1)\delta t$. Starting with $u^{n,0} = u^n$, the method is given by

$$u^{n,1} = u^{n,0} + \delta t\, f\Big(u^{n,0}\Big), \tag{2.7}$$

$$\vdots$$

$$u^{n,K+1} = u^{n,K} + \delta t\, f\Big(u^{n,K}\Big), \tag{2.8}$$

$$u^{n+1} = u^{n,K+1} + \Big(\Delta t - (K+1)\delta t\Big)\, \frac{u^{n,K+1} - u^{n,K}}{\delta t}. \tag{2.9}$$

To express the PFE method in standard Runge-Kutta notation, we follow [KS25] and define

$$k_\ell := f\Big(u^{n,\ell}\Big), \qquad \ell = 0, \dots, K. \tag{2.10}$$

For the Forward Euler inner steps, this implies

$$u^{n,\ell+1} = u^{n,\ell} + \delta t\, k_\ell, \qquad \text{and hence} \qquad k_\ell = \frac{u^{n,\ell+1} - u^{n,\ell}}{\delta t}. \tag{2.11}$$

With this notation, the stages and the final update read

$$k_0 = f\Big(u^{n,0}\Big), \tag{2.12}$$

$$k_1 = f\Big(u^{n,1}\Big) = f\Big(u^{n,0} + \delta t\, k_0\Big), \tag{2.13}$$

$$k_2 = f\Big(u^{n,2}\Big) = f\Big(u^{n,0} + \delta t\, (k_0 + k_1)\Big), \tag{2.14}$$

$$\vdots$$

$$k_K = f\Big(u^{n,K}\Big) = f\left(u^{n,0} + \delta t \sum_{\ell=0}^{K-1} k_\ell\right), \tag{2.15}$$

$$u^{n+1} = u^{n,K+1} + \Big(\Delta t - (K+1)\delta t\Big)\, k_K. \tag{2.16}$$

To read off the Butcher coefficients, we rewrite (2.16) so that it matches the Runge-Kutta update (2.3) with step size Δt.

$$u^{n+1} = u^{n,K+1} + \Big(\Delta t - (K+1)\delta t\Big)\, k_K \tag{2.17}$$

$$= u^n + \delta t \sum_{\ell=0}^{K} k_\ell + \Big(\Delta t - (K+1)\delta t\Big)\, k_K \tag{2.18}$$

$$= u^n + \Delta t \left(\sum_{\ell=0}^{K-1} \frac{\delta t}{\Delta t}\, k_\ell + \left(1 - K\frac{\delta t}{\Delta t}\right) k_K \right). \tag{2.19}$$

With $\gamma := \delta t/\Delta t$, the Butcher tableau of the corresponding Runge-Kutta scheme then reads

$$\begin{array}{c|cccc} 0 & 0 & & & \\ \gamma & \gamma & 0 & & \\ \vdots & \vdots & \ddots & \ddots & \\ K\gamma & \gamma & \cdots & \gamma & 0 \\ \hline & \gamma & \cdots & \gamma & 1-K\gamma \end{array} \tag{2.20}$$

To write the Butcher tableau in compact form, we define

$$\boldsymbol{\gamma} = \gamma(0, 1, \ldots, K)^\top \in \mathbb{R}^{K+1}, \tag{2.21}$$

$$A_\gamma = \gamma \begin{pmatrix} 0 & & & \\ 1 & 0 & & \\ \vdots & \ddots & \ddots & \\ 1 & \cdots & 1 & 0 \end{pmatrix} \in \mathbb{R}^{(K+1)\times(K+1)}, \tag{2.22}$$

$$\boldsymbol{b}_\gamma = \gamma \left(1, \ldots, 1, \frac{1}{\gamma} - K\right)^\top \in \mathbb{R}^{K+1}. \tag{2.23}$$

Then the tableau can be written as

$$\begin{array}{c|c} \boldsymbol{\gamma} & A_\gamma \\ \hline & \boldsymbol{b}_\gamma^\top \end{array} \tag{2.24}$$

2.2.1 Stability of PFE and the choice of K

To illustrate the effect of the inner steps, we consider the scalar test equation

$$u'(t) = \lambda u(t). \tag{2.25}$$

Applying one inner Forward Euler step to this equation gives

$$u^{n,\ell+1} = (1 + \delta t\lambda) u^{n,\ell}. \tag{2.26}$$

Iterating this relation gives

$$u^{n,K} = \big(1 + \delta t\lambda\big)^K u^n. \tag{2.27}$$

Using the PFE update (2.16), we obtain

$$u^{n+1} = R_{\mathrm{PFE}}(\lambda)u^n = \big(1 + \delta t\lambda\big)^K \big(1 + (\Delta t - K\delta t)\lambda\big)u^n, \tag{2.28}$$

which defines the PFE stability function R_{PFE}. We now consider the isolated two-scale spectrum

$$\Lambda_0 = \{-1, -1000\}. \tag{2.29}$$

We choose $\delta t = 10^{-3}$ and $K = 1$, corresponding to $K + 1 = 2$ inner Forward Euler steps. For the fast eigenvalue $\lambda_{\mathrm{f}} = -1000$, we obtain

$$1 + \delta t\lambda_{\mathrm{f}} = 1 + 10^{-3}(-1000) = 0. \tag{2.30}$$

Inserting this into the stability function gives

$$R_{\mathrm{PFE}}(-1000) = \big(1 + \delta t\lambda_{\mathrm{f}}\big)^K \big(1 + (\Delta t - K\delta t)\lambda_{\mathrm{f}}\big) = 0. \tag{2.31}$$

Thus, the fast mode corresponding to λ_{f} is eliminated exactly. For the slow eigenvalue $\lambda_{\mathrm{s}} = -1$, we obtain

$$R_{\mathrm{PFE}}(-1) = (1 - 10^{-3})(1 - (\Delta t - 10^{-3})) = 0.999(1.001 - \Delta t). \tag{2.32}$$

The stability condition $|R_{\mathrm{PFE}}(-1)| \leq 1$ gives

$$\Delta t_{\max}^{\mathrm{PFEK1}} = 1.001 + \frac{1}{0.999} = 2.002001\ldots. \tag{2.33}$$

The calculation above only considers the isolated fast eigenvalue. For a fast spectral cluster, the value of K has to be chosen such that the fast part of the spectrum lies inside the PFE stability region. For the PFE method, this stability region is given by [KS22, Section 3.2]

$$\lambda \in C\left(-\frac{1}{\Delta t}, \frac{1}{\Delta t}\right) \cup C\left(-\frac{1}{\delta t}, \frac{1}{\delta t}\left(\frac{\delta t}{\Delta t}\right)^{1/K}\right), \tag{2.34}$$

where $C(z_0, r)$ denotes a disk with center z_0 and radius r. The first disk corresponds to the slow eigenvalues, whereas the second disk corresponds to the fast eigenvalues. Increasing K increases the radius of the second disk.

For $\delta t = 10^{-3}$ and $\Delta t = 2$, the radius of the fast component is 0.5 for $K = 1$ and 22.36 for $K = 2$. Hence, $K = 1$ is sufficient for the isolated eigenvalue -1000. If the fast eigenvalues form a cluster around -1000, however, the disk for $K = 1$ may be too small to contain the whole cluster. A larger value of K increases the radius of the fast stability disk and can make the method stable for all eigenvalues in the cluster. This effect is investigated in Chapter 4.

2.3 Projective Runge–Kutta

Projective Runge-Kutta (PRK) methods extend the idea of PFE by using an explicit Runge-Kutta method as the outer integrator [LLS16]. Instead of performing only one projective extrapolation, the method consists of S outer stages, each of which include $K+1$ inner steps of size δt, followed by an extrapolation step of the remaining step size $c_s\Delta t - (K+1)\delta t$, where c_s is the corresponding node of the outer Runge-Kutta tableau.

For the first outer stage, we set $u^{n,0} = u^n$ and take $K+1$ inner Forward Euler steps of size δt. The outer stage derivative is then computed from the last inner Forward Euler step,

$$s = 1 : \begin{cases} u^{n,k+1} = u^{n,k} + \delta t\, f\left(u^{n,k}\right), & 0 \le k \le K, \quad (2.35) \\ k_1 = \dfrac{u^{n,K+1} - u^{n,K}}{\delta t}. & (2.36) \end{cases}$$

For the remaining outer stages, the previously computed outer stage derivatives $k_1, \ldots, k_{s-1}$ are used to construct an approximation at the outer stage time $t_n + c_s\Delta t$. Starting from this value, we again take $K+1$ inner Forward Euler steps. The outer stage derivative k_s is then computed from the last inner Forward Euler step,

$$2 \le s \le S : \begin{cases} u_s^{n+c_s,0} = u^{n,K+1} + \left(c_s\Delta t - (K+1)\delta t\right) \displaystyle\sum_{l=1}^{s-1} \frac{a_{s,l}}{c_s}\, k_l, & (2.37) \\ u_s^{n+c_s,k+1} = u_s^{n+c_s,k} + \delta t\, f\left(u_s^{n+c_s,k}\right), \quad 0 \le k \le K, & (2.38) \\ k_s = \dfrac{u_s^{n+c_s,K+1} - u_s^{n+c_s,K}}{\delta t}. & (2.39) \end{cases}$$

Then, the final outer step update is given by

$$u^{n+1} = u^{n,K+1} + \left(\Delta t - (K+1)\delta t\right) \sum_{s=1}^{S} b_s\, k_s. \quad (2.40)$$

To write the PRK method in standard Runge-Kutta form, we follow [KS25] and define

$$k_{s,k+1} := f\left(u_s^{n+c_s,k}\right), \qquad s = 1, \ldots, S, \quad 0 \le k \le K. \quad (2.41)$$

Here, s is the outer stage index and k is the inner iteration index. The outer stage derivative is given by $k_s = k_{s,K+1}$. Then (2.38) becomes

$$u_s^{n+c_s,k} = u_s^{n+c_s,0} + \delta t \sum_{\ell=1}^{k} k_{s,\ell}. \quad (2.42)$$

Substituting this into the definition of $k_{s,k+1}$ gives

$$k_{s,k+1} = f\left(u_s^{n+c_s,0} + \delta t \sum_{\ell=1}^{k} k_{s,\ell}\right). \tag{2.43}$$

Substituting the start value $u_s^{n+c_s,0}$ from (2.37) into the expression above, we obtain

$$s = 1: \quad k_{1,k+1} = f\left(u^{n,0} + \delta t \sum_{\ell=1}^{k} k_{1,\ell}\right), \tag{2.44}$$

$$2 \le s \le S: \quad k_{s,k+1} = f\left(u^{n,0} + \delta t \sum_{\ell=1}^{K+1} k_{1,\ell} + (c_s \Delta t - (K+1)\delta t) \sum_{i=1}^{s-1} \frac{a_{s,i}}{c_s} k_{i,K+1} + \delta t \sum_{\ell=1}^{k} k_{s,\ell}\right). \tag{2.45}$$

Rewriting the step update as in (2.17)–(2.19) gives

$$u^{n+1} = u^{n,0} + \Delta t \left(\frac{\delta t}{\Delta t} \sum_{\ell=1}^{K+1} k_{1,\ell} + \left(1 - (K+1)\frac{\delta t}{\Delta t}\right) \sum_{s=1}^{S} b_s \, k_{s,K+1}\right). \tag{2.46}$$

Using the stage formulation above together with the notation (2.21)–(2.23), the PRK method can be written as an explicit Runge-Kutta method with the following Butcher tableau:

$$\begin{array}{c|ccccc} \boldsymbol{\gamma} & A_\gamma & & & & \\ \boldsymbol{c}_2 + \boldsymbol{\gamma} & A^a_{2,1} & A_\gamma & & & \\ \boldsymbol{c}_3 + \boldsymbol{\gamma} & A^a_{3,1} & 0^a_{3,2} & A_\gamma & & \\ \vdots & \vdots & \vdots & \ddots & \ddots & \\ \boldsymbol{c}_S + \boldsymbol{\gamma} & A^a_{S,1} & 0^a_{S,2} & \cdots & 0^a_{S,S-1} & A_\gamma \\ \hline & \boldsymbol{b}_1^\top & \boldsymbol{b}_2^\top & \cdots & \boldsymbol{b}_{S-1}^\top & \boldsymbol{b}_S^\top \end{array} =: \begin{array}{c|c} \bar{\boldsymbol{c}} & \bar{A}_\gamma \\ \hline & \bar{\boldsymbol{b}}^\top, \end{array} \tag{2.47}$$

with

$$\boldsymbol{c}_s = c_s(1, \ldots, 1)^\top \in \mathbb{R}^{K+1}, \qquad s = 1, \ldots, S. \tag{2.48}$$

Each block has size $(K+1) \times (K+1)$. The diagonal block A_γ represents the $K+1$ inner Forward Euler steps within a single outer stage.

The block $A^a_{s,1}$ describes how the first outer stage enters the later outer stage s. It is given by

$$A^a_{s,1} = \gamma \begin{pmatrix} 1 & \cdots & 1 & 1 + \tilde{a}_{s,1} \\ 1 & \cdots & 1 & 1 + \tilde{a}_{s,1} \\ \vdots & & \vdots & \vdots \\ 1 & \cdots & 1 & 1 + \tilde{a}_{s,1} \end{pmatrix} \in \mathbb{R}^{(K+1)\times(K+1)}, \tag{2.49}$$

where the first K columns correspond to the first K inner Forward Euler steps and the last column contains the additional extrapolation term.

For $2 \leq l \leq s-1$, only the final derivative $k_{l,K+1}$ of the previous outer stage l is used. Therefore, the block $0^a_{s,l}$ has zero entries in all columns except the last one.

$$0^a_{s,l} = \gamma \begin{pmatrix} 0 & \cdots & 0 & \tilde{a}_{s,l} \\ 0 & \cdots & 0 & \tilde{a}_{s,l} \\ \vdots & & \vdots & \vdots \\ 0 & \cdots & 0 & \tilde{a}_{s,l} \end{pmatrix} \in \mathbb{R}^{(K+1)\times(K+1)}, \tag{2.50}$$

with coefficient

$$\tilde{a}_{s,l} = \left(\frac{c_s}{\gamma} - (K+1) \right) \frac{a_{s,l}}{c_s}. \tag{2.51}$$

Here, $\tilde{a}_{s,l}$ represents the projective extrapolation over the remaining part of the stage interval. The weights are given by

$$\boldsymbol{b}_1 = \gamma \left(1, \ldots, 1,\ 1 + \left(\frac{1}{\gamma} - (K+1) \right) b_1 \right)^\top \in \mathbb{R}^{K+1}, \tag{2.52}$$

$$\boldsymbol{b}_s = \gamma \left(0, \ldots, 0,\ \left(\frac{1}{\gamma} - (K+1) \right) b_s \right)^\top \in \mathbb{R}^{K+1}, \qquad 2 \leq s \leq S. \tag{2.53}$$

With this representation, the PRK method is an explicit Runge-Kutta method with coefficients $(\bar{A}_\gamma, \bar{\boldsymbol{b}}, \bar{\boldsymbol{c}})$. Therefore, the usual Runge-Kutta consistency and order conditions can be applied. This is summarized in the following theorem.

Theorem 2.1. *A PRK method is consistent and at least first-order accurate. For $\gamma = \delta t/\Delta t \to 0$, the method recovers the second-order accuracy of the outer Runge-Kutta method.*

Proof. The proof is given in [KS25, Theorem 1]. □

3 Optimization of stability polynomials for Runge-Kutta methods

This chapter describes the construction of the optimized Runge-Kutta methods used in this work. The procedure consists of two steps. In the first step, for a fixed order and number of stages, a stability polynomial is optimized for the given spectrum. The aim is to obtain the largest possible stable step size. In the second step, the optimized polynomial is used as a target for the construction of a corresponding low-storage Runge-Kutta method.

This chapter is mainly based on the stability-polynomial optimization in [KA12], the RK-Opt framework in [PKD13], and the low-storage Runge-Kutta formulations in [Ket10].

3.1 Stability regions of Runge-Kutta methods

To formulate the optimization problem, we first recall the stability function of an explicit Runge-Kutta method. Consider the linear initial value problem

$$u'(t) = Lu(t), \qquad u(0) = u_0, \tag{3.1}$$

with $u_0 \in \mathbb{C}^N$ and $L \in \mathbb{C}^{N\times N}$. Applying an s-stage explicit Runge-Kutta method with step size h to this problem gives

$$u^{n+1} = R(hL)u^n, \tag{3.2}$$

where R is the stability function of the Runge-Kutta method. For an explicit s-stage Runge-Kutta method, this stability function is a polynomial of degree at most s,

$$R(z) = \sum_{j=0}^{s} \alpha_j z^j. \tag{3.3}$$

Since the exact solution of the linear problem is $u(t) = \exp(tL)u_0$, the stability polynomial of a method of order p has to agree with the exponential function up to order p. Therefore,

$$\alpha_j = \frac{1}{j!}, \qquad j = 0, \dots, p. \tag{3.4}$$

Thus, for fixed order p, only the remaining coefficients $\alpha_{p+1}, \dots, \alpha_s$ can be chosen to improve the stability properties.

Definition 3.1. The region of absolute stability of a stability function R is defined by

$$\mathcal{S} = \{z \in \mathbb{C} : |R(z)| \leq 1\}. \tag{3.5}$$

If Λ is the spectrum of the problem, then absolute stability requires that $h\Lambda \subset \mathcal{S}$.

Definition 3.2. For a given spectrum Λ, the maximal stable step size of a stability polynomial R is defined as

$$h_{\text{stable}} = \max \{h \geq 0 : |R(h\lambda)| \leq 1 \text{ for all } \lambda \in \Lambda\}. \tag{3.6}$$

For a given spectrum Λ, a fixed order p, and a fixed number of stages s, the goal is therefore to choose the free coefficients of the stability polynomial such that h_{stable} is as large as possible.

3.1.1 Stability optimization problem

Following [KA12], we now formulate the optimization of stability polynomials as an optimization problem. Since the stability polynomial is written in the monomial basis as $R(z) = \sum_{j=0}^{s} \alpha_j z^j$, the order conditions for the stability polynomial determine the coefficients $\alpha_0, \ldots, \alpha_p$ directly. Hence, only the remaining coefficients $\alpha_{p+1}, \ldots, \alpha_s$ are free. The stability optimization problem can therefore be written as

$$\begin{aligned} \max_{\alpha_{p+1},\ldots,\alpha_s,\, h} \quad & h \\ \text{subject to} \quad & |R(h\lambda)| - 1 \leq 0 \qquad \text{for all } \lambda \in \Lambda. \end{aligned} \tag{3.7}$$

This problem is, in general, nonconvex, since h and the free polynomial coefficients both appear in the stability constraints. A direct global solution of (3.7) is therefore difficult. For this reason, the problem is reformulated.

3.1.2 Reformulation as a least-deviation problem

Instead of directly maximizing the stable step size, we fix a value of h and ask whether the stability condition can be satisfied for this step size. This leads to the least-deviation problem introduced in [KA12]:

$$\min_{\alpha_{p+1},\ldots,\alpha_s} \max_{\lambda \in \Lambda} \left(|R(h\lambda)| - 1\right). \tag{3.8}$$

We denote its optimal value by

$$r_{p,s}(h, \Lambda). \tag{3.9}$$

For fixed h, $R(h\lambda)$ depends linearly on the free coefficients $\alpha_{p+1}, \ldots, \alpha_s$. Therefore, $|R(h\lambda)|$ is a convex function of these coefficients, and so is the maximum over all $\lambda \in \Lambda$. Hence (3.8) is a convex optimization problem.

Using $r_{p,s}(h, \Lambda)$, the original stability optimization problem (3.7) can be viewed as the search for the largest step size h such that

$$r_{p,s}(h, \Lambda) \leq 0. \tag{3.10}$$

Thus, the original problem is reduced to a one-dimensional search in h, while each test of a given step size is carried out by solving the convex problem (3.8). We therefore use a bisection procedure to approximate the largest step size for which $r_{p,s}(h, \Lambda) \leq 0$. Starting from an interval $[h_{\min}, h_{\max}]$, we repeatedly solve (3.8) for the midpoint h. The interval is then updated depending on whether the stability condition is satisfied. This leads to the following algorithm.

Algorithm 1 Bisection for the optimal stable step size [KA12].

Choose $h_{\max} > 0$ such that $r_{p,s}(h_{\max}, \Lambda) > 0$
Set $h_{\min} = 0$
while $h_{\max} - h_{\min} >$ tol **do**
    $h \leftarrow \frac{1}{2}(h_{\min} + h_{\max})$
    Solve the least-deviation problem (3.8)
    if $r_{p,s}(h, \Lambda) \leq 0$ **then**
        $h_{\min} \leftarrow h$
    else
        $h_{\max} \leftarrow h$
    end if
end while
return $h^* = h_{\min}$

The bisection starts with $h_{\min} = 0$, which is feasible since $r_{p,s}(0, \Lambda) = 0$. Since $r_{p,s}(h, \Lambda) \to +\infty$ as $h \to \infty$, an upper bound $h_{\max} > 0$ can be chosen such that $r_{p,s}(h_{\max}, \Lambda) > 0$. Global convergence of the bisection algorithm is guaranteed only if the following condition holds:

$$r_{p,s}(h_0, \Lambda) = 0 \quad \Longrightarrow \quad r_{p,s}(h, \Lambda) \leq 0 \qquad \text{for all } 0 \leq h \leq h_0. \tag{3.11}$$

For $p = 1$, this condition is proved in [KA12]. For $p > 1$, a general convergence result is not available. However, the authors of [KA12] report that they were unable to find an example with $s > p$ for which (3.11) fails. Therefore, the bisection approach is used in practice.

The initial value $h_{\max}$ can be found by starting with a guess and successively doubling it until $r_{p,s}(h_{\max}, \Lambda) > 0$.

3.1.3 Low-storage Runge-Kutta methods

Since the optimized methods may have many stages, storage requirements become relevant. We therefore use low-storage Runge-Kutta formulations when constructing

these methods[Ket10].

An implementation of an s-stage explicit Runge-Kutta method usually requires $s+1$ registers of length N, where N denotes the dimension of the ODE system. For large systems of ODEs, this storage requirement may become a limiting factor. Low-storage Runge-Kutta methods reduce this requirement by using relations between the stages so that only certain combinations of previously computed stages have to be stored. A useful representation for this purpose is the Shu-Osher form [SO88]. In this form, an explicit Runge-Kutta method can be written as

$$y_1 = u^n, \tag{3.12}$$

$$y_i = \sum_{j=1}^{i-1} \left(a_{ij} y_j + \beta_{ij} h\, f(y_j) \right), \qquad i = 2, \ldots, s+1, \tag{3.13}$$

$$u^{n+1} = y_{s+1}. \tag{3.14}$$

Here, a_{ij} and β_{ij} denote the coefficients of the Shu-Osher representation. Coefficients with nonpositive indices are interpreted as zero. This form shows which previously computed stage values are needed to construct the next stage. One class of low-storage Runge-Kutta methods is given by the Williamson $2N$ methods [Wil80]. Their Shu-Osher representation is

$$y_i = a_{i,i-2} y_{i-2} + (1 - a_{i,i-2}) y_{i-1} + \beta_{i,i-1} h\, f(y_{i-1}), \qquad 1 < i \leq s+1. \tag{3.15}$$

Thus, each stage depends only on the two previous stage values. This makes it possible to implement the method using only two solution-sized memory registers S_1 and S_2. Starting from u^n, one step is initialized by

$$S_1 := u^n, \qquad S_2 := 0. \tag{3.16}$$

Then, for $i = 2, \ldots, s+1$,

$$S_2 := \tilde{A}_i S_2 + h\, f(S_1), \tag{3.17}$$

$$S_1 := S_1 + \tilde{B}_i S_2. \tag{3.18}$$

After the final stage, the new solution is given by

$$u^{n+1} = S_1. \tag{3.19}$$

Here, $\tilde{A}_i$ and $\tilde{B}_i$ are the low-storage coefficients of the implementation. They determine the Shu-Osher coefficients through

$$a_{i,i-2} = -\frac{\tilde{B}_i \tilde{A}_i}{\tilde{B}_{i-1}}, \qquad \beta_{i,i-1} = \tilde{B}_i. \tag{3.20}$$

While $2N$ methods can be implemented with only two registers, they allow only a limited number of free parameters. A more flexible class is given by $3S^*$ methods [Ket10], which

use three registers and offer more freedom in the choice of the coefficients. In particular, a $3S^*$ method has $4s-6$ free parameters, whereas a $2N$ method has only $2s-1$. A Shu-Osher representation of a $3S^*$ method is given by

$$y_i = a_{i,1}u^n + a_{i,i-1}y_{i-1} + a_{i,i-2}y_{i-2} + \beta_{i,i-2}h\,f(y_{i-2}) + \beta_{i,i-1}h\,f(y_{i-1}), \qquad 2 \le i \le s+1. \tag{3.21}$$

This form can be implemented using three registers. Let S_1, S_2, S_3 denote three solution-sized memory registers. The corresponding low-storage recursion starts with

$$S_1 := u^n, \qquad S_2 := 0, \qquad S_3 := u^n. \tag{3.22}$$

Then, for $i = 2, \dots, s+1$,

$$S_2 := S_2 + \delta_{i-1}S_1, \tag{3.23}$$

$$S_1 := \gamma_{i,1}S_1 + \gamma_{i,2}S_2 + \gamma_{i,3}S_3 + \beta_{i,i-1}h\,f(S_1). \tag{3.24}$$

After the last stage, the new solution is given by

$$u^{n+1} = S_1. \tag{3.25}$$

Here, $\gamma_{i,1}, \gamma_{i,2}, \gamma_{i,3}$ and δ_i are the low-storage coefficients of the algorithm, while $\beta_{i,i-1}$ are the corresponding Shu-Osher coefficients. In the low-storage algorithm, not all coefficients of the Shu-Osher form appear explicitly. Some are written directly, while others are contained implicitly in the register updates. Assuming $\gamma_{i,2} \neq 0$, the full Shu-Osher form is recovered through the relations given in [Ket10]:

$$\beta_{i+1,i-1} = -\frac{\gamma_{i+1,2}}{\gamma_{i,2}}\,\beta_{i,i-1}, \qquad 2 \le i \le s, \tag{3.26}$$

$$a_{i+1,1} = \gamma_{i+1,3} - \frac{\gamma_{i+1,2}}{\gamma_{i,2}}\,\gamma_{i,3}, \qquad 2 \le i \le s, \tag{3.27}$$

$$a_{i+1,i-1} = -\frac{\gamma_{i+1,2}}{\gamma_{i,2}}\,\gamma_{i,1}, \qquad 2 \le i \le s, \tag{3.28}$$

$$a_{i+1,i} = 1 - a_{i+1,i-1} - a_{i+1,1}, \qquad 2 \le i \le s. \tag{3.29}$$

For the analysis of order and stability, it is helpful to rewrite the method in classical Butcher form. Following [Ket10], let a_0 and β_0 denote the upper $s \times s$ blocks of the Shu-Osher coefficient matrices, and let a_1 and β_1 denote the final rows.

$$A = (I - a_0)^{-1}\beta_0, \qquad b^\top = \beta_1 + a_1 A. \tag{3.30}$$

For further details on Shu-Osher representations and other low-storage Runge-Kutta methods, we refer to [Ket10].

3.1.4 Construction of Runge-Kutta methods

Following [PKD13], we describe how a Runge-Kutta method is constructed from the optimized stability polynomial.

The polynomial optimization described above gives an optimized stability polynomial

$$R^*(z) = \sum_{j=0}^{s} \alpha_j^* z^j, \tag{3.31}$$

and the corresponding optimized step size h^*. However, this does not yet define a Runge-Kutta method. The polynomial R^* describes only the stability behavior for linear problems and does not determine the Butcher coefficients.

To obtain a Runge-Kutta method, we have to construct Butcher coefficients whose stability function matches the optimized polynomial. For coefficients A and b, this stability function is given by

$$R_{A,b}(z) = 1 + z\, b^\top (I - zA)^{-1} \mathbf{1}. \tag{3.32}$$

Here, $\mathbf{1} \in \mathbb{R}^s$ denotes the vector of ones. For an explicit method, A is strictly lower triangular and therefore $A^s = 0$. Hence $R_{A,b}$ is a polynomial of degree at most s, and it can be written as

$$R_{A,b}(z) = 1 + \sum_{j=1}^{s} b^\top A^{j-1} \mathbf{1}\, z^j. \tag{3.33}$$

The aim of the second step is therefore to find coefficients A and b such that the coefficients of $R_{A,b}$ agree with those of R^*:

$$b^\top A^{j-1} \mathbf{1} = \alpha_j^*, \qquad j = 1, \dots, s. \tag{3.34}$$

These equations do not determine the Runge-Kutta method uniquely. An explicit s-stage Runge-Kutta method has $s(s-1)/2$ free entries in the strictly lower triangular matrix A and s entries in the weight vector b. Hence, there are $s(s+1)/2$ Butcher coefficients in total. In contrast, the stability polynomial gives only s matching conditions. Thus, different Runge-Kutta methods can have the same stability polynomial.

The remaining degrees of freedom are used to satisfy the Runge-Kutta order conditions, to obtain a low-storage implementation, and to keep the leading error terms small. Following [PKD13], the Runge-Kutta construction problem can be written as

$$\begin{aligned} \min_{A,b} \quad & C^{(p+1)} \\ \text{subject to} \quad & \tau_i^{(j)}(A,b) = 0, \qquad j = 1, \dots, p, \\ & b^\top A^{j-1} \mathbf{1} = \alpha_j^*, \qquad j = 1, \dots, s, \\ & \Gamma(A,b) = 0. \end{aligned} \tag{3.35}$$

Here, $\tau_i^{(j)}(A,b) = 0$ denotes the Runge-Kutta order conditions. The index j denotes the order, while i labels the conditions of that order.

$C^{(p+1)}$ is the principal error norm. It measures the size of the leading error terms that remain after the order conditions up to order p have been satisfied and is defined by

$$C^{(p+1)} = \left(\sum_i \left(\tau_i^{(p+1)}(A,b) \right)^2 \right)^{1/2}. \tag{3.36}$$

$\Gamma(A,b) = 0$ denotes the restrictions that are necessary for the method to be written in the chosen low-storage form. This means that the Butcher coefficients must be representable by one of the low-storage formulations introduced in Section 3.1.3.

In the implementation, the coefficients of the chosen low-storage formulation are used directly as optimization variables, and the corresponding Butcher coefficients A and b are computed from them. Thus, the low-storage conditions are imposed implicitly rather than added as separate constraints. In this way, the optimized stability polynomial is used to construct a Runge-Kutta method. The resulting method matches the optimized stability polynomial, satisfies the required order conditions, and can be implemented in the chosen low-storage form.

3.1.5 Choice of polynomial basis

So far, we have written the stability polynomial in the monomial basis

$$R(z) = \sum_{j=0}^{s} \alpha_j z^j. \tag{3.37}$$

This basis is useful because the order conditions directly determine the first $p+1$ coefficients of the stability polynomial. For the numerical solution of the least-deviation problem, however, it can lead to conditioning problems [KA12].

Let $\Lambda = \{\lambda_1, \ldots, \lambda_M\}$ denote the spectrum considered in the optimization and set $z_m = h\lambda_m$. Then

$$R(z_m) = \sum_{j=0}^{s} \alpha_j z_m^j, \qquad m = 1, \ldots, M. \tag{3.38}$$

Hence, evaluating the polynomial at the points z_m leads to a Vandermonde matrix,

$$\begin{pmatrix} R(z_1) \\ \vdots \\ R(z_M) \end{pmatrix} = \begin{pmatrix} 1 & z_1 & z_1^2 & \cdots & z_1^s \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & z_M & z_M^2 & \cdots & z_M^s \end{pmatrix} \begin{pmatrix} \alpha_0 \\ \vdots \\ \alpha_s \end{pmatrix}. \tag{3.39}$$

The Vandermonde matrix coming from the monomial basis is often ill-conditioned because its entries can differ strongly in size. As a result, the computed coefficients are sensitive to numerical errors. Hence, even small inaccuracies can have a large effect on the result.

To reduce this difficulty, we consider a more general representation

$$R(z) = \sum_{j=0}^{s} c_j Q_j(z), \qquad Q_j(z) = \sum_{k=0}^{j} q_{jk} z^k, \tag{3.40}$$

where Q_j is a polynomial of degree j. The basis polynomials are chosen so that the resulting matrix is better conditioned on the scaled spectrum.

This change of basis has one important consequence. In the monomial basis, the first $p+1$ coefficients are fixed directly by the order conditions. In a general basis, this is no longer true. Instead, all coefficients $c_0, \dots, c_s$ are treated as unknowns, and the order conditions are imposed as linear constraints.

$$\sum_{j=0}^{s} c_j q_{jk} = \frac{1}{k!}, \qquad k = 0, \dots, p, \tag{3.41}$$

where $q_{jk} = 0$ for $k > j$. Thus, compared with the monomial basis, a general polynomial basis leads to a slightly larger optimization problem and introduces additional equality constraints. Nevertheless, this is often worthwhile, because it can improve the conditioning of the stability constraints.

For spectra located mainly near the negative real axis, a natural choice is given by the Chebyshev polynomials of the first kind [KA12]. To adapt them to the relevant interval, we map $[hx, 0]$ to $[-1, 1]$, where

$$x = \min_{\lambda \in \Lambda} \mathrm{Re}(\lambda). \tag{3.42}$$

This gives the basis functions

$$Q_j(z) = T_j\left(1 - \frac{2z}{hx}\right). \tag{3.43}$$

Here, T_j denotes the Chebyshev polynomial of the first kind, defined recursively by

$$T_0(y) = 1, \qquad T_1(y) = y, \qquad T_{j+1}(y) = 2yT_j(y) - T_{j-1}(y), \qquad j \geq 1. \tag{3.44}$$

On this interval, the Chebyshev polynomials satisfy

$$|T_j(y)| \leq 1, \qquad y \in [-1, 1]. \tag{3.45}$$

Thus, the basis functions remain bounded on the scaled interval. This makes the Chebyshev basis more suitable for spectra near the negative real axis. We will see in Section 4.1.2 that the choice of basis can have a strong influence on the optimized polynomial.

After the polynomial has been optimized in the chosen basis, it is rewritten in the monomial basis for the coefficient-matching step in RK-Opt. Since

$$Q_j(z) = \sum_{k=0}^{j} q_{jk} z^k, \tag{3.46}$$

we obtain

$$R^*(z) = \sum_{j=0}^{s} c_j^* Q_j(z) = \sum_{k=0}^{s} \alpha_k^* z^k, \qquad \alpha_k^* = \sum_{j=k}^{s} c_j^* q_{jk}. \tag{3.47}$$

The coefficients $\alpha_{p+1}^*, \ldots, \alpha_s^*$ are then used as target coefficients in the stability matching conditions (3.34) of RK-Opt.

3.1.6 Chebyshev interpolation

The coefficient matching conditions in (3.34) use the monomial coefficients of the target stability polynomial. For methods with many stages, this can become numerically sensitive because the conditions involve high powers of A and the monomial coefficients of a high-degree polynomial.

We therefore represent the target polynomial by its values at Chebyshev interpolation points. These values are used directly for the stability constraints in PolyOpt and for the matching conditions in RK-Opt.

Let $R^*(z)$ be the target stability polynomial. On the interval $[-1, 1]$, we choose the Chebyshev-Lobatto interpolation points

$$x_j = \cos\left(\frac{j\pi}{s}\right), \qquad j = 0, \ldots, s. \tag{3.48}$$

If the polynomial is represented on the interval $[z_{\min}, 0]$, these points are mapped by

$$z_j = \frac{z_{\min}}{2}(1 - x_j), \qquad j = 0, \ldots, s. \tag{3.49}$$

Here, $z_{\min} = hx$ is the left endpoint of the scaled interval. The target polynomial is then represented by the nodal values

$$R_j = R^*(z_j), \qquad j = 0, \ldots, s. \tag{3.50}$$

These values define a unique interpolation polynomial R_I by

$$R_I(z_j) = R_j, \qquad j = 0, \ldots, s. \tag{3.51}$$

Since R^* has degree at most s, and R_I agrees with R^* at $s + 1$ distinct interpolation points, the two polynomials are identical:

$$R_I(z) = R^*(z) \qquad \text{for all } z. \tag{3.52}$$

To evaluate this polynomial away from the interpolation points, we use barycentric interpolation [Tre19]. Let $z_0, \ldots, z_s$ be the interpolation points. The barycentric weights are defined by

$$w_j = \frac{1}{\prod_{\substack{k=0 \\ k \neq j}}^{s} (z_j - z_k)}, \qquad j = 0, \ldots, s. \tag{3.53}$$

Then the interpolating polynomial is evaluated by the barycentric formula

$$R_I(z) = \frac{\displaystyle\sum_{j=0}^{s} \frac{w_j}{z - z_j} R_j}{\displaystyle\sum_{j=0}^{s} \frac{w_j}{z - z_j}}, \qquad z \notin \{z_0, \dots, z_s\}. \tag{3.54}$$

At an interpolation point, we set

$$R_I(z_j) = R_j. \tag{3.55}$$

Thus, the stability constraints in PolyOpt can be evaluated directly from the interpolation values. The order conditions, however, still require a transformation to the monomial coefficients.

In the standard interpolation formulation, the polynomial is represented by its nodal values

$$R_j = R(z_j), \qquad j = 0, \dots, s, \tag{3.56}$$

which are used as optimization variables. The order conditions are given in terms of the coefficients of the monomial representation

$$R(z) = \sum_{k=0}^{s} \alpha_k z^k. \tag{3.57}$$

Evaluating this representation at all interpolation points gives

$$\begin{pmatrix} 1 & z_0 & z_0^2 & \cdots & z_0^s \\ 1 & z_1 & z_1^2 & \cdots & z_1^s \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & z_s & z_s^2 & \cdots & z_s^s \end{pmatrix} \begin{pmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_s \end{pmatrix} = \begin{pmatrix} R_0 \\ R_1 \\ \vdots \\ R_s \end{pmatrix}. \tag{3.58}$$

Thus, the monomial coefficients are obtained from the nodal values by solving the corresponding Vandermonde system. The order conditions are then imposed as

$$\alpha_k = \frac{1}{k!}, \qquad k = 0, \dots, p. \tag{3.59}$$

Hence, the Vandermonde transformation is required only for the order conditions, while the stability constraints remain in the nodal representation and the RK-Opt construction uses point-value matching instead of coefficient matching. This formulation is used by default. For larger stage numbers, however, the Vandermonde transformation can become ill-conditioned.

For the higher-stage cases considered in Sections 4.5 and 4.6, we additionally compare an alternative formulation based directly on the derivatives of the interpolation polynomial.

Let Π_s denote the space of polynomials of degree at most s, and let $\ell_j \in \Pi_s$ denote the Lagrange basis polynomial satisfying

$$\ell_j(z_i) = \delta_{ij}, \qquad \delta_{ij} = \begin{cases} 1, & i = j, \\ 0, & i \neq j, \end{cases} \tag{3.60}$$

so that

$$R(z) = \sum_{j=0}^{s} R_j \ell_j(z). \tag{3.61}$$

Following [Pér19, Section 2], define

$$a_j = \prod_{\substack{m=0 \\ m \neq j}}^{s} (z_j - z_m) = \frac{1}{w_j}. \tag{3.62}$$

At the interpolation point $z_0 = 0$, the first derivatives are given by

$$\ell_j'(z_0) = \frac{a_0}{a_j(z_0 - z_j)}, \qquad j = 1, \ldots, s, \tag{3.63}$$

together with

$$\ell_0'(z_0) = -\sum_{j=1}^{s} \ell_j'(z_0). \tag{3.64}$$

For $q \geq 2$, the higher derivatives are obtained recursively as

$$\ell_j^{(q)}(z_0) = \frac{q}{z_0 - z_j} \left(\frac{a_0}{a_j} \ell_0^{(q-1)}(z_0) - \ell_j^{(q-1)}(z_0) \right), \qquad j = 1, \ldots, s, \tag{3.65}$$

and

$$\ell_0^{(q)}(z_0) = -\sum_{j=1}^{s} \ell_j^{(q)}(z_0). \tag{3.66}$$

Since the order conditions are equivalent to $R^{(q)}(0) = 1$ for $q = 0, \ldots, p$, they can be imposed directly on the nodal values as

$$\sum_{j=0}^{s} R_j \ell_j^{(q)}(0) = 1, \qquad q = 0, \ldots, p. \tag{3.67}$$

This formulation avoids the Vandermonde transformation and imposes the order conditions directly in terms of the nodal values.

In both formulations, the optimized polynomial is represented by its nodal values R_j. These values are also used when constructing the Runge-Kutta method. Instead of matching monomial coefficients, we match the values of the stability function at selected interpolation points.

It remains to decide how many interpolation points are needed. The stability polynomial of an s-stage explicit Runge-Kutta method has degree at most s. Hence, $s+1$ point values would determine such a polynomial uniquely. However, the stability polynomial is not completely free. The order conditions already fix the first $p+1$ Taylor coefficients of the polynomial. Therefore, fewer additional point values are needed. The following theorem shows that $s-p$ additional nonzero point values are enough to determine the polynomial.

Theorem 3.3. *Let P and Q be two polynomials of degree at most s. Assume that they satisfy the same order conditions up to order p, i.e.*

$$P^{(k)}(0) = Q^{(k)}(0), \qquad k = 0, \dots, p.$$

Let $z_1, \dots, z_{s-p}$ be pairwise distinct points with $z_j \neq 0$. If

$$P(z_j) = Q(z_j), \qquad j = 1, \dots, s-p,$$

then

$$P \equiv Q.$$

Proof. Consider the difference

$$D(z) = P(z) - Q(z).$$

Then D is a polynomial of degree at most s. From the order conditions we obtain

$$D^{(k)}(0) = 0, \qquad k = 0, \dots, p.$$

Hence D has a zero at $z = 0$ of multiplicity at least $p+1$. The additional point conditions give

$$D(z_j) = 0, \qquad j = 1, \dots, s-p.$$

Since the points z_j are pairwise distinct and different from zero, D has at least

$$(p+1) + (s-p) = s+1$$

zeros, counted with multiplicity. A nonzero polynomial of degree at most s cannot have $s+1$ zeros. Therefore $D \equiv 0$, and thus

$$P \equiv Q.$$

□

This shows that, once the order conditions up to order p are satisfied, $s-p$ additional nonzero interpolation values are enough to determine the stability polynomial. We therefore choose $s-p$ matching points from $z_1, \dots, z_s$. Let $\mathcal{I}$ denote the corresponding set of selected indices. The interpolation matching conditions are

$$R_{A,b}(z_j) = R_j, \qquad j \in \mathcal{I}. \tag{3.68}$$

The values $R_{A,b}(z_j)$ are evaluated directly from the stability function

$$R_{A,b}(z) = 1 + z\, b^\top (I - zA)^{-1}\mathbf{1}. \tag{3.69}$$

Thus, we do not match the monomial coefficients $b^\top A^{\ell-1}\mathbf{1}$ directly. Instead, we match values of the stability function at selected interpolation points.

Replacing the coefficient matching conditions in (3.34) by the interpolation conditions (3.68), the RK-Opt construction problem becomes

$$\begin{aligned} \min_{A,b} \quad & C^{(p+1)} \\ \text{subject to} \quad & \tau_i^{(j)}(A,b) = 0, \qquad j = 1,\dots,p, \\ & R_{A,b}(z_j) = R_j, \qquad j \in \mathcal{I}, \\ & \Gamma(A,b) = 0. \end{aligned} \tag{3.70}$$

Thus, the interpolation-based construction uses the same order and low-storage constraints as the coefficient-based construction, but replaces the coefficient matching conditions by point-value matching conditions.

3.1.7 Numerical implementation

The computations are performed in MATLAB using `polyopt` and `rk_opt` from the RK-Opt software package [Ket+20]. The stability-polynomial optimization described in Section 3.1.1 is carried out with `polyopt`. The resulting polynomial is then passed to `rk_opt`, which searches for a Runge-Kutta method in the chosen low-storage class, as described in Section 3.1.4.

The least-deviation problem used in the stability-polynomial optimization is formulated in CVX [GB08; GB14] and solved using the interior-point solvers `SeDuMi` [Stu99] and `SDPT3` [TTT03].

In `rk_opt`, the Runge-Kutta coefficients are computed in the chosen low-storage class using the order conditions, the stability-polynomial matching conditions, and the low-storage restrictions summarized in (3.35).

For the coefficient-based construction, the optimization problem (3.35) is used. In the interpolation-based construction, the coefficient constraints are replaced by the interpolation constraints in (3.68).

After constructing the Runge-Kutta coefficients, we check the stability of the method on the spectrum at the step size h returned by PolyOpt. For this purpose, we compute

$$\max_{\lambda \in \Lambda} |R_{A,b}(h\lambda)|. \tag{3.71}$$

Here, $R_{A,b}$ is evaluated from the Butcher coefficients. Unless stated otherwise, this evaluation is used for all stability checks of the constructed Runge-Kutta methods in the

numerical experiments. The method is considered stable at this step size if this value does not exceed $1 + 10^{-8}$. To compare the efficiency of the methods, we use

$$\frac{h}{\#\mathrm{RHS}}, \tag{3.72}$$

where #RHS denotes the number of right-hand side evaluations per time step. For an explicit s-stage Runge-Kutta method, $\#\mathrm{RHS} = s$, since each stage requires one right-hand side evaluation. Thus, $h/\#\mathrm{RHS}$ is the stable step size per right-hand side evaluation.

Unless stated otherwise, one RK-Opt run with 300 starting points was performed for each method. If this run did not produce a method satisfying the order and stability conditions, up to four additional runs with new starting points were performed for configurations that could improve the later comparison based on $h/\#\mathrm{RHS}$. The stage search was stopped when larger tested stage numbers did not further improve $h/\#\mathrm{RHS}$ or when no method satisfying the order and stability conditions was obtained after five runs. Configurations for which no such method was obtained were not included in the tables.

4 Numerical experiments

This chapter presents the numerical experiments used to compare Projective Integration methods with optimized Runge-Kutta methods. We first consider an isolated two-scale spectrum and then replace the two eigenvalues by circular clusters of increasing size. Afterwards, the methods are applied to a semi-discrete moment model and a generalized two-scale system. The comparison is based on the maximal stable step size, the step size per right-hand side evaluation, and the numerical error.

4.1 Isolated two-scale spectrum

We first consider the isolated two-scale spectrum $\Lambda_0 = \{-1, -1000\}$, which consists of one slow and one fast eigenvalue. Afterwards, we replace each eigenvalue by a circular cluster. This gives a two-scale cluster spectrum with one slow cluster around -1 and one fast cluster around -1000.

As a test problem, we consider the system

$$\begin{aligned} u_1'(t) &= -\alpha u_1(t), \\ u_2'(t) &= \frac{1}{\varepsilon}(u_1(t) - u_2(t)). \end{aligned} \tag{4.1}$$

with initial values

$$u_1(0) = 1, \qquad u_2(0) = 0. \tag{4.2}$$

The system can be written in the form

$$u'(t) = Au(t), \tag{4.3}$$

with

$$A = \begin{pmatrix} -\alpha & 0 \\ \frac{1}{\varepsilon} & -\frac{1}{\varepsilon} \end{pmatrix}. \tag{4.4}$$

The eigenvalues of this matrix are

$$\lambda_{\mathrm{s}} = -\alpha, \qquad \lambda_{\mathrm{f}} = -\frac{1}{\varepsilon}. \tag{4.5}$$

For the numerical experiments, we set

$$\alpha = 1, \qquad \varepsilon = 10^{-3}. \tag{4.6}$$

This gives the eigenvalues

$$\lambda_1 = -1, \qquad \lambda_2 = -1000. \tag{4.7}$$

Thus, λ_1 represents the slow mode, while λ_2 represents the fast mode. The exact solution is given by

$$u_1(t) = u_1(0)\exp(-\alpha t), \tag{4.8}$$

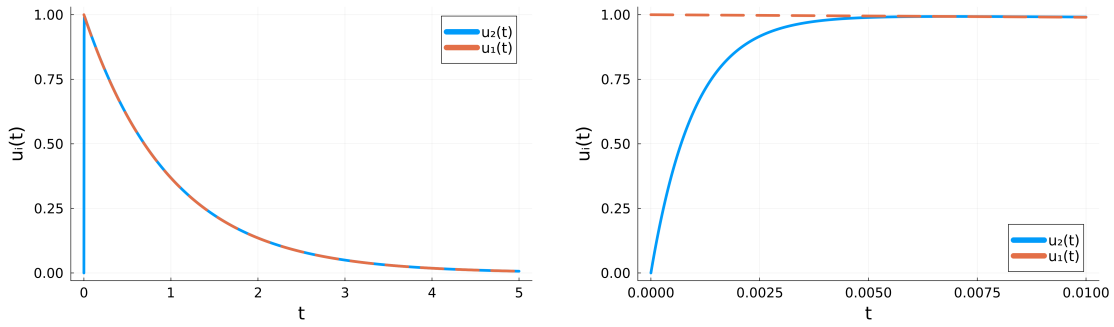
$$u_2(t) = \frac{u_1(0)}{1-\alpha\varepsilon}\exp(-\alpha t) + \frac{(1-\alpha\varepsilon)u_2(0) - u_1(0)}{1-\alpha\varepsilon}\exp\left(-\frac{t}{\varepsilon}\right). \tag{4.9}$$

For the initial values in (4.2), the exact solution simplifies to

$$u_1(t) = \exp(-\alpha t), \tag{4.10}$$

$$u_2(t) = \frac{\exp(-\alpha t) - \exp(-t/\varepsilon)}{1-\alpha\varepsilon}. \tag{4.11}$$

Figure 4.1 illustrates the separation of the two time scales in the problem. On the fast time scale, the slow component u_1 changes only slightly, whereas u_2 rapidly approaches u_1.



(a) Solution up to $T = 5$

(b) Solution up to $T = 0.01$

Figure 4.1: Exact solution of the isolated two-scale problem (4.1) with the initial values (4.2).

The matrix A is diagonalizable but nonnormal. For a normal matrix A, there exists a unitary matrix Q such that

$$A = Q\,\mathrm{diag}(\lambda_1, \ldots, \lambda_N)Q^*.$$

Therefore,

$$\|R(hA)\|_2 = \max_{\lambda \in \mathrm{spec}(A)} |R(h\lambda)|.$$

Hence, if all scaled eigenvalues lie inside the stability region, then

$$\|u^{n+1}\|_2 \le \|R(hA)\|_2 \|u^n\|_2 \le \|u^n\|_2.$$

For a nonnormal matrix, this relation does not hold in general. Therefore, the norm of the numerical solution may increase even if all scaled eigenvalues lie inside the stability region. Pseudospectral effects may therefore also be relevant [KA12, Section 4.3]. In this work, however, the stability analysis is based on the spectrum.

4.1.1 Projective Integration methods

We now apply Projective Integration methods to the isolated two-scale problem introduced above. As discussed in Section 2.2.1, we choose

$$\delta t = \varepsilon = 10^{-3}, \qquad K = 1. \tag{4.12}$$

For the fast eigenvalue $\lambda_2 = -1000$, this choice gives

$$1 + \lambda_2 \delta t = 0, \tag{4.13}$$

so the isolated fast mode is eliminated after the first inner step.

With this parameter choice, we compare Projective Forward Euler (PFE) and Projective Runge-Kutta with a fourth-order Runge-Kutta method as outer integrator (PRK4). The Butcher tableau of PRK4 with $K = 1$ is given in Appendix 6.2. Throughout this chapter, PFEKK and PRK4KK denote PFE and PRK4 with parameter K. If K is not written explicitly, we use $K = 1$. For both methods, we compute the maximal stable outer step size $\Delta t_{\max}$ for the two-point spectrum

$$\Lambda_0 = \{-1, -1000\}. \tag{4.14}$$

We determine $\Delta t_{\max}$ by bisection. For each value of Δt, we check the stability condition

$$\max_{\lambda \in \Lambda_0} |R(\Delta t \lambda)| \leq 1. \tag{4.15}$$

The resulting stability limits are shown in Table 4.1.

Table 4.1: Maximal stable outer step sizes for PFE and PRK4 for the isolated spectrum $\Lambda_0 = \{-1, -1000\}$.

Method	K	δt	#RHS	$\Delta t_{\max}$	$\Delta t_{\max}$/#RHS
PFE	1	10^{-3}	2	2.002001	1.001001
PRK4	1	10^{-3}	8	2.790035	0.348754

For $K = 1$, PFE requires $K + 1 = 2$ right-hand side evaluations per outer step. PRK4 uses $K + 1 = 2$ inner steps in each of its four outer stages and therefore requires 8 right-hand side evaluations per outer step. Although PRK4K1 allows a larger stable outer step size than PFEK1, the stable outer step size per right-hand side evaluation, $\Delta t_{\max}$/#RHS, is larger for PFEK1 in this test. The corresponding stability regions are shown in Figure 4.2.

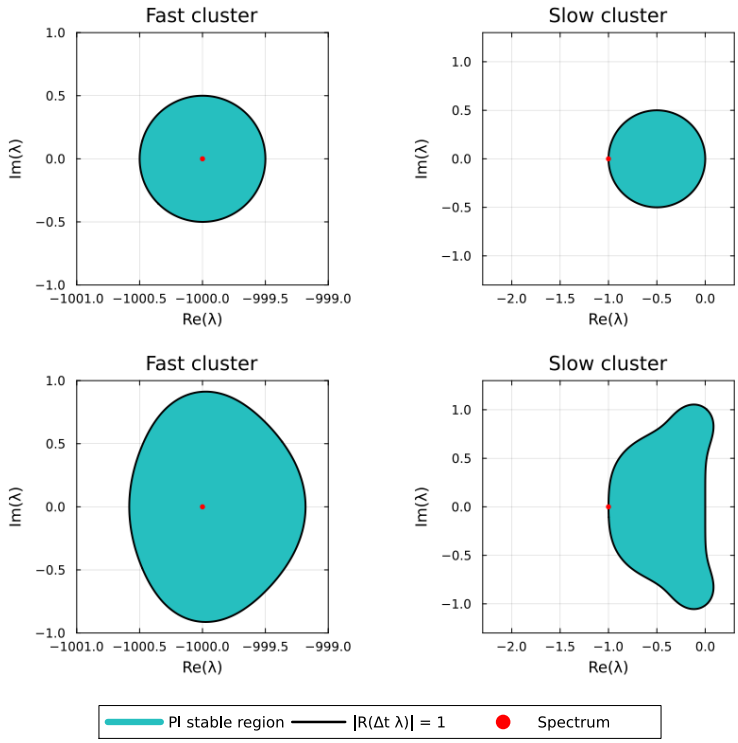


Figure 4.2: Stability regions for the isolated two-scale spectrum at the maximal stable outer step sizes. PFEK1 is shown in the top row and PRK4K1 in the bottom row.

4.1.2 Optimized Runge-Kutta methods

We now optimize stability polynomials for Λ_0. In this section, h denotes the step size returned by PolyOpt, while Δt denotes the outer step size of the PI methods.

The PolyOpt implementation used here requires an input list with at least $s+1$ entries for a stability polynomial of degree s. Since Λ_0 contains only two eigenvalues, each eigenvalue is repeated M times. This does not change the spectrum, but it changes the finite input used in the optimization.

For each polynomial basis, order p, number of stages s, and value of M, PolyOpt returns an optimized step size h and a corresponding stability polynomial R. We then evaluate

$$\max_{\lambda \in \Lambda_0} |R(h\lambda)| \tag{4.16}$$

to verify the stability of the returned polynomial on the given spectrum. In the tables below, $h_{\min}$ and $h_{\max}$ denote the smallest and largest step sizes returned by PolyOpt over all tested values of M.

$$(\max |R|)_{\max} \tag{4.17}$$

denotes the largest value of $\max_{\lambda \in \Lambda_0} |R(h\lambda)|$ over all tested values of M. Thus, $(\max |R|)_{\max} \leq 1$ verifies that all returned polynomials are stable at their corresponding step sizes. We use

$$M \in \{5, 10, 20, 50, 100, 200, 500\}. \tag{4.18}$$

Table 4.2: Minimum and maximum optimized step sizes over all tested values of M for $p = 1$.

Basis	p	s	$h_{\min}$	$h_{\max}$	$(\max \lvert R \rvert)_{\max}$
Monomial	1	3	141.19	1973.07	$2.3000 \cdot 10^{-7}$
Monomial	1	4	0.9805	119.61	$4.4818 \cdot 10^{-5}$
Monomial	1	5	0.1915	12.256	$2.0562 \cdot 10^{-7}$
Chebyshev	1	3	0.0012	321.02	0.9969
Chebyshev	1	4	2.0108	26773	0.9991
Chebyshev	1	5	2.0337	36926	0.9992
Chebinterp	1	3	18072	18072	$2.0416 \cdot 10^{-9}$
Chebinterp	1	4	32128	32128	$1.3547 \cdot 10^{-9}$
Chebinterp	1	5	50200	50200	$1.0520 \cdot 10^{-9}$

Table 4.2 summarizes the results for $p = 1$. The monomial and Chebyshev bases show a strong dependence on the choice of M. In contrast, the Chebyshev interpolation basis gives the same step size for all tested values of M. It also returns much larger step sizes than the other two bases.

However, these large first-order step sizes should be interpreted carefully. The returned polynomial is stable at the two eigenvalues at the returned step size, but this stability can be sensitive to small changes in h. For example, for the Chebyshev interpolation basis with $p = 1$ and $s = 3$, PolyOpt returns

$$h = 18072.$$

At this step size,

$$|R(-h)| \approx 1.020 \cdot 10^{-9}, \qquad |R(-1000h)| \approx 1.191 \cdot 10^{-20}. \tag{4.19}$$

But if the step size is slightly increased by $k = 1.8072 \cdot 10^{-6}$, then

$$\max_{\lambda \in \Lambda_0} |R((h+k)\lambda)| \approx 1.805. \tag{4.20}$$

To determine the size of the stable neighborhood of h, we perform a local search around it. This gives the stable interval

$$I_{\text{loc}} = \left[h - 1.000826 \cdot 10^{-6},\, h + 1.000826 \cdot 10^{-6}\right]. \tag{4.21}$$

To describe the change of the stability value on this interval, we define

$$F(\widehat{h}) = \max_{\lambda \in \Lambda_0} \left|R(\widehat{h}\lambda)\right|, \qquad \widehat{h} \in I_{\text{loc}}. \tag{4.22}$$

Since R is a polynomial and Λ_0 is finite, F is locally Lipschitz continuous. We estimate its local Lipschitz constant numerically by evaluating F at $N = 10000$ equally spaced points $\widehat{h}_1, \ldots, \widehat{h}_N$ in I_{loc} and computing

$$L = \max_{j=1,\ldots,N-1} \frac{\left|F(\widehat{h}_{j+1}) - F(\widehat{h}_j)\right|}{\widehat{h}_{j+1} - \widehat{h}_j}. \tag{4.23}$$

For this example, we obtain

$$L \approx 1.00 \cdot 10^{6}. \tag{4.24}$$

Thus, the small stability values at h do not imply stability for nearby step sizes. Even a very small change in h can move the scaled spectrum outside the stability region.

The results for $p = 2$ are shown in Table 4.3. For $s = 3$, all three bases give nearly identical results. For larger numbers of stages, the returned step sizes depend more strongly on the basis and on M. The monomial basis generally gives smaller values, whereas the Chebyshev basis remains close to $h \approx 2$. For the Chebyshev interpolation, h is also close to 2 for $s = 3$ and $s = 4$. For $s = 5$, however, PolyOpt returns a much larger value for one tested value of M. This suggests that the bisection may fail to find a larger step size.

Since the monomial basis generally gives smaller and less consistent step sizes in these tests, it is not used for the later construction of optimized Runge-Kutta methods. Tables 4.4 and 4.5 summarize the results for $p = 3$ and $p = 4$. For $p = 3$ the step size does not increase clearly with the number of stages and depends only weakly on M. For $p = 4$ the dependence on M is stronger. Nevertheless, the largest step sizes remain in a similar range for both orders.

Table 4.3: Minimum and maximum optimized step sizes over all tested values of M for $p = 2$.

Basis	p	s	$h_{\min}$	$h_{\max}$	$(\max \lvert R \rvert)_{\max}$
Monomial	2	3	2.0012	2.0012	0.9992
Monomial	2	4	0.3065	1.9610	$6.8137 \cdot 10^{-4}$
Monomial	2	5	0.0958	0.1557	0.9043
Chebyshev	2	3	2.0012	2.0012	0.9992
Chebyshev	2	4	1.7166	2.2083	0.9983
Chebyshev	2	5	1.7237	2.2778	0.9998
Chebinterp	2	3	2.0012	2.0012	0.9992
Chebinterp	2	4	2.0051	2.0051	0.9995
Chebinterp	2	5	2.0097	9421	0.9995

Table 4.4: Minimum and maximum optimized step sizes over all tested values of M for $p = 3$.

Basis	p	s	$h_{\min}$	$h_{\max}$	$\lvert R(h_{\max}\lambda_{\mathrm{s}}) \rvert$	$\lvert R(h_{\max}\lambda_{\mathrm{f}}) \rvert$
Chebyshev	3	4	2.5130	2.5130	0.9978	0.9976
Chebyshev	3	5	2.5084	2.5159	0.9984	0.0027
Chebyshev	3	6	2.5166	2.5209	0.9951	$1.3135 \cdot 10^{-4}$
Chebinterp	3	4	2.5130	2.5130	0.9978	0.0125
Chebinterp	3	5	2.5159	2.5159	0.9987	0.0025
Chebinterp	3	6	2.5188	2.5188	0.9986	0.0122

Table 4.5: Minimum and maximum optimized step sizes over all tested values of M for $p = 4$.

Basis	p	s	$h_{\min}$	$h_{\max}$	$\lvert R(h_{\max}\lambda_{\mathrm{s}}) \rvert$	$\lvert R(h_{\max}\lambda_{\mathrm{f}}) \rvert$
Chebyshev	4	5	2.7855	2.7855	0.9978	0.7968
Chebyshev	4	6	0.5515	2.2428	0.4442	0.0005
Chebyshev	4	7	1.1198	2.0664	0.3555	0.0002
Chebyshev	4	8	1.3629	2.3137	0.4877	0.0131
Chebinterp	4	5	1.4369	2.7442	0.6448	$4.1292 \cdot 10^{-7}$
Chebinterp	4	6	2.4994	2.7840	0.9922	$1.6178 \cdot 10^{-5}$
Chebinterp	4	7	1.4984	2.7914	0.9994	0.0007
Chebinterp	4	8	2.6971	2.7948	0.9998	0.0009

4.1.3 Additional grid-based line search

The previous results suggest that the PolyOpt bisection can miss larger feasible step sizes. This can happen if the feasible step sizes do not form one connected interval. In that case, the bisection may miss another stable region at larger values of h.

To investigate this, we choose a large value $M = 1200$ and perform an additional grid-based search over the step size. If the resulting step size is smaller than the largest value obtained in the previous tests, we repeat the search with the value of M for which this largest step size was obtained. At each grid point, we solve the polynomial optimization problem and classify the step size as feasible if

$$\max_{\lambda \in \Lambda_0} |R(h\lambda)| \leq 1. \tag{4.25}$$

The largest feasible value found in the tested range is denoted by h_{line}. For $p = 2$, the previous results indicate feasible step sizes in the range 10^3 to 10^4. We therefore use fixed search intervals around these values with a grid spacing of $\Delta h = 1$. For $p = 3$ and $p = 4$, no such range is known. We therefore choose the intervals relative to the largest bisection result $h_{\max}$:

$$I_1 = [h_{\max}, 2h_{\max}], \qquad I_2 = [2h_{\max}, 10h_{\max}],$$
$$I_3 = [10h_{\max}, 100h_{\max}], \qquad I_4 = [100h_{\max}, 1000h_{\max}].$$

Each interval contains $N_h = 201$ equally spaced test points. The largest feasible grid point over all tested subintervals is denoted by h_{line}. The results of the additional search are shown in Table 4.6.

Table 4.6: Comparison of the largest PolyOpt bisection results with the additional grid-based line search.

Basis	p	s	$h_{\max}$	Search range	h_{line}
Chebyshev	2	4	2.2083	$[1000, 3000]$, $\Delta h = 1$	none
Chebyshev	2	5	2.2778	$[9000, 13000]$, $\Delta h = 1$	none
Chebinterp	2	4	2.0051	$[1000, 3000]$, $\Delta h = 1$	2625
Chebinterp	2	5	9421	$[9000, 13000]$, $\Delta h = 1$	12725
Chebyshev	3	5	2.5159	$[h_{\max}, 10^3 h_{\max}]$	2.5159
Chebinterp	3	5	2.5159	$[h_{\max}, 10^3 h_{\max}]$	2.5159
Chebyshev	4	6	2.2428	$[h_{\max}, 10^3 h_{\max}]$	2.254
Chebinterp	4	6	2.7840	$[h_{\max}, 10^3 h_{\max}]$	2.7840

For $p = 2$, the additional line search finds larger feasible step sizes in some cases. This confirms that the bisection can miss another stable interval at larger values of h. It also explains why the PolyOpt results can vary with M, although the spectrum itself remains unchanged.

For the tested cases with $p = 3$ and $p = 4$, the grid search does not find a larger feasible step size in the considered ranges. As shown in Tables 4.4 and 4.5, in most of these

cases, the stability value at the slow eigenvalue is close to 1, while the value at the fast eigenvalue is much smaller. This indicates that the slow eigenvalue limits the step size of the polynomials returned by PolyOpt. Increasing the number of stages can improve the damping of the fast mode, but it does not necessarily increase h. Since additional stages also require more right-hand-side evaluations, we select the stable method with the larger value of $h/\#\text{RHS}$ for the later comparison.

4.1.4 Construction of Runge-Kutta methods with RK-Opt

After computing optimized stability polynomials with PolyOpt, we use RK-Opt to construct explicit $3S^*$ low-storage Runge-Kutta methods whose stability functions match the corresponding optimized polynomials. Each method is tested at the step size h returned by PolyOpt. A method is stable if

$$\max_{\lambda \in \Lambda_0} |R_{\text{RK}}(h\lambda)| \leq 1 + 10^{-8}. \tag{4.26}$$

We use the Chebyshev interpolation basis, because it gave the most reproducible results in the previous tests. The interpolation interval is $[z_{\min}, 0]$, where

$$z_{\min} = h \min_{\lambda \in \Lambda_0} \text{Re}(\lambda)$$

denotes the left endpoint of the scaled spectrum. We first place $s + 1$ Chebyshev interpolation points on this interval. For a method of order p with s stages, the order conditions leave $s - p$ degrees of freedom in the stability polynomial. We therefore select the $s - p$ interpolation points closest to $z_{\min} = h\lambda_{\text{f}}$. This choice is motivated by the fact that the fast eigenvalue is more difficult to capture and the behavior at $z = 0$ is already constrained by the order conditions. Alternative interpolation conditions are investigated in more detail in Section 4.4.

For each order, the stable method with the largest value $h/\#\text{RHS}$ is marked as selected. The results are summarized in Table 4.7.

Table 4.7: Optimized 3S* Runge-Kutta methods for $\Lambda_0 = \{-1, -1000\}$. The methods are tested at their corresponding PolyOpt step sizes h.

p	s	h	$h/\#\text{RHS}$	$\max \lvert R_{\text{RK}} \rvert$	Decision
1	3	18072	6024	$3.235 \cdot 10^{-7}$	Stable
1	4	32128	8032	$1.42 \cdot 10^{-6}$	Selected
2	4	2625	656.2	0.02316	Selected
2	5	12725	2545	$6.836 \cdot 10^{12}$	Unstable
3	4	2.513	0.6282	0.9978	Selected
3	5	2.516	0.5032	0.9987	Stable
4	5	2.744	0.5488	0.9374	Selected
4	6	2.7840	0.4640	0.9222	Stable

The methods marked as Selected are used for the comparison with the PI methods.

4.1.5 Comparison with Projective Integration methods

We now compare the selected optimized Runge-Kutta methods with the Projective Integration methods PFE and PRK4.

One difficulty in this comparison is the treatment of the final time T. For the optimized Runge-Kutta methods, the set of stable step sizes is not one connected interval. Therefore, a smaller remaining step size is not automatically stable. For this reason, only full time steps are used for these methods, and the error is evaluated at the resulting final time. This issue does not arise for the PI methods, since their final outer step can be adjusted to reach T exactly.

We use the maximum norm over the two components. Thus, for the numerical solution $u_{\text{num}}(T)$ and the exact solution $u_{\text{exact}}(T)$, the absolute error is defined by

$$e_{\text{abs}} = \|u_{\text{num}} - u_{\text{exact}}\|_\infty = \max_{i=1,2} |u_{\text{num},i} - u_{\text{exact},i}|. \tag{4.27}$$

The relative error is computed in the same norm,

$$e_{\text{rel}} = \frac{\|u_{\text{num}} - u_{\text{exact}}\|_\infty}{\|u_{\text{exact}}\|_\infty}. \tag{4.28}$$

The relative errors have to be interpreted carefully. Since we use large stable step sizes, the accuracy is not necessarily high. In addition, the exact solution of the two-scale test problem becomes very small for large final times. First, we consider $T = 2.8$. The results are shown in Table 4.8.

Table 4.8: Errors for the isolated two-scale test problem with final time $T = 2.8$.

Method	Step size	Step size/#RHS	abs. error	rel. error
RK $p = 3,\ s = 4$	2.513	0.6282	0.9899	150.6
RK $p = 4,\ s = 5$	2.744	0.5488	0.8755	211.6
PFE	2.002	1.001	0.2634	4.327
PRK4	2.790	0.3488	0.9302	15.28

PFE gives the smallest absolute error among the methods shown in the table. Although the optimized Runge-Kutta methods are stable at their optimized step sizes, their absolute errors are much larger than that of PFE. This shows that using the largest stable step size does not necessarily give an accurate result. The first- and second-order optimized methods are not included because their stable step sizes are much larger than the final time.

Next, we consider $T = 100$. The results are shown in Table 4.9. The exact solution is much smaller than 1, the relative errors are not useful for the comparison. We therefore

Table 4.9: Errors for the isolated two-scale test problem with final time $T = 100$.

Method	Step size	Step size/#RHS	abs. error
RK $p = 3,\ s = 4$	2.513	0.6282	0.9157
RK $p = 4,\ s = 5$	2.744	0.5488	0.0917
PFE	2.002	1.001	0.9010
PRK4	2.790	0.3488	0.5153

focus on the absolute errors. Most methods still give absolute errors of order one, while the fourth-order optimized method gives the smallest absolute error.

Finally, we choose $T = 32128$ so that the first-order optimized method can also perform one full time step. The results are shown in Table 4.10.

Table 4.10: Errors for the isolated two-scale test problem with final time $T = 32128$.

Method	Step size	Step size/#RHS	abs. error
RK $p = 1,\ s = 4$	32128	8032	$1.110 \cdot 10^{-4}$
RK $p = 2,\ s = 4$	2625	656.2	$1.267 \cdot 10^{-9}$
RK $p = 3,\ s = 4$	2.513	0.6282	$4.398 \cdot 10^{-13}$
RK $p = 4,\ s = 5$	2.744	0.5488	$1.136 \cdot 10^{-322}$
PFE	2.002	1.001	0.8890
PRK4	2.790	0.3488	0.4759

The comparatively large PI errors are mainly caused by the weak damping of the slow mode. The absolute value of the PI stability function is closer to one than that of the optimized Runge-Kutta methods. The optimized methods therefore damp the slow mode more strongly and produce smaller errors. The increase in the PI error for some larger final times is caused by the remaining outer step, whose size depends on the final time and therefore damps the solution differently.

So far, the PI methods have been tested at their maximal stable outer step sizes. We now study the effect of smaller outer step sizes on their accuracy using a work-precision diagram at $T = 100$, shown in Figure 4.3. From this diagram, we select points that combine small absolute errors with a low number of right-hand side evaluations. The corresponding step sizes and errors are listed in Table 4.11.

Table 4.11: Errors for the isolated two-scale test problem with final time $T = 100$ and reduced step sizes.

Method	Δt	Δt/#RHS	abs. error	rel. error
PFE	1.2	0.6	$3.724 \cdot 10^{-44}$	1
PRK4	1.12	0.14	$9.141 \cdot 10^{-43}$	24.55

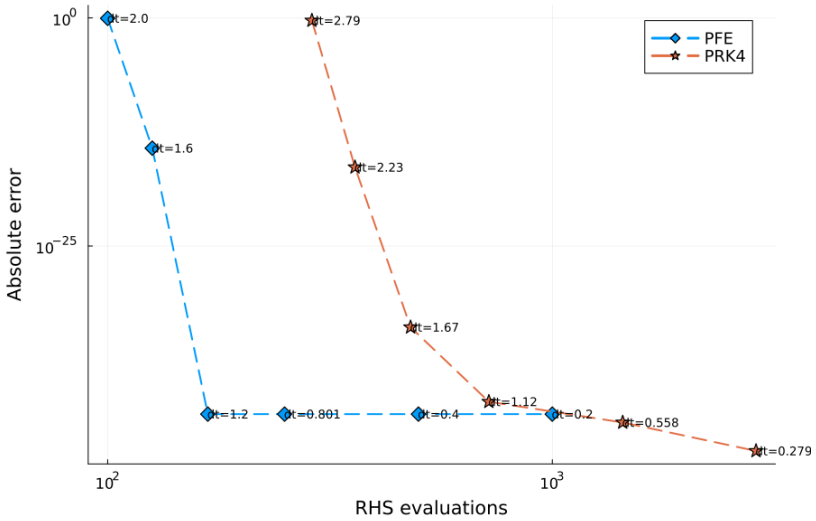


Figure 4.3: Work-precision diagram for PFE and PRK4 at $T = 100$, showing the absolute error against the number of right-hand side evaluations for different step sizes.

The methods have different advantages. The first-order optimized Runge-Kutta method has by far the largest value of $h/\#\text{RHS}$. The second-order optimized method also achieves a large value of $h/\#\text{RHS}$, while providing a higher order of accuracy. These methods are therefore useful when large stable step sizes are more important than accuracy. In contrast, the fourth-order optimized method gives the smallest absolute errors at the larger final times when the methods are used with their maximal stable step sizes.

The PI methods are more flexible because their outer step size Δt can be reduced when a smaller error is required. This can strongly reduce the error, but it also requires more right-hand side evaluations.

Therefore, the best method depends on the aim of the computation. The first-order optimized Runge-Kutta method gives the largest step size per right-hand side evaluation, while the fourth-order optimized method gives the smallest absolute errors when the methods are used with their maximal stable step sizes. The PI methods allow the outer step size to be reduced when higher accuracy is required.

4.2 Definition of the two-scale cluster spectrum

In the next experiments, we replace the two isolated eigenvalues by circular clusters around the slow and the fast eigenvalues. Since the cluster centers are at -1 and -1000, using the same absolute radius for both clusters would cause the slow cluster to extend into the right half-plane. We therefore choose the radii relative to the absolute values of

the corresponding eigenvalues. We consider the linear test problem

$$u'(t) = Au(t), \qquad u(0) = u_0, \tag{4.29}$$

with the diagonal matrix

$$A = \operatorname{diag}(\lambda_1, \dots, \lambda_N), \qquad A \in \mathbb{C}^{N \times N}, \tag{4.30}$$

where N is even and

$$\lambda_j = \begin{cases} -1 + r_{\mathrm{s}} e^{\mathrm{i}\theta_j}, & j = 1, \dots, \frac{N}{2}, \\ -1000 + r_{\mathrm{f}} e^{\mathrm{i}\theta_{j-N/2}}, & j = \frac{N}{2} + 1, \dots, N, \end{cases} \tag{4.31}$$

where

$$\theta_j = \frac{2\pi(j-1)}{N/2}, \qquad j = 1, \dots, \frac{N}{2}. \tag{4.32}$$

We denote the resulting cluster spectrum by

$$\Lambda = \{\lambda_1, \dots, \lambda_N\}. \tag{4.33}$$

The radii are defined using a relative cluster width $\kappa > 0$:

$$r_{\mathrm{slow}} = \kappa |\lambda_{\mathrm{slow}}|, \qquad r_{\mathrm{fast}} = \kappa |\lambda_{\mathrm{fast}}|. \tag{4.34}$$

Since the cluster centers are $\lambda_{\mathrm{slow}} = -1$ and $\lambda_{\mathrm{fast}} = -1000$, we obtain

$$r_{\mathrm{slow}} = \kappa, \qquad r_{\mathrm{fast}} = 1000\kappa. \tag{4.35}$$

Thus, the spectrum of A consists of two circular clusters centered at -1 and -1000, with radii r_{slow} and r_{fast}. For the time-integration experiments, we choose the initial value

$$u_0 = \mathbf{1} = (1, \dots, 1)^\top \in \mathbb{C}^N. \tag{4.36}$$

Since A is diagonal, the exact solution is given componentwise by

$$u_j(t) = \exp(\lambda_j t) u_j(0), \qquad j = 1, \dots, N. \tag{4.37}$$

In all cluster experiments, we use $N = 600$. Thus, each circular cluster contains 300 equally spaced eigenvalues. The resulting spectrum is used as input for the stability-polynomial optimization.

4.2.1 Projective Integration methods

We now apply the Projective Integration methods to the two-scale cluster spectrum. In the isolated two-eigenvalue case, the inner time step was chosen as $\delta t = \varepsilon$, since one inner Forward Euler step eliminates the fast eigenvalue $-1/\varepsilon$ exactly. For the cluster spectrum, this is no longer true. We therefore compute which inner step size gives the strongest damping over the whole fast cluster. For one inner Forward Euler step applied to the scalar test equation

$$u'(t) = \lambda u(t), \tag{4.38}$$

the update is given by

$$u^{n+1} = R_{\mathrm{FE}}(\lambda) u^n, \tag{4.39}$$

where the stability function is given by

$$R_{\mathrm{FE}}(\lambda) = 1 + \delta t\, \lambda. \tag{4.40}$$

The fast cluster is given by

$$\lambda = -\frac{1}{\varepsilon} + r e^{\mathrm{i}\theta}, \qquad \theta \in [0, 2\pi), \tag{4.41}$$

where, for simplicity, we write $r = r_{\mathrm{fast}}$. To determine the inner step size that gives the strongest damping, we write δt as a multiple of the fast time scale,

$$\delta t = x\varepsilon, \qquad x > 0. \tag{4.42}$$

Substituting this into the Forward Euler factor gives

$$R_{\mathrm{FE}}(\lambda) = 1 + x\varepsilon \left(-\frac{1}{\varepsilon} + r e^{\mathrm{i}\theta} \right) \tag{4.43}$$

$$= 1 - x + x\varepsilon r e^{\mathrm{i}\theta}. \tag{4.44}$$

Hence, the largest absolute value of the stability function over the fast cluster is

$$\max_{\theta \in [0,2\pi)} \left| 1 - x + x\varepsilon r e^{\mathrm{i}\theta} \right| = |1 - x| + x\varepsilon r. \tag{4.45}$$

For all cluster radii considered below, $\varepsilon r_{\mathrm{fast}} < 1$. Under this condition, the expression above is minimized at $x = 1$. Indeed, for $0 < x \leq 1$,

$$|1 - x| + x\varepsilon r = 1 - x + x\varepsilon r \tag{4.46}$$

$$= 1 - (1 - \varepsilon r)x, \tag{4.47}$$

which decreases as x increases. For $x \geq 1$,

$$|1 - x| + x\varepsilon r = x - 1 + x\varepsilon r \tag{4.48}$$

$$= (1 + \varepsilon r)x - 1, \tag{4.49}$$

which increases as x increases. Thus, the minimum is attained at $x = 1$, and therefore

$$\delta t = \varepsilon. \tag{4.50}$$

The calculation above determines the inner step size δt, but not the number of inner steps $K + 1$. In the isolated case, $K = 1$ was sufficient because the fast eigenvalue is already eliminated by one inner Forward Euler step. For the cluster spectrum, this is no longer true.

Increasing K gives more inner damping of the fast modes and can therefore allow a larger stable outer step size. However, it also increases the number of right-hand side evaluations. We therefore compute the maximal stable outer step size $\Delta t_{\max}$ for each method and each tested value of K. For each method, we select the value of K with the largest value of $\Delta t_{\max}/\#\text{RHS}$.

4.3 Two-scale cluster spectrum with fast radius $r_{\text{fast}} = 5$

We first consider the cluster spectrum with $r_{\text{fast}} = 5$. This corresponds to

$$\kappa = 0.005, \qquad r_{\text{slow}} = 0.005. \tag{4.51}$$

In the following, we first consider the Projective Integration methods and then compare them with the optimized Runge-Kutta methods.

4.3.1 Projective Integration methods

We apply PFE and PRK4 to the cluster spectrum with $r_{\text{fast}} = 5$. We use the inner step size $\delta t = \varepsilon = 10^{-3}$. The resulting stability limits are shown in Table 4.12.

Table 4.12: Maximal stable outer step sizes for PFE and PRK4 for the cluster spectrum with $r_{\text{fast}} = 5$.

Method	K	δt	#RHS	$\Delta t_{\max}$	$\Delta t_{\max}/\#\text{RHS}$
PFE	1	10^{-3}	2	0.201000	0.100500
PFE	2	10^{-3}	3	1.994053	0.664684
PFE	3	10^{-3}	4	1.996056	0.499014
PRK4	1	10^{-3}	8	0.326716	0.040839
PRK4	2	10^{-3}	12	2.779945	0.231662
PRK4	3	10^{-3}	16	2.783713	0.173982

The larger fast cluster requires more inner damping. Unlike for the isolated spectrum, $K = 1$ no longer gives the largest value of $\Delta t_{\max}/\#\text{RHS}$. For both PFE and PRK4,

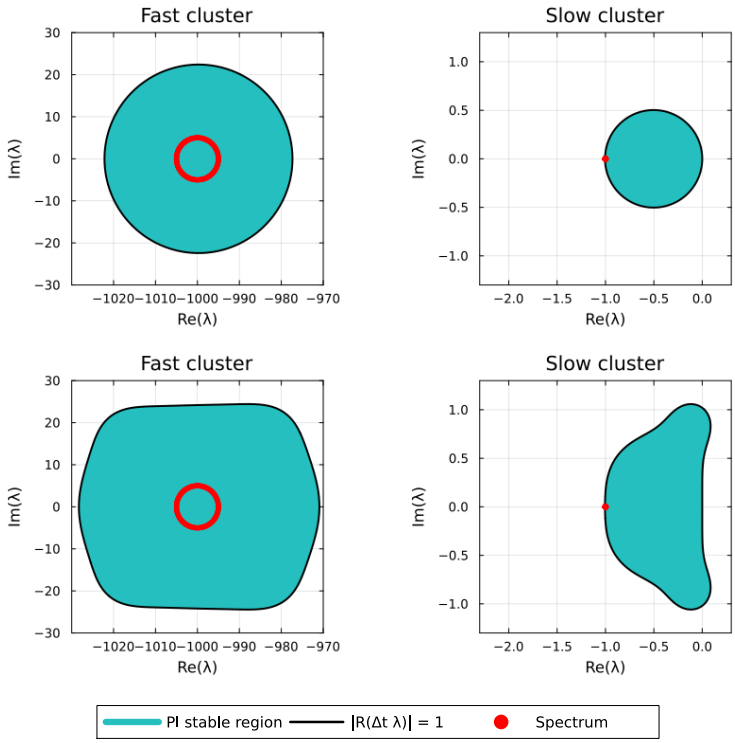


Figure 4.4: Stability regions for the cluster spectrum with $r_{\text{fast}} = 5$ at the maximal stable outer step sizes. PFEK2 is shown in the top row and PRK4K2 at the bottom row.

this value is largest for $K = 2$. We therefore use PFEK2 and PRK4K2 in the following comparison. The corresponding stability regions are shown in Figure 4.4.

To illustrate the effect of K, Figure 4.5 compares PFEK1 and PFEK2 at the maximal stable outer step size of PFEK2, $\Delta t = 1.994053$. At this step size, the fast cluster is not fully contained in the stability region of PFEK1, whereas it is fully contained in the stability region of PFEK2. Thus, the additional inner step provides enough damping for the fast cluster and allows a larger stable outer step size.

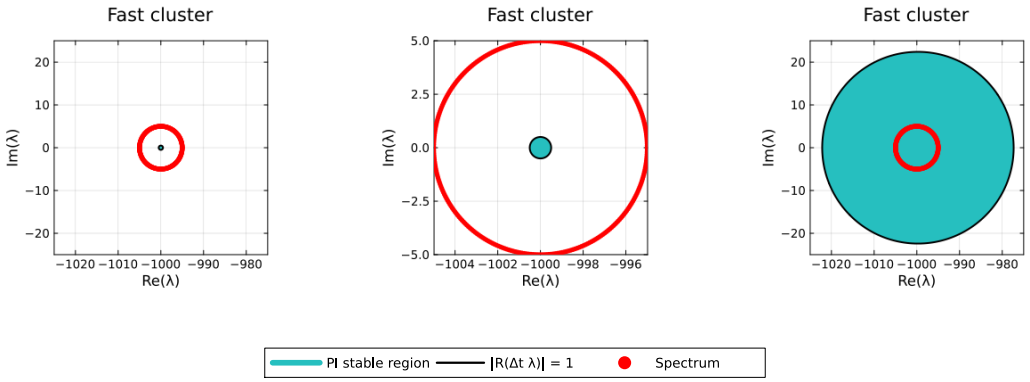


Figure 4.5: Comparison of the fast stability regions of PFEK1 and PFEK2 for the cluster spectrum with $r_{\text{fast}} = 5$ at the maximal stable outer step size of PFEK2. From left to right: PFEK1, a zoom of PFEK1, and PFEK2.

4.3.2 Optimized Runge-Kutta methods

We now consider the stability-polynomial optimization for the cluster spectrum with $r_{\text{fast}} = 5$. Here, M denotes the number of points used to sample the cluster boundaries in PolyOpt and is independent of the $N = 600$ eigenvalues used in the time-integration tests. A larger value of M gives a finer representation of the spectrum and reduces the risk of missing relevant parts of the cluster boundaries. We therefore use larger values of M than in the isolated two-point case.

$$M \in \{300, 500, 1000, 2000, 3000, 4000\}. \tag{4.52}$$

As in the isolated case, $h_{\min}$ and $h_{\max}$ denote the minimum and maximum optimized step sizes over all tested values of M. Furthermore,

$$(\max |R|)_{\max} \tag{4.53}$$

denotes the largest value of $\max_{\lambda \in \Lambda} |R(h\lambda)|$ over all tested values of M. The results for $p = 1$ are shown in Table 4.13.

Table 4.13: Minimum and maximum optimized step sizes over all tested values of M for $p = 1$.

Basis	p	s	$h_{\min}$	$h_{\max}$	$(\max \lvert R \rvert)_{\max}$
Chebyshev	1	5	1.9069	9.9815	0.99945
Chebyshev	1	6	6.6925	167.140	0.82238
Chebyshev	1	7	86.0428	202.787	0.99875
Chebinterp	1	5	9.98152	9.98152	0.99945
Chebinterp	1	6	202.787	202.787	0.99990
Chebinterp	1	7	203.142	203.142	0.99988

The Chebyshev interpolation basis gives the same step sizes for all tested values of M. Therefore, no additional grid-based search is performed for the first-order methods. The results for $p = 2$ are shown in Table 4.14.

Table 4.14: Minimum and maximum optimized step sizes over all tested values of M for $p = 2$.

Basis	p	s	$h_{\min}$	$h_{\max}$	$(\max \lvert R \rvert)_{\max}$
Chebyshev	2	10	2.0227	2.9529	0.99994
Chebyshev	2	11	2.0305	18.7647	0.99955
Chebyshev	2	12	2.0415	2.1716	0.99977
Chebinterp	2	10	7.7118	18.6987	0.97368
Chebinterp	2	11	10.8319	20.5130	0.99288
Chebinterp	2	12	12.6915	20.7885	0.93077

The results depend more strongly on M. The large range of returned step sizes suggests that the PolyOpt bisection may miss another stable interval. Since the Chebyshev interpolation basis gives the largest values of $h_{\max}$, we use this basis for the additional grid-based search in the next section. The results for $p = 3$ are shown in Table 4.15.

Table 4.15: Minimum and maximum optimized step sizes over all tested values of M for $p = 3$.

Basis	p	s	$h_{\min}$	$h_{\max}$	$(\max \lvert R \rvert)_{\max}$
Chebyshev	3	6	0.3673	0.3673	0.99943
Chebyshev	3	7	0.5652	2.1298	0.99924
Chebyshev	3	8	2.0497	2.5005	0.99964
Chebyshev	3	9	1.2276	2.5064	0.98911
Chebinterp	3	6	0.3673	0.3673	0.99943
Chebinterp	3	7	2.1298	2.1298	0.99924
Chebinterp	3	8	2.5044	2.5082	0.99974
Chebinterp	3	9	2.5113	2.5113	0.99960

The Chebyshev interpolation basis gives almost the same step size for all tested values of M. Therefore, no additional grid-based search is performed for the third-order methods.

The results for $p = 4$ are shown in Table 4.16.

Table 4.16: Minimum and maximum optimized step sizes over all tested values of M for $p = 4$.

Basis	p	s	$h_{\min}$	$h_{\max}$	$(\max \lvert R \rvert)_{\max}$
Chebyshev	4	8	0.2309	0.4476	0.99997
Chebyshev	4	9	0.4650	1.5896	0.99567
Chebyshev	4	10	0.4981	1.5411	0.99722
Chebinterp	4	8	0.4476	0.4476	0.99996
Chebinterp	4	9	1.6482	1.6701	0.99857
Chebinterp	4	10	1.5953	1.6901	0.99965

The Chebyshev interpolation basis gives the largest value of $h_{\max}$ for all tested numbers of stages. However, the returned step sizes still depend on M for all tested values of s. We therefore perform an additional grid-based search for fourth-order methods.

4.3.3 Additional grid-based line search

As in the isolated case, the PolyOpt results can depend on M when the feasible step sizes do not form one connected interval. The bisection can then miss another stable interval at larger values of h.

We therefore perform an additional grid-based search for the second- and fourth-order Chebyshev interpolation polynomials. For $p = 2$, we use $M = 7000$. For $p = 4$, we use $M = 4000$, since larger values require too much memory.

For each tested value of h, the PolyOpt problem is solved with fixed step size. A step size is called feasible if

$$\max_{\lambda \in \Lambda} |R(h\lambda)| \leq 1. \tag{4.54}$$

As in the isolated case, each search range is divided into subintervals of width Δh. We test $N_h = 50$ equally spaced grid points in each subinterval. For $p = 2$, $s = 12$, we use $N_h = 100$, since no feasible point was found with $N_h = 50$. The largest feasible grid point over all tested subintervals is denoted by h_{line}. The results are shown in Table 4.17.

Table 4.17: Comparison of the largest PolyOpt bisection results with the additional grid-based line search.

Basis	p	s	$h_{\max}$	Search range	h_{line}
Chebinterp	2	10	18.69	$[1, 31]$, $\Delta h = 5$	22.22
Chebinterp	2	11	20.51	$[1, 31]$, $\Delta h = 5$	27.43
Chebinterp	2	12	20.7885	$[26, 36]$, $\Delta h = 5$	27.919
Chebinterp	4	8	0.4476	$[0, 3]$, $\Delta h = 1$	0.4285
Chebinterp	4	9	1.670	$[1, 3]$, $\Delta h = 1$	1.653
Chebinterp	4	10	1.690	$[1, 3]$, $\Delta h = 1$	2.551

The additional search gives larger feasible step sizes in most tested cases. The case $p = 2$, $s = 12$ is not considered in the RK-Opt construction, since its value of $h_{\text{line}}/\#\text{RHS}$ is smaller than that for $p = 2$, $s = 11$. The remaining values of h_{line} are used for the RK-Opt construction below.

4.3.4 Construction of Runge-Kutta methods with RK-Opt

After computing optimized stability polynomials with PolyOpt, we use RK-Opt to construct explicit low-storage $3S^*$ Runge-Kutta methods whose stability functions match the corresponding optimized polynomials. We use the same interpolation setup as in the isolated two-point experiment.

For the first- and third-order methods, we use the step sizes returned by the PolyOpt bisection. For the second- and fourth-order candidates, we use the line-search result whenever it gives a larger feasible step size. Otherwise, we retain the bisection result. As in the isolated case, a method is accepted as stable if

$$\max_{\lambda \in \Lambda} |R_{\text{RK}}(h\lambda)| \leq 1 + 10^{-8}. \tag{4.55}$$

For each order, the stable method with the largest value of $h/\#\text{RHS}$ is marked as selected. The results are summarized in Table 4.18.

Table 4.18: Optimized $3S^*$ Runge-Kutta methods for the cluster spectrum with $r_{\text{fast}} = 5$. The methods are tested at their corresponding PolyOpt step sizes h.

p	s	h	$h/\#\text{RHS}$	$\max \lvert R_{\text{RK}} \rvert$	Decision
1	6	202.8	33.80	0.99992	Selected
1	7	203.1	29.02	0.9944	Stable
2	10	22.22	2.222	0.9315	Stable
2	11	27.43	2.494	0.1061	Selected
3	7	2.130	0.3043	0.99946	Stable
3	8	2.508	0.3135	0.99924	Selected
3	9	2.511	0.2790	0.9973	Stable
4	9	1.670	0.1856	0.985	Selected
4	10	2.551	0.2551	126.67	Unstable

The methods marked as Selected are used for the comparison with the PI methods.

4.3.5 Comparison with Projective Integration methods

We now compare the selected optimized Runge-Kutta methods with PFEK2 and PRK4K2. The absolute and relative errors are computed in the maximum norm over all components. We use the same treatment of the final time as in the isolated case. For the optimized

Runge-Kutta methods, only full time steps are used, and the error is evaluated at the resulting final time.

First, we choose $T = 27.4$ so that the second-order optimized Runge-Kutta method performs one full time step. The first-order optimized method is not included because its step size is much larger than this final time. The results are shown in Table 4.19.

Table 4.19: Errors for the two-scale cluster test problem with $r_{\text{fast}} = 5$ and final time $T = 27.4$.

Method	Step size	Step size/#RHS	abs. error	rel. error
RK $p = 2,\ s = 11$	27.43	2.494	0.1061	$7.555 \cdot 10^{10}$
RK $p = 3,\ s = 8$	2.508	0.3135	0.9917	$8.299 \cdot 10^{11}$
RK $p = 4,\ s = 9$	1.670	0.1856	0.9928	$1.843 \cdot 10^{12}$
PFEK2	1.994	0.6647	0.4825	$3.339 \cdot 10^{11}$
PRK4K2	2.780	0.2317	0.5465	$3.782 \cdot 10^{11}$

The second-order optimized Runge-Kutta method gives both the largest value of $h/\#\text{RHS}$ and the smallest absolute error among the methods shown in the table.

Next, we choose $T = 202$ so that the first-order optimized method can also perform one full time step. Since the norm of the exact solution is much smaller than one, the relative errors are not useful for the comparison. The results are shown in Table 4.20.

Table 4.20: Errors for the two-scale cluster test problem with $r_{\text{fast}} = 5$ and final time $T = 202$.

Method	Step size	Step size/#RHS	abs. error
RK $p = 1,\ s = 6$	202.8	33.80	0.9999
RK $p = 2,\ s = 11$	27.43	2.494	$1.607 \cdot 10^{-8}$
RK $p = 3,\ s = 8$	2.508	0.3135	0.9404
RK $p = 4,\ s = 9$	1.670	0.1856	0.9463
PFEK2	1.994	0.6647	0.3967
PRK4K2	2.780	0.2317	0.2932

At this final time, the first-order optimized Runge-Kutta method gives the largest value of $h/\#\text{RHS}$, while the second-order method gives the smallest absolute error.

Cluster-wise error comparison. For the PI methods, the inner steps strongly damp the fast modes. Their error is therefore mainly determined by the slow cluster. We now examine whether the same behavior is observed for the optimized Runge-Kutta methods.

Let Λ_s and Λ_f denote the slow and fast clusters. For each selected optimized method, we compute the maximal error on both clusters at final time T,

$$e_s = \max_{\lambda_j \in \Lambda_s} |u_{j,\mathrm{num}}(T) - u_j(T)|, \tag{4.56}$$

and

$$e_f = \max_{\lambda_j \in \Lambda_f} |u_{j,\mathrm{num}}(T) - u_j(T)|. \tag{4.57}$$

To relate the errors to the damping of the two clusters, we also consider

$$R_{s,\max} = \max_{\lambda \in \Lambda_s} |R_{\mathrm{RK}}(h\lambda)|, \qquad R_{f,\max} = \max_{\lambda \in \Lambda_f} |R_{\mathrm{RK}}(h\lambda)|. \tag{4.58}$$

The results for $T = 202$ are shown in Table 4.21.

Table 4.21: Cluster-wise errors and stability values of the selected optimized Runge-Kutta methods for $r_{\mathrm{fast}} = 5$ and $T = 202$.

Method	e_s	e_f	$R_{s,\max}$	$R_{f,\max}$
RK $p = 1,\ s = 6$	0.9999	0.9700	0.999923	0.96999
RK $p = 2,\ s = 11$	$1.607 \cdot 10^{-8}$	$7.104 \cdot 10^{-10}$	0.1061	0.0719
RK $p = 3,\ s = 8$	0.9404	$6.708 \cdot 10^{-6}$	0.99924	0.69927
RK $p = 4,\ s = 9$	$2.568 \cdot 10^{-69}$	0.9463	0.2711	0.9848

For the first-, second-, and third-order methods, the error is determined by the slow cluster. In all three cases, $R_{s,\max}$ is larger than $R_{f,\max}$, so the slow modes are damped less strongly.

For the fourth-order method, the situation is reversed. The stability value is much larger on the fast cluster, and the fast cluster determines the total error.

Hence, unlike for the PI methods, the error of an optimized Runge-Kutta method is not always determined by the slow cluster. It depends on how the stability function obtained from the optimization behaves on the two parts of the spectrum. In many of the other optimization results, the largest stability value is attained on the slow cluster, so the stability is often limited by the slow part of the spectrum. However, the results above show that this is not a general rule.

So far, the PI methods have been tested at their maximal stable outer step sizes. We now study the effect of smaller outer step sizes on their accuracy using a work-precision diagram at $T = 202$, shown in Figure 4.6. From this diagram, we select points that combine small absolute errors with a low number of right-hand side evaluations. The corresponding outer step sizes and errors are listed in Table 4.22.

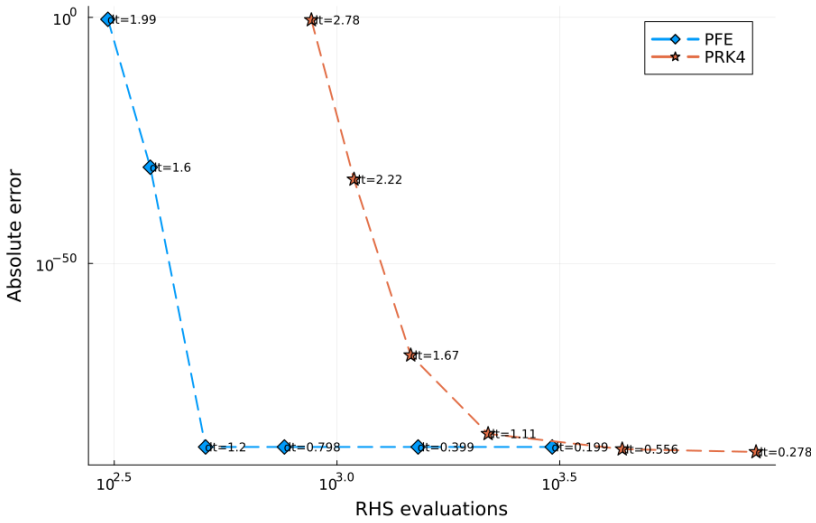


Figure 4.6: Work-precision diagram for PFEK2 and PRK4K2 at $T = 202$, showing the absolute error against the number of right-hand side evaluations for different step sizes.

Table 4.22: Errors for the two-scale cluster test problem with $r_{\text{fast}} = 5$, final time $T = 202$, and reduced step sizes.

Method	Δt	$\Delta t/\#\text{RHS}$	abs. error	rel. error
PFEK2	1.200	0.4	$5.142 \cdot 10^{-88}$	1
PRK4K2	1.110	0.0925	$2.345 \cdot 10^{-85}$	456.1

The first-order optimized Runge-Kutta method gives by far the largest value of $h/\#\text{RHS}$, while the second-order optimized method gives the smallest absolute error at both final times when all methods are used with their maximal stable step sizes.

The PI methods are more flexible because their outer step size Δt can be reduced when a smaller error is required. This can strongly reduce the error, but it also requires more right-hand side evaluations.

Therefore, the best method again depends on the aim of the computation. The first-order optimized Runge-Kutta method gives the largest step size per right-hand side evaluation, while the second-order optimized method gives the smallest absolute errors when the methods are used with their maximal stable step sizes. The PI methods allow the outer step size to be reduced when higher accuracy is required.

4.4 Two-scale cluster spectrum with fast radius $r_{\text{fast}} = 50$

We now increase the fast cluster radius to $r_{\text{fast}} = 50$. This corresponds to

$$\kappa = 0.05, \qquad r_{\text{slow}} = 0.05.$$

Thus, compared with the previous test, the fast cluster is much wider. In the following, we first consider the Projective Integration methods and then compare them with the optimized Runge-Kutta methods.

4.4.1 Projective Integration methods

We apply PFE and PRK4 to the cluster spectrum with $r_{\text{fast}} = 50$. We again use the inner step size $\delta t = \varepsilon = 10^{-3}$. The resulting stability limits are shown in Table 4.23.

Table 4.23: Maximal stable outer step sizes for PFE and PRK4 for the cluster spectrum with $r_{\text{fast}} = 50$.

Method	K	δt	#RHS	$\Delta t_{\max}$	$\Delta t_{\max}/\#\text{RHS}$
PFE	2	10^{-3}	3	0.383906	0.127969
PFE	3	10^{-3}	4	1.910773	0.477693
PFE	4	10^{-3}	5	1.912777	0.382555
PRK4	2	10^{-3}	12	0.652809	0.054401
PRK4	3	10^{-3}	16	2.664944	0.166559
PRK4	4	10^{-3}	20	2.668714	0.133436

For $r_{\text{fast}} = 50$, stronger inner damping is required. Unlike for $r_{\text{fast}} = 5$, $K = 3$ gives the largest value of $\Delta t_{\max}/\#\text{RHS}$ for both PFE and PRK4. We therefore use PFEK3 and PRK4K3 in the following comparison. The corresponding stability regions are shown in Figure 4.7.

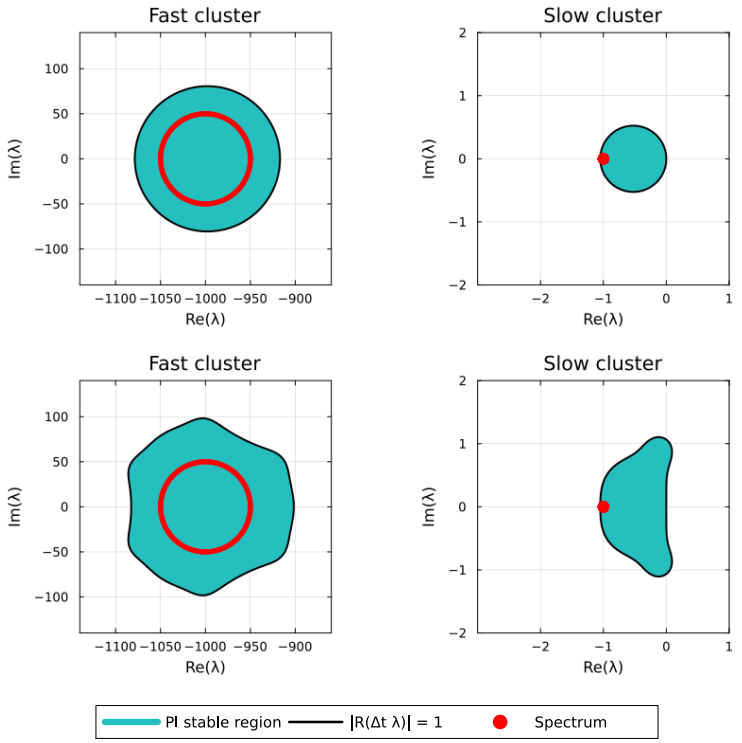


Figure 4.7: Stability regions for the cluster spectrum with $r_{\text{fast}} = 50$ at the maximal stable outer step sizes. PFEK3 is shown in the top row and PRK4K3 in the bottom row.

4.4.2 Optimized Runge-Kutta methods

We now consider the optimized Runge-Kutta methods for the cluster spectrum with $r_{\text{fast}} = 50$. In contrast to the previous cluster experiment, we do not investigate the dependence on M further. Since further increases in M no longer changed the optimized step sizes, we fix $M = 300$ sampling points on each cluster boundary, corresponding to $N = 600$ eigenvalues in total. This fixed spectrum is used in PolyOpt and for all stability tests. For better visibility, the stability plots show only 30 equally spaced eigenvalues from each cluster.

4.4.3 $3S^*$ methods with the Chebyshev basis

We first use RK-Opt to construct $3S^*$ low-storage Runge-Kutta methods from the stability polynomials optimized in the Chebyshev basis. The methods are tested at the corresponding step sizes h obtained from PolyOpt. A method is accepted if

$$\max_{\lambda \in \Lambda} |R_{\text{RK}}(h\lambda)| \leq 1 + 10^{-8}. \tag{4.59}$$

For each order, the stable method with the largest value of $h/\#\text{RHS}$ among the methods in this table is marked as Selected. The results are shown in Table 4.24.

Table 4.24: Optimized $3S^*$ Runge-Kutta methods using the Chebyshev basis for the cluster spectrum with $r_{\text{fast}} = 50$. The methods are tested at their corresponding PolyOpt step sizes h.

p	s	h	$h/\#\text{RHS}$	$\max \lvert R_{\text{RK}} \rvert$	Decision
1	7	5.175	0.7392	1.000037	Local grid search
1	8	22.11	2.763	0.99933	Selected
1	9	22.17	2.464	0.9994	Stable
2	6	0.5717	0.09529	0.9987	Stable
2	7	1.914	0.2734	0.991	Selected
2	8	1.917	0.2396	12.77	Unstable
3	8	0.2742	0.03428	0.99795	Selected
3	9	0.7355	0.08173	1.004822	Local grid search
3	10	1.984	0.1984	40.82	Unstable
4	10	0.2054	0.02054	0.9989	Stable
4	11	0.4276	0.03887	0.9999985	Selected
4	12	0.8916	0.07430	$9.998 \cdot 10^8$	Unstable

For $p = 4$, RK-Opt was run 7 times for $s = 11$, and a stable method was obtained. For $s = 12$, 17 runs were performed, but none returned a stable method.

The optimized step sizes again do not increase uniformly with the number of stages. For $p = 1$, the step size changes only slightly from $s = 8$ to $s = 9$, and for $p = 2$, the values

for $s = 7$ and $s = 8$ are nearly equal. This is consistent with the effect observed for the isolated spectrum in Section 4.1. Additional stages can improve the damping of the fast part of the spectrum without an increase in the maximal stable step size.

For the methods marked as Local grid search in Table 4.24, we perform a local grid search around the corresponding PolyOpt step size. For each fixed RK method, we evaluate its stability function at 30001 equally spaced values of $\widetilde{h}$ in the interval $[0.999h, 1.005h]$. A tested step size $\widetilde{h}$ is classified as stable if

$$\max_{\lambda \in \Lambda} \left| R_{\mathrm{RK}}(\widetilde{h}\lambda) \right| \leq 1. \tag{4.60}$$

The results are shown in Table 4.25.

Table 4.25: Local grid search for the $3S^*$ methods using the Chebyshev basis. Here, h denotes the PolyOpt step size, while h_{loc} denotes the largest stable step size found in the tested interval.

p	s	h	$\max \lvert R_{\mathrm{RK}}(h\lambda)\rvert$	h_{loc}	$\max \lvert R_{\mathrm{RK}}(h_{\mathrm{loc}}\lambda)\rvert$	Result
1	7	5.174652	1.000037	5.174673	0.99999	Stable
3	9	0.7355	1.004822	–	–	No stable step size

For $(p, s) = (1, 7)$, the local search finds a stable step size very close to the PolyOpt step size. For $(p, s) = (3, 9)$, no stable step size is found in the tested local interval. Therefore, this method is unstable near the PolyOpt step size.

As an additional test, we also run RK-Opt with the Chebyshev basis and nondecreasing stage nodes,

$$c_1 \leq c_2 \leq \cdots \leq c_s. \tag{4.61}$$

For fourth-order methods, RK-Opt converges only up to $s = 10$. The additional constraint therefore does not improve the results and is not considered further.

Evaluation of the stability function. For the RK method obtained from RK-Opt, we compare three ways to evaluate the stability function. For an RK method, all three are equivalent in exact arithmetic.

The first evaluation applies the $3S^*$ low-storage recursion to the scalar test equation

$$u'(t) = \lambda u(t). \tag{4.62}$$

With $z = h\lambda$ and $u^n = 1$, the registers are initialized by

$$S_1 := 1, \qquad S_2 := 0, \qquad S_3 := 1. \tag{4.63}$$

For $i = 2, \ldots, s + 1$, we get the recursion

$$\begin{aligned} S_2 &:= S_2 + \delta_{i-1} S_1, \\ S_1 &:= \gamma_{i,1} S_1 + \gamma_{i,2} S_2 + \gamma_{i,3} S_3 + \beta_{i,i-1} z S_1. \end{aligned} \tag{4.64}$$

After the final stage, $S_1 = u^{n+1}$. Since $u^{n+1} = R_{\mathrm{LS}}(z)u^n$ and $u^n = 1$, we obtain

$$R_{\mathrm{LS}}(z) = S_1. \tag{4.65}$$

This evaluation uses the low-storage coefficients directly and is the form used in the time integration.

The second evaluation uses the Butcher coefficients A and b,

$$R_{\mathrm{B}}(z) = 1 + zb^T(I - zA)^{-1}\mathbf{1}, \tag{4.66}$$

In the interpolation-based construction, this form is used to evaluate the stability function at the selected matching points.

The third evaluation writes the stability function as the power series

$$R_{\mathrm{P}}(z) = 1 + \sum_{j=1}^{s} \left(b^T A^{j-1}\mathbf{1}\right) z^j. \tag{4.67}$$

This form is used to match the coefficients of the target stability polynomial in RK-Opt. We now compare the three stability evaluations for the unstable methods with $(p, s) = (2, 8)$ and $(p, s) = (3, 10)$ from Table 4.24.

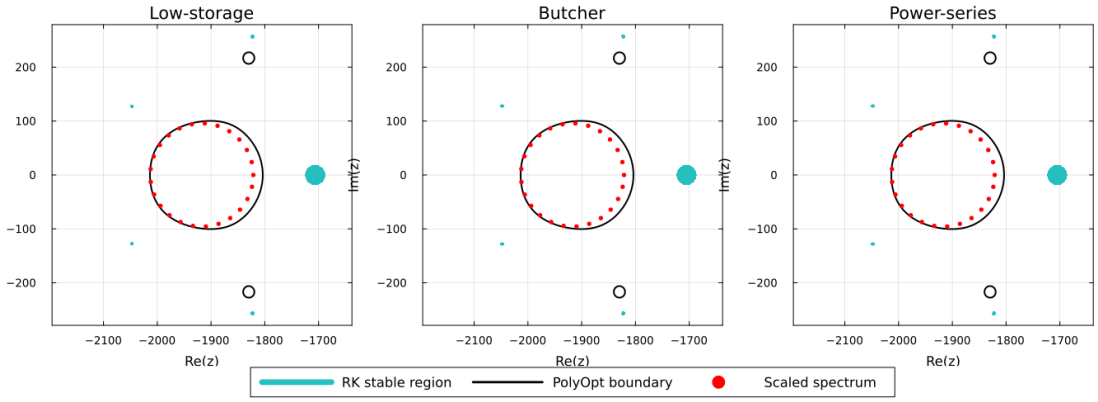


Figure 4.8: Stability regions obtained from the $3S^*$ low-storage evaluation (4.64), the Butcher evaluation (4.66), and the power-series evaluation (4.67) for the $3S^*$ method with $p = 2$ and $s = 8$, using the Chebyshev basis at the PolyOpt step size h.

Figure 4.8 shows that the stability polynomial computed by PolyOpt is stable on the scaled spectrum. However, the RK method obtained with RK-Opt is not stable at the PolyOpt step size, since its stability region does not contain the complete scaled spectrum.

For a quantitative comparison for this method, we compute

$$\max_{\lambda \in \Lambda_f} |R_X(h\lambda)|, \qquad X \in \{\mathrm{LS}, \mathrm{B}, \mathrm{P}\}. \tag{4.68}$$

The resulting values range from 12.60 to 12.77. Thus, all three evaluations clearly exceed the stability limit of one.

We next consider the unstable method with $(p, s) = (3, 10)$. Figure 4.9 shows the corresponding stability regions.

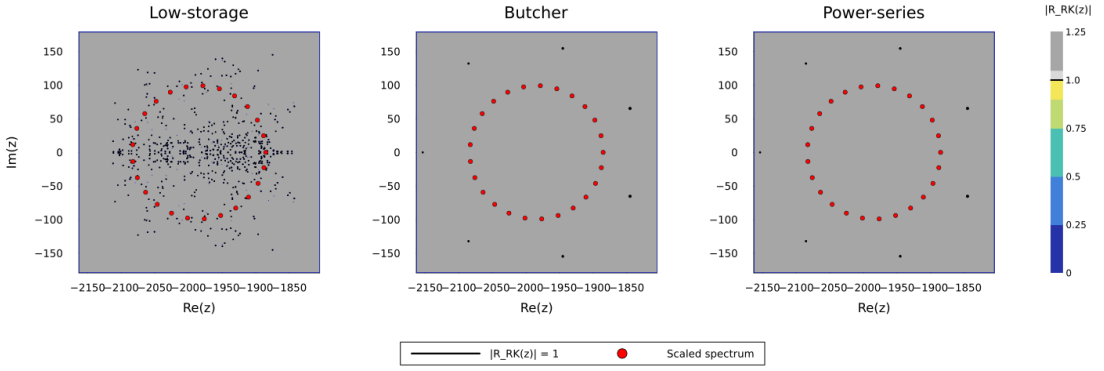


Figure 4.9: Pseudocolor plot of stability regions obtained from the low-storage evaluation (4.64), the Butcher evaluation (4.66), and the power-series evaluation (4.67) for the $3S^*$ method with $p = 3$ and $s = 10$, using the Chebyshev basis at the PolyOpt step size h.

In this case, the low-storage stability region differs visibly from the Butcher and power-series results. To quantify this difference, we compare the Float64 and 256-bit BigFloat results for each evaluation by computing the maximum pointwise relative difference

$$\max_k \frac{|R_{X,\mathrm{Float64}}(z_k) - R_{X,\mathrm{BigFloat}}(z_k)|}{|R_{X,\mathrm{BigFloat}}(z_k)|}. \tag{4.69}$$

The results are shown in Table 4.26.

Table 4.26: Maximum pointwise relative differences between Float64 and 256-bit BigFloat evaluations for the $3S^*$ method with $p = 3$ and $s = 10$ on the scaled fast cluster.

Evaluation	Maximum relative difference
Low-storage	4.885
Butcher	$1.611 \cdot 10^{-2}$
Power series	$1.919 \cdot 10^{-3}$

The low-storage evaluation has by far the largest difference between Float64 and BigFloat. This suggests that the visible differences in Figure 4.9 are mainly caused by floating-point effects in the low-storage recursion.

A possible explanation is cancellation in the low-storage recursion. On the fast cluster, $|z|$ is large, so the intermediate values in the recursion can become large. If these values cancel each other, small round-off errors can affect the final value of $R_{\mathrm{LS}}(z)$.

4.4.4 $2N$ methods

We next use RK-Opt to construct $2N$ low-storage Runge-Kutta methods from the stability polynomials optimized in the Chebyshev basis. As before, the methods are tested at their corresponding PolyOpt step sizes h. The results are summarized in Table 4.27.

Table 4.27: Optimized $2N$ Runge-Kutta methods using the Chebyshev basis for the cluster spectrum with $r_{\mathrm{fast}} = 50$. The methods are tested at their corresponding PolyOpt step sizes h.

p	s	h	$h/\#\mathrm{RHS}$	$\max \lvert R_{\mathrm{RK}} \rvert$	Decision
1	7	5.175	0.7392	0.99987	Stable
1	8	22.11	2.763	0.99933	Selected
1	9	22.17	2.464	$4.708 \cdot 10^{12}$	Unstable
2	7	1.914	0.2734	0.99910	Selected
2	8	1.917	0.2396	0.99980	Stable
2	9	1.925	0.2138	5.568	Unstable
3	8	0.2742	0.03428	0.9979	Stable
3	9	0.7355	0.08173	0.9956	Stable
3	10	1.984	0.1984	1.0021	Local grid search
4	8	0.04919	0.006148	0.9983	Stable
4	9	0.09974	0.01108	0.999954	Selected

For third order, the methods with $s = 8$ and $s = 9$ are stable at the PolyOpt step size. For $(p, s) = (3, 10)$, the stability value is slightly larger than one. We therefore apply the same local grid search as above. The results are shown in Table 4.28.

Table 4.28: Local grid search for the $2N$ method with $p = 3$ and $s = 10$ obtained using the Chebyshev basis. Here, h denotes the PolyOpt step size, while h_{loc} denotes the largest stable step size found in the tested interval.

p	s	h	$\max \lvert R_{\mathrm{RK}}(h\lambda) \rvert$	h_{loc}	$\max \lvert R_{\mathrm{RK}}(h_{\mathrm{loc}}\lambda) \rvert$	Result
3	10	1.9844206	1.0021	1.9844054	0.99999	Stable

Table 4.28 shows that a stable step size h_{loc} is found very close to the PolyOpt step size. The method has the largest value of $h_{\mathrm{loc}}/\#\mathrm{RHS}$ among the third-order $2N$ methods and

is therefore selected among these methods. For fourth order, RK-Opt converged only up to $s = 9$ in the tested runs.

For the $2N$ methods, we compare the same three evaluations as in the previous subsection. For the low-storage evaluation used in the plots below, we apply the $2N$ recursion to the scalar test equation. With $z = h\lambda$ and $u^n = 1$, the registers are initialized by

$$S_1 := 1, \qquad S_2 := 0. \tag{4.70}$$

For $i = 2, \ldots, s+1$, this gives

$$\begin{aligned} S_2 &:= \widetilde{A}_i S_2 + z S_1, \\ S_1 &:= S_1 + \widetilde{B}_i S_2. \end{aligned} \tag{4.71}$$

After the final stage, the low-storage stability function is

$$R_{\mathrm{LS}}(z) = S_1. \tag{4.72}$$

To illustrate an unstable $2N$ method, we consider $(p, s) = (2, 9)$. Figure 4.10 shows the same behavior as the $3S^*$ method in Figure 4.8. The stability polynomial computed by PolyOpt is stable on the scaled spectrum. However, the $2N$ method obtained with RK-Opt is not stable at the PolyOpt step size.

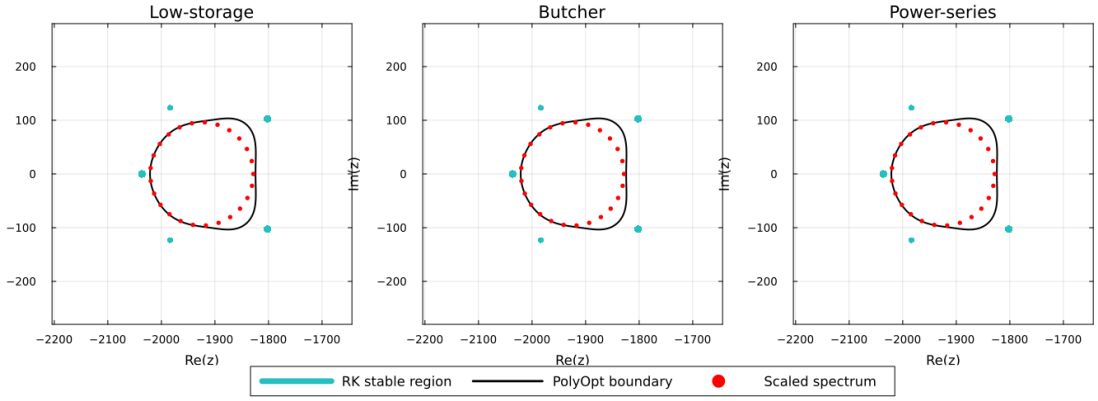


Figure 4.10: Stability regions obtained from the $2N$ low-storage evaluation (4.71), the Butcher evaluation (4.66), and the power-series evaluation (4.67) for the $2N$ method with $p = 2$ and $s = 9$, using the Chebyshev basis at the PolyOpt step size h.

As an additional test, we also run RK-Opt for $2N$ methods using the Chebyshev basis and nondecreasing stage nodes,

$$c_1 \le c_2 \le \cdots \le c_s. \tag{4.73}$$

The results are shown in Table 4.29.

Table 4.29: Optimized $2N$ Runge-Kutta methods using the Chebyshev basis and nondecreasing stage nodes for the cluster spectrum with $r_{\text{fast}} = 50$. The methods are tested at their PolyOpt step sizes h.

p	s	h	$h/\#\text{RHS}$	$\max \lvert R_{\text{RK}} \rvert$	Decision
1	7	5.175	0.7392	0.99982	Stable
1	8	22.11	2.763	0.99933	Selected
1	9	22.17	2.464	1.00496	Unstable
2	7	1.914	0.2734	0.99910	Selected
2	8	1.917	0.2396	0.99980	Stable
3	9	0.7355	0.08173	0.99956	Stable
3	10	1.984	0.1984	0.9986	Selected
4	8	0.04919	0.006148	0.9981	Stable
4	9	0.09974	0.01108	0.999960	Selected

The results are similar to those obtained without the additional constraint. For third order, the method with $(p, s) = (3, 10)$ is stable directly at the PolyOpt step size. For fourth order, RK-Opt again converged only up to $s = 9$. Thus, nondecreasing stage nodes do not give a general improvement.

We also tested the additional restriction

$$0 \le b_i \le 1, \qquad i = 1, \dots, s.$$

For fourth order, RK-Opt converged only up to $s = 5$, compared with $s = 9$ without the additional bounds. Therefore, this restriction is not considered further.

4.4.5 Chebyshev interpolation

We next use the Chebyshev interpolation conditions. As before, the interpolation interval is $[z_{\min}, 0]$, where

$$z_{\min} = h \min_{\lambda \in \Lambda} \text{Re}(\lambda)$$

denotes the left endpoint of the scaled spectrum. For a method with s stages, the target stability polynomial is represented by its values at $s + 1$ Chebyshev interpolation points on this interval. We first consider $s - p$ interpolation points closest to the fast cluster as matching conditions in RK-Opt.

The resulting methods are shown in Table 4.30. For $p = 4$, RK-Opt was run 7 times for $s = 13$, and a stable method was obtained. For $s = 14$, 15 runs were performed, but none returned a method that was stable at the PolyOpt step size.

Table 4.30: Optimized $3S^*$ Runge-Kutta methods using Chebyshev interpolation for the cluster spectrum with $r_{\text{fast}} = 50$. The methods are tested at their PolyOpt step sizes h.

p	s	h	$h/\#\text{RHS}$	$\max \lvert R_{\text{RK}} \rvert$	Decision
1	7	5.175	0.7392	0.9980	Stable
1	8	22.11	2.763	0.99933	Selected
2	7	1.914	0.2734	0.99909	Selected
2	8	1.917	0.2396	$1.796 \cdot 10^5$	Unstable
3	10	1.984	0.1984	0.9981	Stable
3	11	2.405	0.2186	0.99680	Selected
4	13	0.8583	0.06603	0.7943	Stable
4	14	1.617	0.1155	1.0074	Local grid search

For $(p, s) = (4, 14)$, we perform the same local grid search as before to determine whether the method is stable at a step size close to the PolyOpt value. The results are shown in Table 4.31.

Table 4.31: Local grid search for the $3S^*$ method with $p = 4$ and $s = 14$, using Chebyshev interpolation. Here, h denotes the PolyOpt step size, while h_{loc} denotes the largest stable step size found in the tested interval.

p	s	h	$\max \lvert R_{\text{RK}}(h\lambda) \rvert$	h_{loc}	$\max \lvert R_{\text{RK}}(h_{\text{loc}}\lambda) \rvert$	Result
4	14	1.617	1.0074	–	–	No stable step size

It shows that no stable step size is found in the tested local interval. Therefore, the method with $(p, s) = (4, 14)$ is unstable.

We also tested the interpolation-based matching conditions with the $2N$ low-storage formulation. However, the resulting construction problem was too restrictive. In the tested runs, RK-Opt did not return a fourth-order method even for $s = 5$. Therefore, this combination is not considered further.

Reduced number of interpolation conditions. Since $s - p$ interpolation values uniquely determine a polynomial of degree at most s, we now test the effect of using one fewer value. For $(p, s) = (4, 14)$, we use $s - p - 1 = 9$ instead of $s - p = 10$ interpolation points closest to the fast cluster.

The results are shown in Figure 4.11. It shows the stability regions for the method obtained with the reduced set of interpolation conditions. With only $s - p - 1$ matching conditions, the stability polynomial of the Runge-Kutta method is no longer guaranteed to match the target stability polynomial. The low-storage stability region also differs from the Butcher and power-series regions and does not contain the complete scaled spectrum.

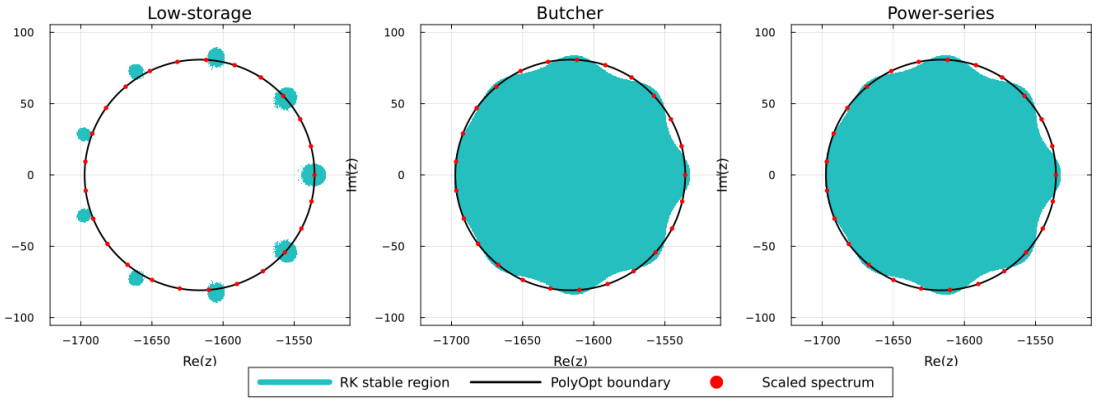


Figure 4.11: Stability regions obtained from the $3S^*$ low-storage evaluation (4.64), the Butcher evaluation (4.66), and the power-series evaluation (4.67) for the $3S^*$ method with $p = 4$ and $s = 14$, using $s - p - 1$ interpolation conditions near the fast cluster at the PolyOpt step size h.

Alternative distributions of interpolation conditions. We also test different distributions of the interpolation conditions. The points are either spread over the full interval $[z_{\min}, 0]$, divided equally between the slow and fast clusters, or placed only on the fast scale. The results are summarized in Table 4.32.

Table 4.32: Tested distributions of the Chebyshev interpolation conditions in the RK-Opt construction of fourth-order $3S^*$ methods. The methods are tested at their PolyOpt step sizes h.

p	s	Distribution	$\max \lvert R_{\mathrm{RK}} \rvert$	Decision
4	8	Spread over the full interval	0.9981	Stable
4	9	Spread over the full interval	1.001	Unstable
4	5	0 slow, 1 fast	0.9968	Stable
4	10	Points on the fast scale	0.99	Stable

Among the tested choices, the previously used distribution with interpolation points closest to the fast cluster gives stable methods for larger values of s. We therefore keep this distribution in the following tests.

4.4.6 Repeated RK-Opt runs

The RK-Opt construction starts with randomly generated initial guesses. Therefore, runs with identical settings can return different Runge-Kutta methods.

To test this dependence, we repeat each setup ten times. PolyOpt returns the same target stability polynomial R^* and step size h in every run. Thus, only the randomly

generated initial guesses in RK-Opt change between the runs.

We consider $p \in \{3, 4\}$ and $s \in \{9, 10\}$, since the previous experiments showed difficulties in finding stable methods for some of these setups. For the Chebyshev interpolation setup, we use the $s - p$ interpolation points closest to the fast cluster, since this distribution gave the best results in the previous experiments. A run is classified as no method returned if RK-Opt does not converge or if the required order conditions are not satisfied. For each remaining run, we compute

$$\max_{\lambda \in \Lambda} |R_{\mathrm{RK}}(h\lambda)| . \tag{4.74}$$

The method is classified as stable if

$$\max_{\lambda \in \Lambda} |R_{\mathrm{RK}}(h\lambda)| \leq 1 + 10^{-8}, \tag{4.75}$$

near the stability boundary if

$$1 + 10^{-8} < \max_{\lambda \in \Lambda} |R_{\mathrm{RK}}(h\lambda)| \leq 1.2, \tag{4.76}$$

and unstable if

$$\max_{\lambda \in \Lambda} |R_{\mathrm{RK}}(h\lambda)| > 1.2. \tag{4.77}$$

The results are shown in Tables 4.33 and 4.34. Each row contains the results of ten runs.

Table 4.33: Outcomes of ten repeated RK-Opt runs for $p = 3$ and $r_{\mathrm{fast}} = 50$.

Basis	Low-storage form	s	Stable	Near boundary	Unstable	No method returned
Chebyshev	$3S^*$	9	2	5	3	0
Chebyshev	$3S^*$	10	2	4	4	0
Chebinterp	$3S^*$	9	2	0	0	8
Chebinterp	$3S^*$	10	1	0	0	9
Chebyshev	$2N$	9	2	5	0	3
Chebyshev	$2N$	10	1	3	0	6
Chebyshev (monotone)	$2N$	9	4	4	0	2
Chebyshev (monotone)	$2N$	10	0	0	0	10

Table 4.34: Outcomes of ten repeated RK-Opt runs for $p = 4$ and $r_{\mathrm{fast}} = 50$.

Basis	Low-storage form	s	Stable	Near boundary	Unstable	No method returned
Chebyshev	$3S^*$	9	7	3	0	0
Chebyshev	$3S^*$	10	3	7	0	0
Chebinterp	$3S^*$	9	10	0	0	0
Chebinterp	$3S^*$	10	6	0	0	4
Chebyshev	$2N$	9	4	1	0	5
Chebyshev	$2N$	10	0	1	0	9
Chebyshev (monotone)	$2N$	9	0	0	0	10
Chebyshev (monotone)	$2N$	10	0	0	0	10

For the standard Chebyshev setup with the $3S^*$ low-storage form, RK-Opt returns a method in every run. However, the returned methods are not always stable. For $p = 3$,

only two of the ten runs are stable for both $s = 9$ and $s = 10$, while the remaining methods are near the stability boundary or unstable.

The Chebyshev interpolation setup shows a different pattern. In the repeated runs, RK-Opt either returns a stable $3S^*$ method or does not return a method. For $p = 4$ and $s = 9$, all ten runs return stable methods.

The standard $2N$ setup returns fewer methods than the standard $3S^*$ setup. The monotone constraint improves some third-order results, but does not give a general improvement. In particular, no fourth-order monotone $2N$ method is returned.

Overall, the Chebyshev interpolation setup gives the most consistent stability results. In these runs and in most of the previous tests, RK-Opt either returns a stable method or does not return a method.

4.4.7 Low-storage evaluation in RK-Opt

For the methods obtained using Chebyshev interpolation, RK-Opt evaluates the pointwise matching conditions using the Butcher representation (4.66). The stability values in Table 4.30 are evaluated in the same way. In the numerical time integration, however, the $3S^*$ low-storage recursion is used. Since the two evaluations can differ because of floating-point effects discussed above, a method that is stable in the Butcher evaluation may be unstable in the low-storage implementation.

We therefore also evaluate the methods considered for the later error comparison using the low-storage recursion. The results are shown in Table 4.35.

Table 4.35: Comparison of the Butcher and low-storage stability evaluations for $3S^*$ methods obtained using Chebyshev interpolation. The methods are tested at their PolyOpt step sizes h.

p	s	h	$\max \lvert R_{\mathrm{B}} \rvert$	$\max \lvert R_{\mathrm{LS}} \rvert$
1	8	22.11	0.99933	0.99944
2	7	1.914	0.999014	0.999014
3	10	1.984	0.9981	1.002397
3	11	2.405	0.99679	1.123197
4	13	0.8583	0.7943156	0.7943499

The first-, second-, and fourth-order methods remain stable in the low-storage evaluation. However, the two third-order methods are unstable, although they are stable in the Butcher evaluation. We therefore repeat the RK-Opt construction for these two methods using the low-storage evaluation directly in the matching conditions. For fourth order, we also repeat the stage search to examine whether this matching formulation returns stable methods for the same stage numbers as the standard Butcher-based matching.

Instead of evaluating the stability function in (3.68) with the Butcher evaluation (4.66), we use the low-storage stability function (4.65). The target stability polynomial and the PolyOpt step size remain unchanged. Only the evaluation used in the RK-Opt matching conditions is changed.

The resulting methods are shown in Table 4.36.

Table 4.36: Optimized $3S^*$ Runge-Kutta methods using Chebyshev interpolation and the low-storage evaluation in the RK-Opt matching conditions for the cluster spectrum with $r_{\text{fast}} = 50$. The methods are tested at their PolyOpt step sizes h.

p	s	h	$h/\#\text{RHS}$	$\max \lvert R_{\text{B}} \rvert$	$\max \lvert R_{\text{LS}} \rvert$
3	10	1.984	0.1984	0.99775	0.99775
3	11	2.405	0.2186	0.96799	0.96799
4	11	0.4276	0.03887	0.99957	0.99956
4	12	0.5734	0.04779	0.58032	0.58032

All methods are stable in the low-storage evaluation. For fourth order, RK-Opt returns methods only up to $s = 12$; none of the five runs for $s = 13$ returns a method. Since the selected method with $p = 4$ and $s = 13$ is already stable in the low-storage evaluation, it is kept unchanged. For third order, the method with $s = 11$ has the larger value of $h/\#\text{RHS}$ and is therefore used in the later error comparison.

4.4.8 Comparison with Projective Integration methods

Based on the previous tests, we use the selected $3S^*$ methods obtained using Chebyshev interpolation for the comparison with the PI methods. The first-, second-, and fourth-order methods are already stable in the low-storage evaluation and therefore remain unchanged. For third order, we use the method with $p = 3$ and $s = 11$ obtained using the low-storage evaluation in the RK-Opt matching conditions. The selected methods are listed in Table 4.37.

Table 4.37: Selected optimized $3S^*$ Runge-Kutta methods using Chebyshev interpolation with interpolation points closest to the fast cluster for the cluster spectrum with $r_{\text{fast}} = 50$.

p	s	h	$h/\#\text{RHS}$
1	8	22.11	2.763
2	7	1.914	0.2734
3	11	2.405	0.2186
4	13	0.8583	0.06603

We now compare the selected optimized Runge-Kutta methods with PFEK3 and PRK4K3. The absolute and relative errors are computed in the maximum norm over all components. We use the same treatment of the final time as in the isolated case. For the optimized Runge-Kutta methods, only full time steps are used, and the error is evaluated at the resulting final time.

First, we choose $T = 22$ so that all selected optimized Runge-Kutta methods can perform at least one full time step. The results are shown in Table 4.38.

Table 4.38: Errors for the two-scale cluster test problem with $r_{\text{fast}} = 50$ and final time $T = 22$.

Method	Step size	Step size/#RHS	abs. error	rel. error
RK $p = 1,\ s = 8$	22.11	2.763	0.9994	$1.321 \cdot 10^{9}$
RK $p = 2,\ s = 7$	1.914	0.2734	0.9882	$2.955 \cdot 10^{9}$
RK $p = 3,\ s = 11$	2.405	0.2186	0.9684	$8.099 \cdot 10^{9}$
RK $p = 4,\ s = 13$	0.8583	0.06603	0.002510	$4.047 \cdot 10^{6}$
PFEK3	1.911	0.4777	0.02920	$3.485 \cdot 10^{7}$
PRK4K3	2.665	0.1666	0.4909	$5.858 \cdot 10^{8}$

The first-order optimized Runge-Kutta method has by far the largest value of $h/\#\text{RHS}$. However, the fourth-order method gives the smallest absolute error.

Next, we consider $T = 100$. Since the exact solution is much smaller than one, the relative errors are not useful for the comparison. The results are shown in Table 4.39.

Table 4.39: Absolute errors for the two-scale cluster test problem with $r_{\text{fast}} = 50$ and final time $T = 100$.

Method	Step size	Step size/#RHS	abs. error
RK $p = 1,\ s = 8$	22.11	2.763	0.9966
RK $p = 2,\ s = 7$	1.914	0.2734	0.9491
RK $p = 3,\ s = 11$	2.405	0.2186	0.8740
RK $p = 4,\ s = 13$	0.8583	0.06603	$1.987 \cdot 10^{-12}$
PFEK3	1.911	0.4777	0.3289
PRK4K3	2.665	0.1666	0.2753

For this final time, the first-order method again has by far the largest value of $h/\#\text{RHS}$, but its absolute error is close to one. The fourth-order RK method again gives the smallest absolute error.

As in the previous comparisons, the PI methods have so far been tested at their maximal stable outer step sizes. We now study the effect of smaller outer step sizes on their accuracy using the work-precision diagram at $T = 100$ in Figure 4.12.

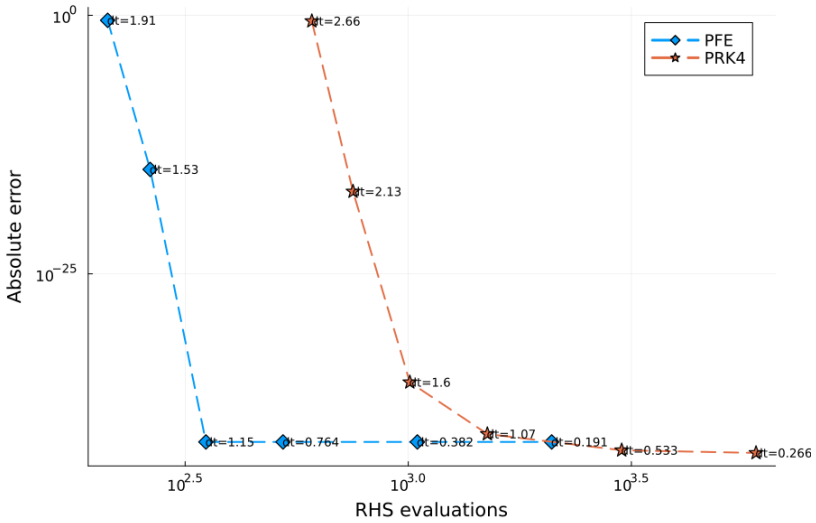


Figure 4.12: Work-precision diagram for PFEK3 and PRK4K3 at $T = 100$, showing the absolute error against the number of right-hand side evaluations for different step sizes.

From this diagram, we select points that combine small absolute errors with a low number of right-hand side evaluations. The corresponding outer step sizes and errors are listed in Table 4.40.

Table 4.40: Errors for the two-scale cluster test problem with $r_{\text{fast}} = 50$, final time $T = 100$, and reduced step sizes.

Method	Δt	$\Delta t/\#\text{RHS}$	abs. error	rel. error
PFEK3	1.150	0.2875	$5.521 \cdot 10^{-42}$	1
PRK4K3	1.070	0.06688	$3.356 \cdot 10^{-41}$	6.079

The first-order optimized Runge-Kutta method has by far the largest value of $h/\#\text{RHS}$. It is therefore a good choice when large stable step sizes are more important than accuracy. In contrast, the fourth-order optimized Runge-Kutta method gives the smallest absolute error at both final times when the methods are used with their maximal stable step sizes.

The PI methods are more flexible because their outer step size Δt can be reduced when a smaller error is required. This can strongly reduce the error, but it also requires more right-hand side evaluations.

Therefore, the best method again depends on the aim of the computation. The first-order optimized Runge-Kutta method gives the largest step size per right-hand side evaluation, while the fourth-order optimized method gives the smallest absolute errors when the

methods are used with their maximal stable step sizes. The PI methods allow the outer step size to be adjusted when a smaller error is required.

4.5 Semi-discrete moment model test problem

As an additional test problem, we consider a spectrum obtained from the semi-discretization of a relaxation model. This setup is based on the spectral analysis in [KS22]. The resulting clustered spectrum is used as input for the stability optimization.

We begin with a stiff hyperbolic balance law with relaxation,

$$\partial_t w + A(w)\partial_x w = -\frac{1}{\varepsilon(x)} S(w). \tag{4.78}$$

Here, $w = w(t,x) \in \mathbb{R}^N$ is the vector of unknowns, $A(w) \in \mathbb{R}^{N\times N}$ is the system matrix of the transport term, and $S(w) \in \mathbb{R}^N$ is the relaxation term. Small values of the relaxation time $\varepsilon(x) > 0$ lead to a stiff source term. The system matrix $A(w)$ of the hyperbolic moment model is taken from [KS22, Eq. (2.5)] and is given in Appendix 6.1.

As an example of this class, we consider a hyperbolic moment model for rarefied gas dynamics. The vector of unknowns is given by

$$w = (\rho, u, \theta, f_3, \dots, f_M)^T \in \mathbb{R}^{M+1}, \tag{4.79}$$

where ρ, u, and θ denote density, velocity, and temperature. The remaining variables $f_3, \dots, f_M$ are additional moment variables. The source term $S(w) \in \mathbb{R}^{M+1}$ on the right-hand side describes the collision term. It can be modeled by the BGK model [BGK54] as

$$-\frac{1}{\varepsilon(x)} S(w) = -\frac{1}{\varepsilon(x)} \operatorname{diag}(0,0,0,1,\dots,1) w. \tag{4.80}$$

For small values of $\varepsilon(x)$, the additional variables f_i quickly relax to zero and the model is governed by the slowly evolving variables ρ, u, and θ. This results in the separation of the different time scales.

For the spectral analysis, the matrix $A(w)$ is replaced by a constant matrix A. Following the PVM formulation in [KS22], the spatial discretization can be written in finite-volume form as

$$\frac{w_i^{n+1} - w_i^n}{\Delta t} = -\frac{1}{\Delta x}\left(A\frac{w_{i+1}^n - w_{i-1}^n}{2} + Q\frac{-w_{i+1}^n + 2w_i^n - w_{i-1}^n}{2} \right) - \frac{1}{\varepsilon(x_i)} S w_i^n. \tag{4.81}$$

The terms on the right-hand side correspond to the numerical flux, the numerical diffusion, and the possibly stiff source term. Writing (4.81) as a semi-discrete system for

$$W = (w_1, \dots, w_{n_x})^T \in \mathbb{R}^{(M+1)n_x}, \tag{4.82}$$

and assuming periodic boundary conditions, the system can be written as

$$W'(t) = \mathcal{A}W(t), \tag{4.83}$$

where $\mathcal{A}$ has the block form

$$\mathcal{A} = \begin{pmatrix} d_1 & b & & c \\ c & d_2 & b & \\ & \ddots & \ddots & \ddots \\ b & & c & d_{n_x} \end{pmatrix}, \tag{4.84}$$

with

$$d_i = -\frac{1}{\Delta x} Q - \frac{1}{\varepsilon(x_i)} S, \tag{4.85}$$

$$b = \frac{1}{2\Delta x}(Q - A), \tag{4.86}$$

$$c = \frac{1}{2\Delta x}(Q + A). \tag{4.87}$$

The transport matrix A is fixed by the chosen model, whereas the matrix Q depends on the spatial discretization. For the hyperbolic moment model, applying the following spectral result requires linearization around an equilibrium state and a suitable symmetrization [KS22, Assumption (A1)]. The result characterizes the location of the eigenvalues and is used for the subsequent stability analysis and stability optimization.

Theorem 4.1. *Let $\mathcal{A}$ be the semi-discrete matrix obtained from the spatial discretization of the linearized relaxation system and let $\varepsilon_i = \varepsilon(x_i)$. Assume that the transport matrix A is symmetric and that the relaxation term has the diagonal form*

$$-\frac{1}{\varepsilon_i} S = -\frac{1}{\varepsilon_i} \operatorname{diag}(0, 0, 0, 1, \ldots, 1).$$

Then the spectrum of $\mathcal{A}$ is contained in a union of disks,

$$\sigma(\mathcal{A}) \subset C(\lambda_s, R) \cup \left(\bigcup_i C(\lambda_{\varepsilon_i}, R) \right), \tag{4.88}$$

where $C(\lambda, R)$ denotes a disk in the complex plane with center λ and radius R. The centers and the radius are given by

$$\lambda_s = -\frac{1}{\Delta x} \lambda(Q), \tag{4.89}$$

$$\lambda_{\varepsilon_i} = -\frac{1}{\Delta x} \lambda(Q) - \frac{1}{\varepsilon_i}, \tag{4.90}$$

$$R = \frac{1}{2\Delta x} \left(|\lambda_{\max}(Q - A)| + |\lambda_{\max}(Q + A)| \right). \tag{4.91}$$

Proof. The proof can be found in [KS22, Theorem 2.2]. □

The theorem shows that the spectrum is contained in one slow cluster and one or several fast clusters. In the following, we use the upwind discretization, for which $Q = |A|$. Substituting this choice into the formulas above gives

$$\lambda_s = -\frac{|\lambda(A)|}{\Delta x}, \tag{4.92}$$

$$\lambda_{\varepsilon_i} = -\frac{|\lambda(A)|}{\Delta x} - \frac{1}{\varepsilon_i}, \tag{4.93}$$

$$R = \frac{1}{2\Delta x}\left(|\lambda_{\max}(|A| - A)| + |\lambda_{\max}(|A| + A)|\right) \leq \frac{\lambda_{\max}(A)}{\Delta x}, \tag{4.94}$$

where $\lambda_{\max}(A) > 0$. In the numerical experiments, we use $M = 4$, so that

$$w = (\rho, u, \theta, f_3, f_4)^T. \tag{4.95}$$

The transport matrix is obtained by linearizing the hyperbolic moment model around the constant reference state

$$w_* = (1, \pi, 1, 0, 0)^T. \tag{4.96}$$

The spatial discretization uses $n_x = 50$ cells with $\Delta x = 1/50$. Since $M + 1 = 5$, the semi-discrete matrix $\mathcal{A}$ has dimension 250×250. In contrast to the previous two-scale test problems, the semi-discrete moment model is used to compare the methods on a more realistic spectrum with wider slow and fast clusters. The comparison is based on the maximal stable step size and the step size per right-hand side evaluation, where #RHS denotes the number of right-hand side evaluations per time step.

For both relaxation times considered below, each cluster boundary is sampled using $M_{\text{spec}} = 800$ equally spaced points in PolyOpt. The stability tests are performed on the eigenvalues of the corresponding semi-discrete matrix. For all RK-Opt computations in this section, each run uses 200 starting points.

4.5.1 Results for $\varepsilon = 10^{-3}$

We first consider the relaxation time

$$\varepsilon = 10^{-3}. \tag{4.97}$$

For $\Delta x = 1/50$, the slow and fast clusters are centered at

$$\lambda_s = -299.928, \qquad \lambda_f = -1299.928, \tag{4.98}$$

and both clusters have radius

$$R = 299.928. \tag{4.99}$$

Projective Integration methods

We apply PFE and PRK4 to the semi-discrete moment model. As in the previous test cases, the inner step size is chosen according to the center of the fast cluster:

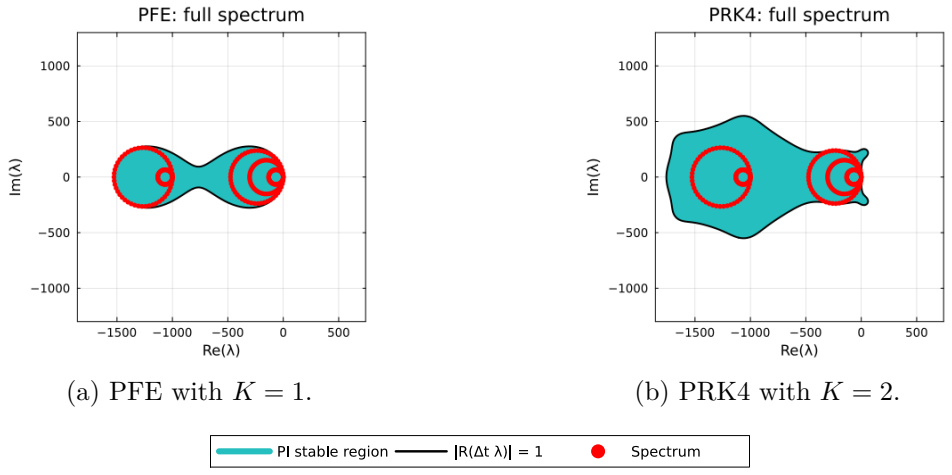
$$\delta t = -\frac{1}{\lambda_f} = \frac{1}{1299.928} \approx 7.693 \cdot 10^{-4}. \tag{4.100}$$

The resulting stability limits are shown in Table 4.41.

Table 4.41: Maximal stable outer step sizes for PFE and PRK4 applied to the semi-discrete moment model with $\varepsilon = 10^{-3}$.

Method	K	δt	#RHS	$\Delta t_{\max}$	$\Delta t_{\max}/\#\text{RHS}$
PFE	1	$7.693 \cdot 10^{-4}$	2	0.005160	0.002580
PFE	2	$7.693 \cdot 10^{-4}$	3	0.006741	0.002247
PRK4	1	$7.693 \cdot 10^{-4}$	8	0.008718	0.001090
PRK4	2	$7.693 \cdot 10^{-4}$	12	0.013581	0.001132
PRK4	3	$7.693 \cdot 10^{-4}$	16	0.016364	0.001023

For PFE, the largest value of $\Delta t_{\max}/\#\text{RHS}$ is obtained for $K = 1$. For PRK4, the largest value is obtained for $K = 2$. The corresponding stability regions are shown in Figure 4.13. For both PI methods, the fast and slow stability regions overlap and form one connected



(a) PFE with $K = 1$.

(b) PRK4 with $K = 2$.

Figure 4.13: Stability regions for the semi-discrete moment model with $\varepsilon = 10^{-3}$ at the maximal stable outer step sizes. PFEK1 is shown on the left and PRK4K2 on the right.

stability region.

Optimized Runge-Kutta methods

Compared with the previous test cases, RK-Opt returns methods with larger stage numbers for this problem. Since the computational cost of the RK-Opt construction increases considerably with the number of stages, we first compare the different polynomial bases and RK-Opt setups for fixed order $p = 3$. One RK-Opt run is performed for each tested setup and stage number. Only setups for which a stable method is obtained for at least one tested stage number are included in the table.

As before, the interpolation interval is $[z_{\min}, 0]$, where

$$z_{\min} = h \min_{\lambda \in \Lambda} \mathrm{Re}(\lambda).$$

We compare three different distributions of the interpolation conditions. The $s - p$ matching points are either chosen closest to the scaled fast-cluster center $h\lambda_f$, distributed over the full interval $[z_{\min}, 0]$, or distributed evenly between the slow and fast clusters. We refer to these distributions as fast-focused, full-interval, and cluster-split. The results are shown in Table 4.42.

Table 4.42: Largest tested stage numbers for which the performed RK-Opt run returns a stable method for the semi-discrete moment model with $\varepsilon = 10^{-3}$. Here, $s_{\max}$ denotes the largest tested number of stages for which the returned method is stable at the PolyOpt step size h.

Basis	RK-Opt setup	$s_{\max}$	h	$h/\#\mathrm{RHS}$	$\max \lvert R_{\mathrm{RK}} \rvert$
Chebyshev	3S*	19	0.03764	0.001981	1
Chebyshev	3S*, monotone c	19	0.03764	0.001981	1
Chebyshev	2N	14	0.02596	0.001854	1
Chebyshev	2N, monotone c	14	0.02596	0.001854	1
Chebinterp	3S*, fast-focused	13	0.02342	0.001802	1
Chebinterp	3S*, full-interval	14	0.02607	0.001862	1
Chebinterp	3S*, cluster-split	16	0.03052	0.001908	1

Among the tested setups, the Chebyshev basis with the $3S^*$ formulation gives both the largest stage number and the largest value of $h/\#\mathrm{RHS}$.

For the interpolation-based constructions, the cluster-split distribution gives the largest stage number, with $s_{\max} = 16$. Unlike in the previous cluster experiments, the fast-focused distribution is no longer the best choice. A possible explanation is that the slow cluster is much wider and therefore has a stronger influence on the stability restriction.

For the cluster-split distribution, we compare the Vandermonde-based and derivative-based order conditions (3.67). The results are shown in Table 4.43.

Table 4.43: Comparison of the Vandermonde-based and derivative-based formulations of the PolyOpt order conditions for the cluster-split distribution with $p = 3$ and $\varepsilon = 10^{-3}$.

Order conditions	$s_{\max}$	h	$h/\#\text{RHS}$	$\max \lvert R_{\text{RK}} \rvert$
Vandermonde-based	16	0.03052	0.001908	1
Derivative-based	18	0.03536	0.001964	1

In the performed runs, the derivative-based formulation reaches $s_{\max} = 18$, compared with $s_{\max} = 16$ for the Vandermonde-based formulation. Thus, for this setup, the derivative-based formulation returns a stable method for a larger tested stage number. However, the Chebyshev-basis construction with the $3S^*$ formulation still gives the largest stage number and the largest value of $h/\#\text{RHS}$.

Based on these results, we use the Chebyshev basis together with the $3S^*$ low-storage formulation for all orders. The resulting methods are shown in Table 4.44. For each order, the method used in the comparison with the PI methods is marked as Selected.

Table 4.44: Optimized $3S^*$ Runge-Kutta methods for the semi-discrete moment model with $\varepsilon = 10^{-3}$. The methods are tested at the corresponding PolyOpt step sizes h.

p	s	h	$h/\#\text{RHS}$	$\max \lvert R_{\text{RK}} \rvert$	Decision
1	9	0.02085	0.002316	1	Stable
1	10	0.02375	0.002375	1	Selected
1	11	0.02589	0.002353	1	Stable
2	19	0.04179	0.002200	1	Stable
2	20	0.04440	0.002220	1	Selected
3	18	0.03536	0.001964	1	Stable
3	19	0.03764	0.001981	1	Selected
4	20	0.01913	0.000957	1	Stable
4	21	0.03785	0.001802	1	Selected

The stability values in Table 4.44 are computed using the Butcher evaluation (4.66), which is also used in RK-Opt. We additionally evaluate the selected methods using the $3S^*$ low-storage recursion (4.64). In contrast to the cluster spectrum with $r_{\text{fast}} = 50$, both evaluations give the same stability values for all selected methods.

Comparison with Projective Integration methods

We now compare the selected optimized Runge-Kutta methods with the PI methods. The results are shown in Table 4.45.

Table 4.45: Comparison of the selected optimized Runge-Kutta methods with the PI methods for $\varepsilon = 10^{-3}$.

Method	Step size	Step size/#RHS
RK $p = 1,\ s = 10$	0.02375	0.002375
RK $p = 2,\ s = 20$	0.04440	0.002220
RK $p = 3,\ s = 19$	0.03764	0.001981
RK $p = 4,\ s = 21$	0.03785	0.001802
PFEK1	0.005160	0.002580
PRK4K2	0.013581	0.001132

The optimized Runge-Kutta methods allow larger maximal stable step sizes than the PI methods. In contrast to the previous test cases, however, PFEK1 gives the largest value of $\Delta t_{\max}/\#\text{RHS}$ among all methods and therefore gives the best result with respect to this measure.

4.5.2 Results for $\varepsilon = 10^{-4}$

We next consider the relaxation time

$$\varepsilon = 10^{-4}. \tag{4.101}$$

For $\Delta x = 1/50$, the slow and fast clusters are centered at

$$\lambda_s = -299.928, \qquad \lambda_f = -10299.928, \tag{4.102}$$

and both have radius

$$R = 299.928. \tag{4.103}$$

Compared with the case $\varepsilon = 10^{-3}$, the fast cluster is located further to the left.

Projective Integration methods

We again apply PFE and PRK4 to the semi-discrete moment model. As in the previous case, the inner step size is chosen according to the center of the fast cluster:

$$\delta t = -\frac{1}{\lambda_f} = \frac{1}{10299.928} \approx 9.709 \cdot 10^{-5}. \tag{4.104}$$

The resulting stability limits are shown in Table 4.46. For both methods, the largest value of $\Delta t_{\max}/\#\text{RHS}$ is obtained for $K = 1$. The corresponding stability regions are shown in Figure 4.14.

Table 4.46: Maximal stable outer step sizes for PFE and PRK4 applied to the semi-discrete moment model with $\varepsilon = 10^{-4}$.

Method	K	δt	#RHS	$\Delta t_{\max}$	$\Delta t_{\max}/\#\text{RHS}$
PFE	1	$9.709 \cdot 10^{-5}$	2	0.003631	0.001816
PFE	2	$9.709 \cdot 10^{-5}$	3	0.004488	0.001496
PRK4	1	$9.709 \cdot 10^{-5}$	8	0.006197	0.000775
PRK4	2	$9.709 \cdot 10^{-5}$	12	0.006581	0.000548

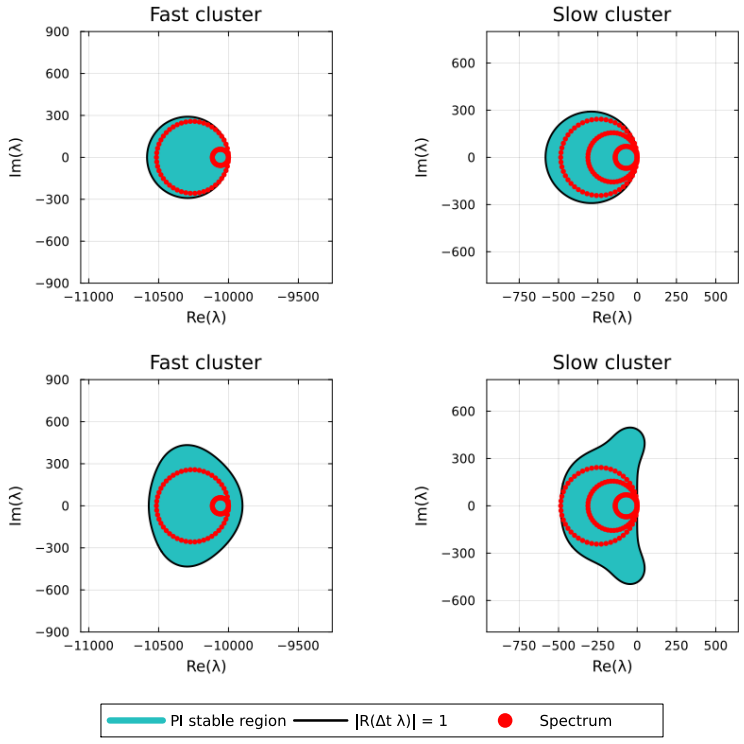


Figure 4.14: Stability regions for the semi-discrete moment model with $\varepsilon = 10^{-4}$ at the maximal stable outer step sizes. PFEK1 is shown in the top row and PRK4K1 in the bottom row.

Optimized Runge-Kutta methods

We use the same RK-Opt setups and the same distributions of the interpolation conditions as for $\varepsilon = 10^{-3}$. As before, we first compare the different polynomial bases and RK-Opt setups for fixed order $p = 3$. Since the computational cost of the RK-Opt construction increases considerably with the number of stages, one RK-Opt run is performed for each tested setup and stage number. The results are shown in Table 4.47. Among the tested

Table 4.47: Largest tested stage numbers for which the performed RK-Opt run returns a stable method for the semi-discrete moment model with $\varepsilon = 10^{-4}$. Here, $s_{\max}$ denotes the largest tested number of stages for which the returned method is stable at the PolyOpt step size h.

Basis	RK-Opt setup	$s_{\max}$	h	$h/\#\text{RHS}$	$\max \lvert R_{\text{RK}} \rvert$
Chebyshev	3S*	15	0.01751	0.001167	1
Chebyshev	3S*, monotone c	15	0.01751	0.001167	1
Chebyshev	2N	13	0.01446	0.001112	1
Chebyshev	2N, monotone c	13	0.01446	0.001112	1
Chebinterp	3S*, fast-focused	11	0.01123	0.001021	1
Chebinterp	3S*, full-interval	11	0.01123	0.001021	1
Chebinterp	3S*, cluster-split	13	0.01445	0.001111	1

setups, the Chebyshev basis with the $3S^*$ formulation gives again both the largest stage number and the largest value of $h/\#\text{RHS}$.

For the interpolation-based constructions, the cluster-split distribution gives the largest tested stable stage number, $s_{\max} = 13$. For this distribution, we compare the Vandermonde-based and derivative-based order conditions (3.67). The results are shown in Table 4.48.

Table 4.48: Comparison of the Vandermonde-based and derivative-based formulations of the PolyOpt order conditions for the cluster-split distribution with $p = 3$ and $\varepsilon = 10^{-4}$.

Order conditions	$s_{\max}$	h	$h/\#\text{RHS}$	$\max \lvert R_{\text{RK}} \rvert$
Vandermonde-based	13	0.01445	0.00111	1
Derivative-based	13	0.01415	0.001088	1

For the cluster-split distribution, both formulations reach $s_{\max} = 13$. Thus, the derivative-based formulation does not improve the largest tested stable stage number for this case.

Based on these results, we use the Chebyshev basis together with the $3S^*$ low-storage formulation for all orders. The resulting methods are shown in Table 4.49. For each order, the method used in the comparison with the PI methods is marked as Selected.

Table 4.49: Optimized $3S^*$ Runge-Kutta methods for the semi-discrete moment model with $\varepsilon = 10^{-4}$. The methods are tested at the corresponding PolyOpt step sizes h.

p	s	h	$h/\#\text{RHS}$	$\max \lvert R_{\text{RK}} \rvert$	Decision
1	10	0.01735	0.001735	1	Selected
1	11	0.01800	0.001636	1	Stable
2	12	0.01784	0.001486	1	Selected
2	13	0.01818	0.001399	1	Stable
3	15	0.01751	0.001167	1	Selected
3	16	0.02054	0.001284	7.683	Unstable
4	15	0.01454	0.0009696	1	Selected
4	16	0.01677	0.001048	1.812	Unstable

As for $\varepsilon = 10^{-3}$, the stability values in Table 4.49 are computed using the Butcher evaluation (4.66). We additionally evaluate the selected methods using the $3S^*$ low-storage recursion (4.64). Both evaluations give the same stability values for all selected methods.

For $p = 4$ and $s = 16$, we observe the same behavior as for the unstable method in Figure 4.8. The stability polynomial computed by PolyOpt is stable on the scaled spectrum. However, the Runge-Kutta method obtained from RK-Opt is unstable at the PolyOpt step size, since its stability region does not contain the complete scaled spectrum. The corresponding stability regions are shown in Figure 4.15.

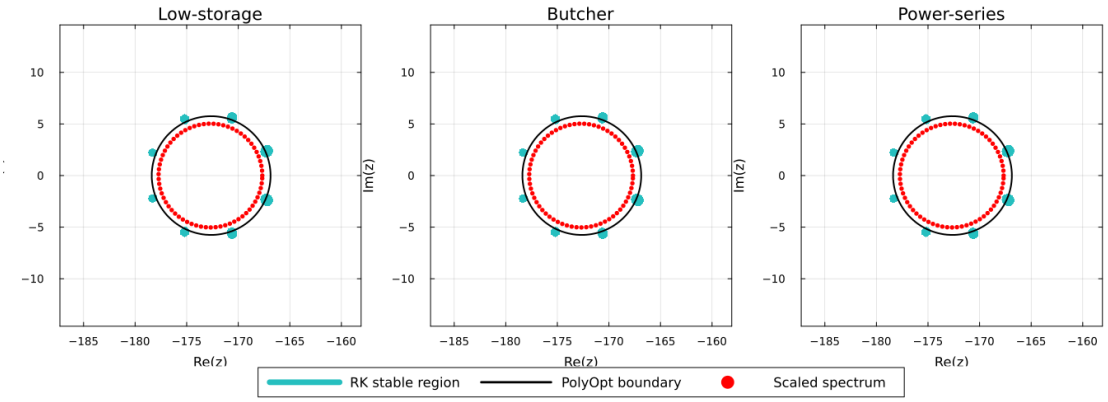


Figure 4.15: Stability regions obtained from the $3S^*$ low-storage evaluation (4.64), the Butcher evaluation (4.66), and the power-series evaluation (4.67) for the $3S^*$ method with $p = 4$ and $s = 16$, using the Chebyshev basis at the PolyOpt step size h.

Comparison with Projective Integration methods

We now compare the selected optimized Runge-Kutta methods with the PI methods. The results are shown in Table 4.50.

Table 4.50: Comparison of the selected optimized Runge-Kutta methods with the PI methods for $\varepsilon = 10^{-4}$.

Method	Step size	Step size/#RHS
RK $p = 1,\ s = 10$	0.01735	0.001735
RK $p = 2,\ s = 12$	0.01784	0.001486
RK $p = 3,\ s = 15$	0.01751	0.001167
RK $p = 4,\ s = 15$	0.01454	0.0009696
PFEK1	0.003631	0.001816
PRK4K1	0.006197	0.000775

It shows the same behavior as for $\varepsilon = 10^{-3}$. The optimized Runge-Kutta methods allow larger maximal stable step sizes than the PI methods. However, PFEK1 again gives the largest value of $\Delta t_{\max}/\#\text{RHS}$ among all methods and therefore gives the best result with respect to this measure.

4.6 Generalization of the two-scale system

As an additional test problem, we consider a coupled, higher-dimensional generalization of the two-scale system from Section 4.1. The problem was suggested by Julian Koellermeier in a private communication. In contrast to the diagonal cluster systems considered above, the slow and fast components are coupled through the matrix B_N. The corresponding initial-value problem is

$$\frac{\mathrm{d}u}{\mathrm{d}t} = Au, \qquad u(0) = u_0, \tag{4.105}$$

where

$$u(t) = \begin{pmatrix} u_{\mathrm{s}}(t) \\ u_{\mathrm{f}}(t) \end{pmatrix} \in \mathbb{R}^{2N} \tag{4.106}$$

contains N slow and N fast components. The system matrix is defined by

$$A = \begin{pmatrix} -\alpha B_N & \mathbf{0} \\ \varepsilon^{-1} B_N & -\varepsilon^{-1} B_N \end{pmatrix}, \tag{4.107}$$

where

$$B_N = \begin{pmatrix} 2 & 0 & \cdots & 0 & -1 \\ -1 & 2 & \ddots & & 0 \\ 0 & -1 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & 2 & 0 \\ 0 & \cdots & 0 & -1 & 2 \end{pmatrix} \in \mathbb{R}^{N \times N}. \tag{4.108}$$

Equivalently, the system can be written as

$$u_{\mathrm{s}}' = -\alpha B_N u_{\mathrm{s}}, \qquad u_{\mathrm{f}}' = \varepsilon^{-1} B_N \left(u_{\mathrm{s}} - u_{\mathrm{f}}\right). \tag{4.109}$$

In the following, we use

$$N = 20, \qquad \alpha = 10, \qquad \varepsilon = 10^{-3}, \tag{4.110}$$

together with the initial value

$$u_0 = \left(1, 2, \ldots, N, 0, \ldots, 0\right)^{\mathsf{T}}. \tag{4.111}$$

For these parameters, the spectrum consists of a slow cluster centered at -20 with radius 10 and a fast cluster centered at -2000 with radius 1000. The same spectrum is used in PolyOpt and for all stability tests. Since A is constant, the exact solution is given by

$$u_{\mathrm{ex}}(t) = \exp(tA) u_0. \tag{4.112}$$

The matrix A is also nonnormal. Therefore, even if all scaled eigenvalues lie inside the stability region, the norm of the numerical solution does not necessarily decrease at every time step. Nevertheless, the stability optimization and the stability checks below are based on the spectrum.

4.6.1 Projective Integration methods

We apply PFE and PRK4 to the generalized two-scale system. As in the previous test cases, the inner step size is chosen according to the center of the fast cluster:

$$\delta t = -\frac{1}{\lambda_f} = \frac{1}{2000} = 5 \cdot 10^{-4}. \tag{4.113}$$

Because of the large radius of the fast cluster, we consider larger values of K. The resulting stability limits are shown in Table 4.51.

Table 4.51: Maximal stable outer step sizes for PFE and PRK4 applied to the generalized two-scale system.

Method	K	δt	#RHS	$\Delta t_{\max}$	$\Delta t_{\max}/\#\text{RHS}$
PFE	7	$5 \cdot 10^{-4}$	8	0.046500	0.005812
PFE	8	$5 \cdot 10^{-4}$	9	0.074951	0.008328
PFE	9	$5 \cdot 10^{-4}$	10	0.076024	0.007602
PRK4	7	$5 \cdot 10^{-4}$	32	0.076227	0.002382
PRK4	8	$5 \cdot 10^{-4}$	36	0.108946	0.003026
PRK4	9	$5 \cdot 10^{-4}$	40	0.110973	0.002774

For both PFE and PRK4, the largest value of $\Delta t_{\max}/\#\text{RHS}$ is obtained for $K = 8$. The corresponding stability regions are shown in Figure 4.16.

4.6.2 Optimized Runge-Kutta methods

As in the semi-discrete moment model, the computational cost of the RK-Opt construction increases considerably with the number of stages. We therefore first compare the different polynomial bases and RK-Opt setups for fixed order $p = 3$. One RK-Opt run with 200 starting points is performed for each tested setup and stage number. For each setup, we report the largest tested stage number for which the returned method is stable at the PolyOpt step size. As before, the interpolation interval is $[z_{\min}, 0]$, where

$$z_{\min} = h \min_{\lambda \in \Lambda} \operatorname{Re}(\lambda). \tag{4.114}$$

We compare the same three distributions of the interpolation conditions as for the semi-discrete moment model. The $s - p$ matching points are either chosen closest to the scaled fast-cluster center $h\lambda_{\mathrm{f}}$, distributed over the full interval $[z_{\min}, 0]$, or divided between the slow and fast clusters. We refer to these distributions as fast-focused, full-interval, and cluster-split. The results are shown in Table 4.52.

Table 4.52: Largest tested stage numbers for which the performed RK-Opt run returns a stable method for the generalized two-scale system at fixed order $p = 3$. Here, $s_{\max}$ denotes the largest tested number of stages for which the returned method is stable at the PolyOpt step size h.

Basis	RK-Opt setup	$s_{\max}$	h	$h/\#\text{RHS}$	$\max\lvert R_{\mathrm{RK}}\rvert$
Chebyshev	3S*	18	0.04631	0.002573	0.998586
Chebyshev, monotone$c$	3S*	18	0.04631	0.002573	0.998586
Chebyshev	$2N$	14	0.02084	0.001489	0.998486
Chebyshev	$2N$, monotone c	14	0.02084	0.001489	0.998485
Chebinterp	3S*, fast-focused	12	0.01408	0.001173	0.996930
Chebinterp	3S*, full-interval	12	0.01408	0.001173	0.996935
Chebinterp	3S*, cluster-split	12	0.01408	0.001173	0.996960

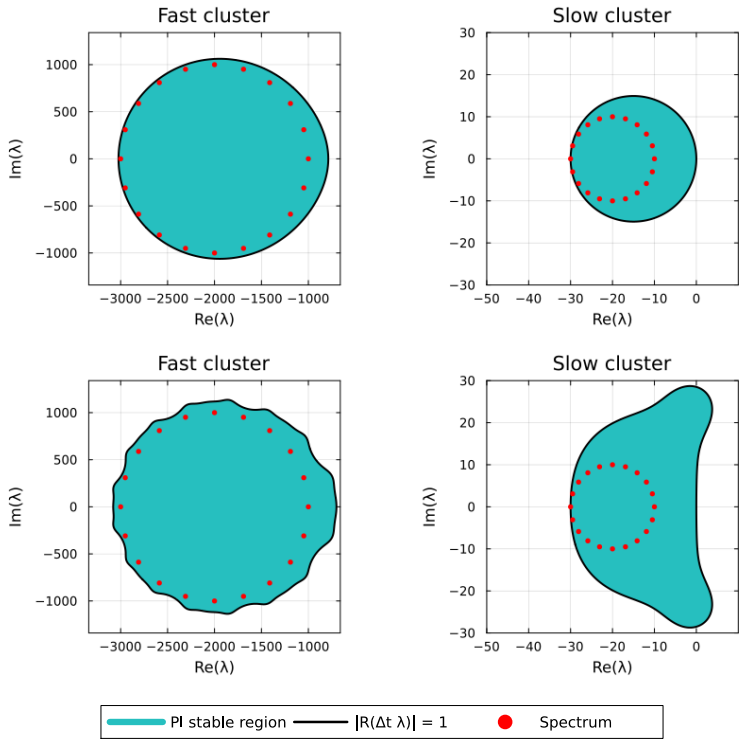


Figure 4.16: Stability regions for the generalized two-scale spectrum at the maximal stable outer step sizes. PFEK8 is shown in the top row and PRK4K8 in the bottom row.

Among the tested setups, the Chebyshev basis with the $3S^*$ formulation gives both the largest stage number and the largest value of $h/\#\text{RHS}$ among the tested setups.

All three interpolation-based constructions reach $s_{\max} = 12$. For the full-interval distribution, we compare the Vandermonde-based and derivative-based order conditions (3.67). The results are shown in Table 4.53.

Table 4.53: Comparison of the Vandermonde-based and derivative-based formulations of the PolyOpt order conditions for the full-interval distribution with $p = 3$.

Order conditions	$s_{\max}$	h	$h/\#\text{RHS}$	$\max\lvert R_{\text{RK}}\rvert$
Vandermonde-based	12	0.01408	0.001173	0.996930
Derivative-based	14	0.02084	0.001489	0.998486

In the performed runs, the derivative-based formulation reaches $s_{\max} = 14$, compared with $s_{\max} = 12$ for the Vandermonde-based formulation. Thus, for the full-interval distribution, it returns a stable method for a larger tested stage number.

Since the Chebyshev-basis construction with the 3S* low-storage formulation still gives the largest stage number, it is used for all orders. The resulting methods are shown in Table 4.54.

Table 4.54: Optimized 3S* Runge-Kutta methods for the generalized two-scale system. The methods are tested at the corresponding PolyOpt step sizes h.

p	s	h	$h/\#\text{RHS}$	$\max\lvert R_{\text{RK}}\rvert$	Decision
1	13	0.1067	0.008209	0.99927	Selected
1	14	0.1322	0.009444	$4.80 \cdot 10^9$	Unstable
2	14	0.06663	0.00476	0.99945	Stable
2	15	0.07945	0.005296	0.99944	Selected
3	18	0.04631	0.002573	0.998586	Selected
3	19	0.05689	0.00299	1.04	Unstable
4	19	0.02708	0.001425	0.999970	Stable
4	20	0.03136	0.001568	0.99920	Selected

The stability values in Table 4.54 are computed using the Butcher evaluation (4.66), which is also used in RK-Opt. We additionally evaluate the selected methods using the 3S* low-storage recursion (4.64). The results are shown in Table 4.55.

Table 4.55: Comparison of the Butcher and low-storage stability evaluations for the selected 3S* methods for the generalized two-scale system.

p	s	h	$\max\|R_{\mathrm{B}}\|$	$\max\|R_{\mathrm{LS}}\|$
1	13	0.106713	0.999273	0.999273
2	15	0.0794471	0.99944	0.999442
3	18	0.0463131	0.998586	0.998586
4	20	0.0313612	0.999202	0.999203

For all selected methods, both evaluations give the same stability values. This differs from the cluster spectrum with $r_{\mathrm{fast}} = 50$, where the evaluations gave different results.

4.6.3 Comparison with Projective Integration methods

We now compare the selected optimized Runge-Kutta methods with PFEK8 and PRK4K8. The absolute and relative errors are computed in the maximum norm over all components. As in the previous test cases, only full time steps are used for the optimized Runge-Kutta methods, and the error is evaluated at the resulting final time.

First, we choose the final time $T = 0.1067$ so that all selected optimized Runge-Kutta methods can perform at least one full time step. The results are shown in Table 4.56.

Table 4.56: Errors for the generalized two-scale test problem with final time $T = 0.1067$.

Method	Step size	Step size/#RHS	abs. error	rel. error
RK $p = 1,\ s = 13$	0.1067	0.008209	19.14	2.909
RK $p = 2,\ s = 15$	0.07945	0.005296	12.22	3.219
RK $p = 3,\ s = 18$	0.04631	0.002573	19.10	4.077
RK $p = 4,\ s = 20$	0.03136	0.001568	21.94	4.063
PFEK8	0.07495	0.008328	25.67	3.881
PRK4K8	0.1089	0.003026	4.529	0.6847

PFEK8 has the largest value of $\Delta t_{\max}/\#\mathrm{RHS}$ among all methods, whereas PRK4K8 gives the smallest absolute and relative errors.

Among the optimized Runge-Kutta methods, the first-order method has the largest value of $h/\#\mathrm{RHS}$, while the second-order method gives the smallest absolute error.

Next, we consider the final time $T = 50$. Since the exact solution is much smaller than one, the relative errors are not useful for the comparison. We therefore compare only the absolute errors. The results are shown in Table 4.57.

Table 4.57: Absolute errors for the generalized two-scale test problem with target final time $T = 50$.

Method	Step size	Step size/#RHS	abs. error
RK $p = 1,\ s = 13$	0.1067	0.008209	14.58
RK $p = 2,\ s = 15$	0.07945	0.005296	7.577
RK $p = 3,\ s = 18$	0.04631	0.002573	3.775
RK $p = 4,\ s = 20$	0.03136	0.001568	5.222
PFEK8	0.07495	0.008328	0.3888
PRK4K8	0.1089	0.003026	0.4033

PFEK8 and PRK4K8 give nearly the same absolute error, and both errors are smaller than those of all selected optimized Runge-Kutta methods at their maximal stable step sizes. Among the optimized Runge-Kutta methods, the third-order method gives the smallest absolute error.

In the previous test problems, the PI methods usually gave larger errors than the optimized Runge-Kutta methods when the maximal stable step sizes were used. For the generalized two-scale system, however, both PI methods give clearly smaller errors at the longer final time. Since the matrix A is nonnormal, this different behavior may be related to pseudospectral effects. Following [KA12, Section 4.3], the η-pseudospectrum of A is defined by

$$\Lambda_\eta(A) = \left\{ z \in \mathbb{C} : \left\| (zI - A)^{-1} \right\|_2 > \eta^{-1} \right\}. \tag{4.115}$$

We use $\eta = 40$, since this value makes the pseudospectrum around both spectral clusters visible. The stability regions together with the pseudospectrum are shown in Figure 4.17. The main difference appears around the fast cluster. The stability regions of PFEK8 and PRK4K8 contain a larger part of the pseudospectrum than the stability region of the optimized Runge-Kutta method, for which a larger part lies outside the stability region. This may explain the smaller errors of the PI methods at their maximal stable step sizes.

After comparing the methods at their maximal stable step sizes, we use the work-precision diagram at $T = 50$ in Figure 4.18 to study the effect of smaller outer step sizes on the accuracy of the PI methods. From this diagram, we select points that combine small absolute errors with a low number of right-hand side evaluations. The corresponding outer step sizes and errors are shown in Table 4.58. The PI methods give the best

Table 4.58: Errors for the generalized two-scale test problem with target final time $T = 50$ and reduced outer step sizes.

Method	Δt	$\Delta t/\#\text{RHS}$	abs. error	rel. error
PFEK8	0.045	0.005	$7.556 \cdot 10^{-217}$	1
PRK4K8	0.08720	0.002422	$1.385 \cdot 10^{-212}$	$1.833 \cdot 10^{4}$

overall results for the generalized two-scale system. PFEK8 gives the largest value of

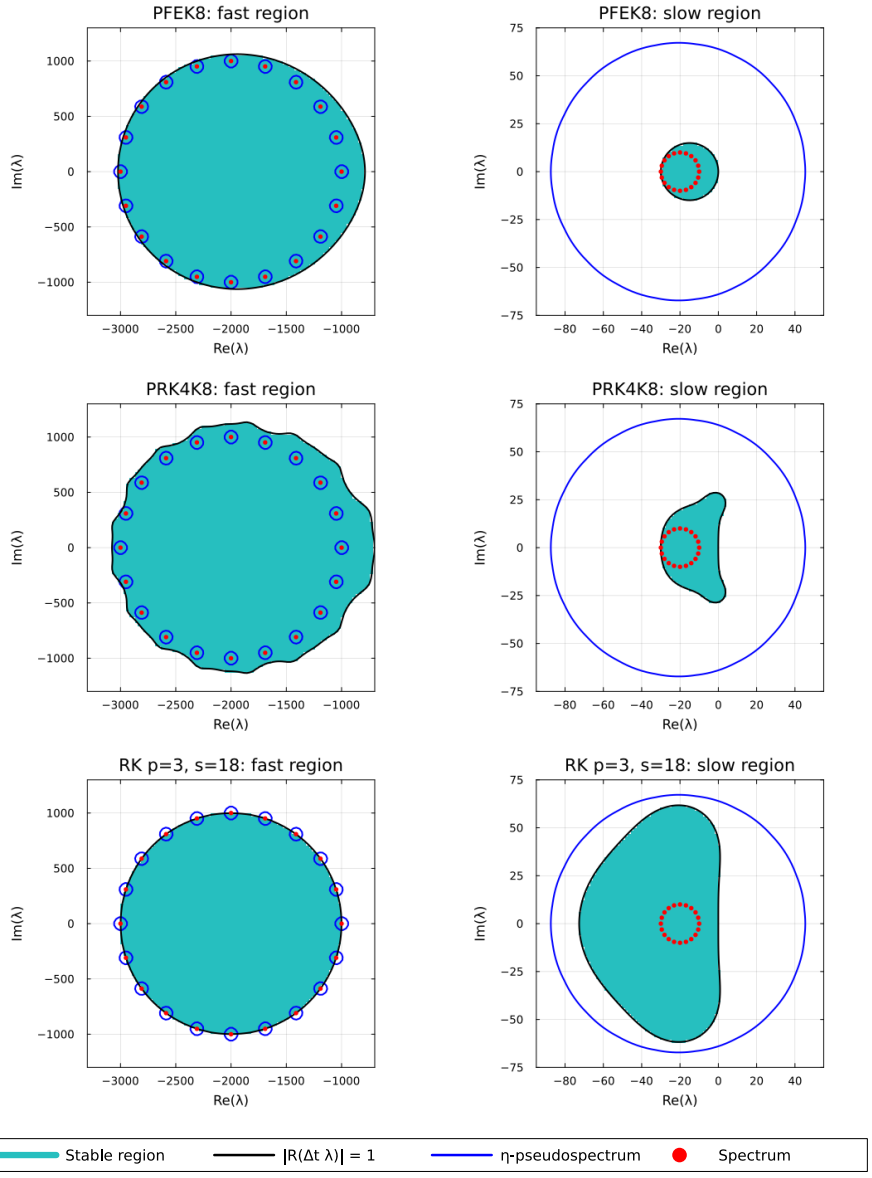


Figure 4.17: Stability regions and the η-pseudospectrum of the generalized two-scale system. PFEK8 is shown in the first row, PRK4K8 in the second row, and the optimized Runge-Kutta method with $p = 3$ and $s = 18$ in the third row. PFEK8 and PRK4K8 are evaluated at their maximal stable outer step sizes, while the optimized Runge-Kutta method is evaluated at the corresponding PolyOpt step size.

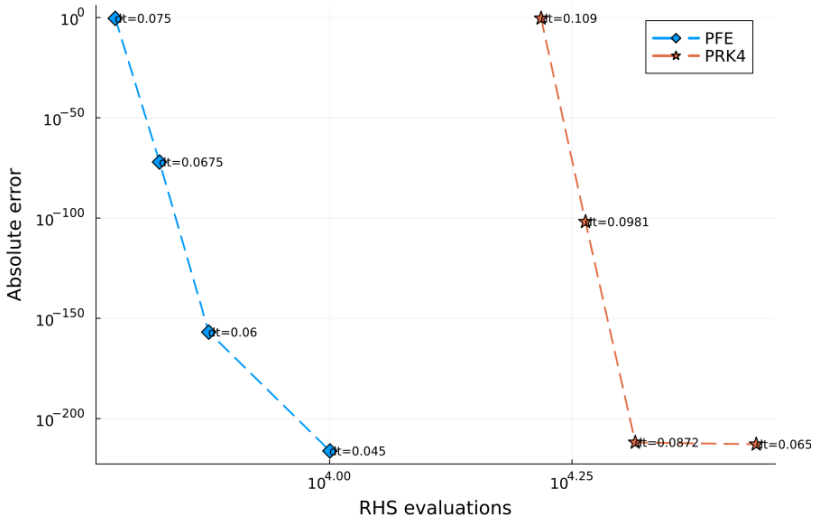


Figure 4.18: Work-precision diagram for PFEK8 and PRK4K8 at $T = 50$, showing the absolute error against the number of right-hand side evaluations for different outer step sizes.

$\Delta t_{\max}/\#\text{RHS}$. The PI methods also give the smallest errors at the longer final time. Reducing the outer step size further decreases their errors.

5 Conclusion and outlook

In this thesis, Projective Integration methods were compared with explicit Runge-Kutta methods whose stability polynomials were optimized for the spectrum of the problem. The comparison was carried out for an isolated two-scale spectrum, two-scale cluster spectra with different radii, a semi-discrete hyperbolic moment model, and a generalized two-scale system. The methods were compared using their maximal stable step sizes, the step size per right-hand side evaluation, and their numerical errors.

The results show that the comparison depends strongly on the underlying spectrum and on the order of the methods. For the isolated spectrum and the cluster spectra, the largest step size per right-hand side evaluation is often obtained with the first- or second-order optimized Runge-Kutta methods. Even for $r_{\text{fast}} = 50$, the first-order optimized Runge-Kutta method still gives the largest value, see Section 4.4. For the semi-discrete moment model and the generalized two-scale system, where the spectral clusters are much wider, PFE gives the largest value. For the optimized Runge-Kutta methods, increasing the number of stages did not always improve the step size per right-hand side evaluation. In several cases, this value remained nearly unchanged or decreased for larger stage numbers.

At the maximal stable step sizes, the selected optimized Runge-Kutta methods gave smaller errors in most of the error comparisons. For the generalized two-scale system, however, the PI methods gave smaller errors at the longer final time, see Section 4.6. An important advantage of the PI methods is that their outer step size can be reduced if higher accuracy is required. This lowers the error, but requires more right-hand side evaluations. For the optimized Runge-Kutta methods, the stable step sizes did not form a connected interval, so the step size could not simply be reduced. Therefore, only full time steps with the selected step size were used.

The numerical experiments also showed that the PolyOpt results could depend on the spectral sampling. In particular, for the isolated spectrum and the cluster spectra with smaller radii, the bisection sometimes missed stable intervals at larger step sizes. In these cases, an additional grid-based line search was required, see Sections 4.1.3 and 4.3.3.

Chebyshev interpolation gave the most consistent stability results. However, the results depended on how the interpolation points were distributed, and different distributions worked best for different spectra. The standard Chebyshev basis avoids this additional choice and reached larger stage numbers for the semi-discrete moment model and the

generalized two-scale system. For the higher-stage interpolation-based tests, the derivative-based order conditions gave better results than the Vandermonde formulation. They should therefore be preferred when Chebyshev interpolation is used.

The interpolation-based extensions used in this thesis were implemented in RK-Opt and submitted as a pull request to the RK-Opt repository.

Several aspects could be considered in future work. The optimization in this thesis is based on the eigenvalues of the problem. For nonnormal systems, however, the eigenvalues alone may not fully describe the stability behavior. Future work could therefore investigate the optimization of stability polynomials based on pseudospectra [KA12].

The experiments also showed that the construction of the Runge-Kutta methods depends on the distribution of the interpolation points. Different distributions gave better results for different spectra. Future work could therefore investigate whether the point distribution can be derived from the spectrum or whether general selection rules can be developed.

Accuracy could also be included more directly in the optimization. The largest stable step size did not always lead to the smallest numerical error. Future work could therefore optimize the methods for a given accuracy or computational cost, as also considered in [PKD13].

6 Appendix

6.1 Flux Jacobian of the hyperbolic moment model

The flux Jacobian $A(\boldsymbol{w})$ of the hyperbolic moment model is given by

$$A(\boldsymbol{w}) = \begin{pmatrix} u & \rho & & & & & & & \\ \frac{\theta}{\rho} & u & 1 & & & & & & \\ & 2\theta & u & \frac{6}{\rho} & & & & & \\ & 4f_3 & \frac{\rho\theta}{2} & u & 4 & & & & \\ -\frac{\theta f_3}{\rho} & 5f_4 & \frac{3f_3}{2} & \theta & u & 5 & & & \\ \vdots & \vdots & & \vdots & & \ddots & \ddots & & \\ -\frac{\theta f_{M-2}}{\rho} & Mf_{M-1} & \frac{(M-2)f_{M-2}+\theta f_{M-4}}{2} & -\frac{M(M+1)f_M}{2\theta} & -\frac{3f_{M-3}}{\rho} & \theta & u & M & \\ -\frac{\theta f_{M-1}}{\rho} & (M+1)f_M & -f_{M-1}+\frac{\theta f_{M-3}}{2} & \frac{3(M+1)f_M}{\rho\theta} & -\frac{3f_{M-2}}{\rho} & & \theta & u & \end{pmatrix} \tag{6.1}$$

6.2 Butcher tableau of the PRK4 method

For PRK4($K=1$), that is, for $K+1=2$ inner steps, the Butcher tableau is given by

$$\begin{array}{c|cccccccc} 0 & 0 & & & & & & & \\ \gamma & \gamma & 0 & & & & & & \\ \frac{1}{3} & \gamma & \frac{1}{3}-\gamma & 0 & & & & & \\ \frac{1}{3}+\gamma & \gamma & \frac{1}{3}-\gamma & \gamma & 0 & & & & \\ \frac{2}{3} & \gamma & -\frac{1}{3}+2\gamma & 0 & 1-3\gamma & 0 & & & \\ \frac{2}{3}+\gamma & \gamma & -\frac{1}{3}+2\gamma & 0 & 1-3\gamma & \gamma & 0 & & \\ 1 & \gamma & 1-\gamma & 0 & -1+2\gamma & 0 & 1-2\gamma & 0 & \\ 1+\gamma & \gamma & 1-\gamma & 0 & -1+2\gamma & 0 & 1-2\gamma & \gamma & 0 \\ \hline & \gamma & \frac{1}{8}+\frac{3}{4}\gamma & 0 & \frac{3}{8}-\frac{6}{8}\gamma & 0 & \frac{3}{8}-\frac{6}{8}\gamma & 0 & \frac{1}{8}-\frac{1}{4}\gamma \end{array} \tag{6.2}$$

where

$$\gamma = \frac{\delta t}{\Delta t}.$$

6.3 Use of artificial intelligence (Nutzung KI-Tools)

During the preparation of this thesis, ChatGPT by OpenAI was used to correct grammar and spelling errors in selected passages, to explain programming syntax, and to assist with debugging by interpreting error messages and identifying possible sources of errors in the code. All suggestions were checked by the author before use.

6.4 GitHub repository

The MATLAB and Julia code used for the numerical experiments, as well as the code used to generate the figures and tables presented in this thesis, is available in the following GitHub repository:

`https://github.com/chrisba23/RK-method-optimization.git`

Bibliography

[AK22] Amrita Amrita and Julian Koellermeier. "Projective Integration for Hyperbolic Shallow Water Moment Equations". In: *Axioms* 11.5 (2022). DOI: 10.3390/axioms11050235 (cit. on p. 6).

[BGK54] P. L. Bhatnagar, E. P. Gross, and M. Krook. "A Model for Collision Processes in Gases. I. Small Amplitude Processes in Charged and Neutral One-Component Systems". In: *Phys. Rev.* 94 (3 1954), pp. 511–525. DOI: 10.1103/PhysRev.94.511 (cit. on p. 69).

[GB08] Michael C. Grant and Stephen P. Boyd. "Graph Implementations for Nonsmooth Convex Programs". In: *Recent Advances in Learning and Control*. Ed. by Vincent D. Blondel, Stephen P. Boyd, and Hidenori Kimura. Vol. 371. Lecture Notes in Control and Information Sciences. London: Springer, 2008, pp. 95–110 (cit. on p. 28).

[GB14] Michael Grant and Stephen Boyd. *CVX: Matlab Software for Disciplined Convex Programming, version 2.1*. https://cvxr.com/cvx. Mar. 2014 (cit. on p. 28).

[GK03] C. W. Gear and Ioannis G. Kevrekidis. "Projective Methods for Stiff Differential Equations: Problems with Gaps in Their Eigenvalue Spectrum". In: *SIAM Journal on Scientific Computing* 24.4 (2003), pp. 1091–1106. DOI: 10.1137/S1064827501388157 (cit. on pp. 7, 10).

[HNW08] E. Hairer, S.P. Nørsett, and G. Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*. Springer Series in Computational Mathematics. Springer Berlin Heidelberg, 2008. ISBN: 9783540566700. URL: https://books.google.de/books?id=F93u7VcSRyYC (cit. on pp. 9, 10).

[KA12] David Ketcheson and Aron Ahmadia. "Optimal stability polynomials for numerical integration of initial value problems". In: *Communications in Applied Mathematics and Computational Science* 7.2 (Dec. 2012), pp. 247–271. ISSN: 1559-3940. DOI: 10.2140/camcos.2012.7.247 (cit. on pp. 7, 16–18, 22, 23, 32, 85, 89).

[Ket+20] David I. Ketcheson et al. "RK-Opt: A package for the design of numerical ODE solvers". In: *Journal of Open Source Software* 5.54 (2020), p. 2514. DOI: 10.21105/joss.02514 (cit. on p. 28).

[Ket10] David I. Ketcheson. “Runge–Kutta methods with minimum storage implementations”. In: *Journal of Computational Physics* 229.5 (2010), pp. 1763–1773. ISSN: 0021-9991. DOI: 10.1016/j.jcp.2009.11.006 (cit. on pp. 7, 16, 19, 20).

[KS21] Julian Koellermeier and Giovanni Samaey. “Projective integration schemes for hyperbolic moment equations”. In: *Kinetic and Related Models* 14.2 (2021), pp. 353–387. DOI: 10.3934/krm.2021008 (cit. on p. 6).

[KS22] Julian Koellermeier and Giovanni Samaey. “Spatially Adaptive Projective Integration Schemes For Stiff Hyperbolic Balance Laws With Spectral Gaps”. In: *The SMAI Journal of computational mathematics* 8 (2022), pp. 295–325. DOI: 10.5802/smai-jcm.88 (cit. on pp. 12, 69, 70).

[KS25] Julian Koellermeier and Giovanni Samaey. “Projective integration methods in the Runge–Kutta framework and the extension to adaptivity in time”. In: *Journal of Computational and Applied Mathematics* 454 (2025), p. 116147. ISSN: 0377-0427. DOI: 10.1016/j.cam.2024.116147 (cit. on pp. 7, 9, 10, 13, 15).

[LLS16] Pauline Lafitte, Annelies Lejon, and Giovanni Samaey. “A High-Order Asymptotic-Preserving Scheme for Kinetic Equations Using Projective Integration”. In: *SIAM Journal on Numerical Analysis* 54.1 (2016), pp. 1–33. DOI: 10.1137/140966708 (cit. on p. 13).

[LS12] Pauline Lafitte and Giovanni Samaey. “Asymptotic-preserving Projective Integration Schemes for Kinetic Equations in the Diffusion Limit”. In: *SIAM Journal on Scientific Computing* 34.2 (2012), A579–A602. DOI: 10.1137/100795954 (cit. on p. 6).

[Pér19] Miguel Pérez-Saborid. *A simple MATLAB program to compute differentiation matrices for arbitrary meshes via Lagrangian interpolation*. 2019. arXiv: 1910.13256 [math.NA]. URL: https://arxiv.org/abs/1910.13256 (cit. on p. 26).

[PKD13] M. Parsani, David I. Ketcheson, and W. Deconinck. “Optimized Explicit Runge–Kutta Schemes for the Spectral Difference Method Applied to Wave Propagation Problems”. In: *SIAM Journal on Scientific Computing* 35.2 (2013), A957–A986. DOI: 10.1137/120885899 (cit. on pp. 7, 16, 21, 89).

[PR05] Lorenzo Pareschi and Giovanni Russo. “Implicit-Explicit Runge-Kutta Schemes and Applications to Hyperbolic Systems with Relaxation”. In: *Journal of Scientific Computing* 25.1-2 (2005), pp. 129–155. DOI: 10.1007/s10915-004-4636-4 (cit. on p. 6).

[Ran+22] Hendrik Ranocha et al. “Optimized Runge–Kutta Methods with Automatic Step Size Control for Compressible Computational Fluid Dynamics”. In: *Communications on Applied Mathematics and Computation* 4.4 (2022), pp. 1191–1228. DOI: 10.1007/s42967-021-00159-w.

[Ran25] Hendrik Ranocha. *Vorlesungsnotizen "Grundlagen der Numerik"*. Lecture notes, Johannes Gutenberg University Mainz, version from 2 July 2025. 2025.

[SO88] Chi-Wang Shu and Stanley Osher. "Efficient implementation of essentially non-oscillatory shock-capturing schemes". In: *Journal of Computational Physics* 77.2 (1988), pp. 439–471. DOI: https://doi.org/10.1016/0021-9991(88)90177-5 (cit. on p. 19).

[Stu99] Jos F. Sturm. "Using SeDuMi 1.02, A Matlab toolbox for optimization over symmetric cones". In: *Optimization Methods and Software* 11.1–4 (1999), pp. 625–653. DOI: 10.1080/10556789908805766 (cit. on p. 28).

[Tre19] Lloyd N. Trefethen. *Approximation Theory and Approximation Practice, Extended Edition*. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2019. DOI: 10.1137/1.9781611975949 (cit. on p. 24).

[TTT03] Reha H. Tütüncü, Kim-Chuan Toh, and Michael J. Todd. "Solving semidefinite-quadratic-linear programs using SDPT3". In: *Mathematical Programming* 95.2 (2003), pp. 189–217. DOI: 10.1007/s10107-002-0347-5 (cit. on p. 28).

[Wil80] J.H Williamson. "Low-storage Runge-Kutta schemes". In: *Journal of Computational Physics* 35.1 (1980), pp. 48–56. DOI: https://doi.org/10.1016/0021-9991(80)90033-9 (cit. on p. 19).